

Neural Networks: Optimization

Recap

- Neural networks are universal approximators
- We must *train* them to approximate any function
- Networks are trained to minimize total “error” on a training set
 - We do so through empirical risk minimization
- We use variants of gradient descent to do so
 - Gradients are computed through backpropagation

Recap

- Vanilla gradient descent may be too slow or unstable
- Better convergence can be obtained through
 - Second order methods that normalize the variation across dimensions
 - Adaptive or decaying learning rates that can improve convergence
 - Methods like Rprop that decouple the dimensions can improve convergence
 - *Momentum methods which emphasize directions of steady improvement and deemphasize unstable directions*

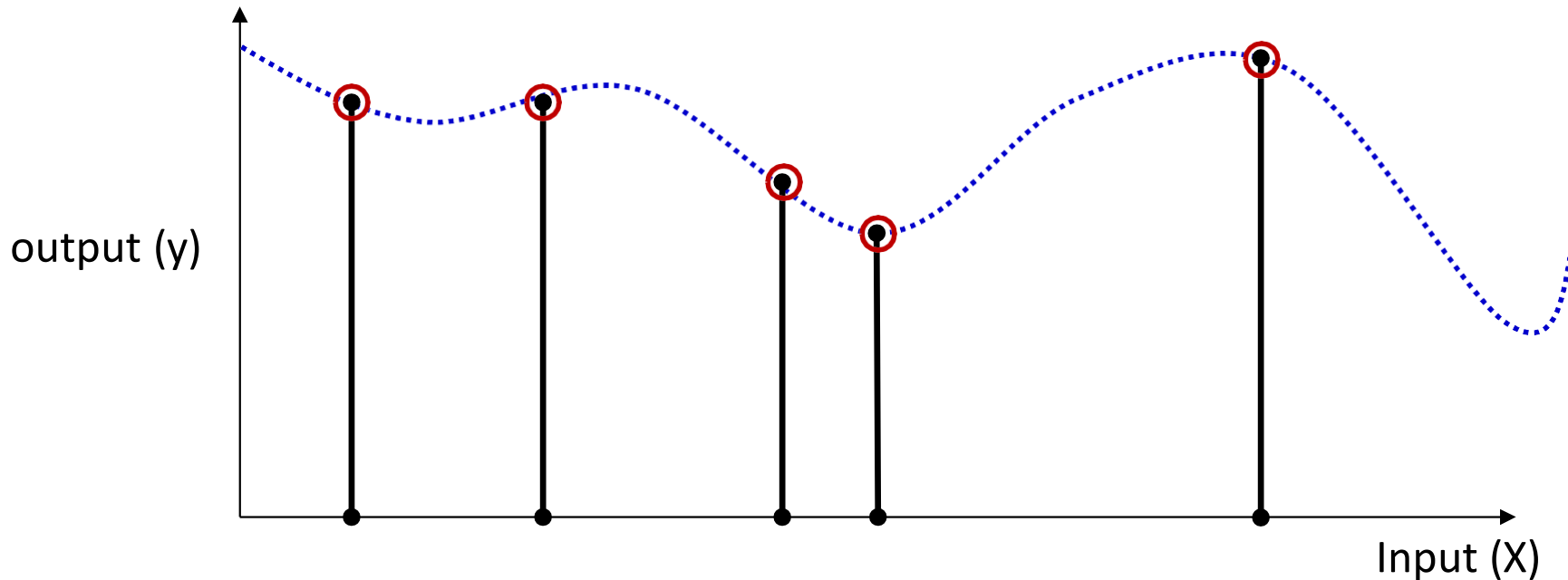
Moving on...

- Incremental updates
- Revisiting “trend” algorithms
- Generalization
- Tricks of the trade
 - Divergences..
 - Activations
 - Normalizations

Moving on: Topics for the day

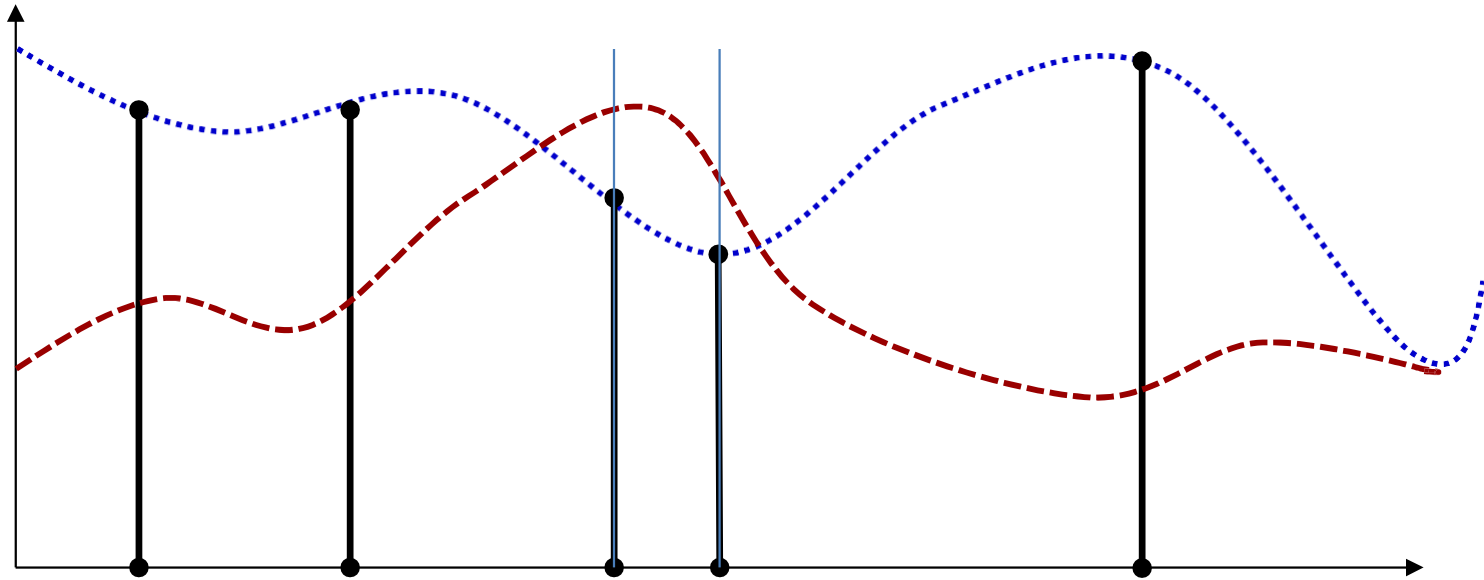
- Incremental updates
- Revisiting “trend” algorithms
- Generalization
- Tricks of the trade
 - Divergences..
 - Activations
 - Normalizations

The training formulation



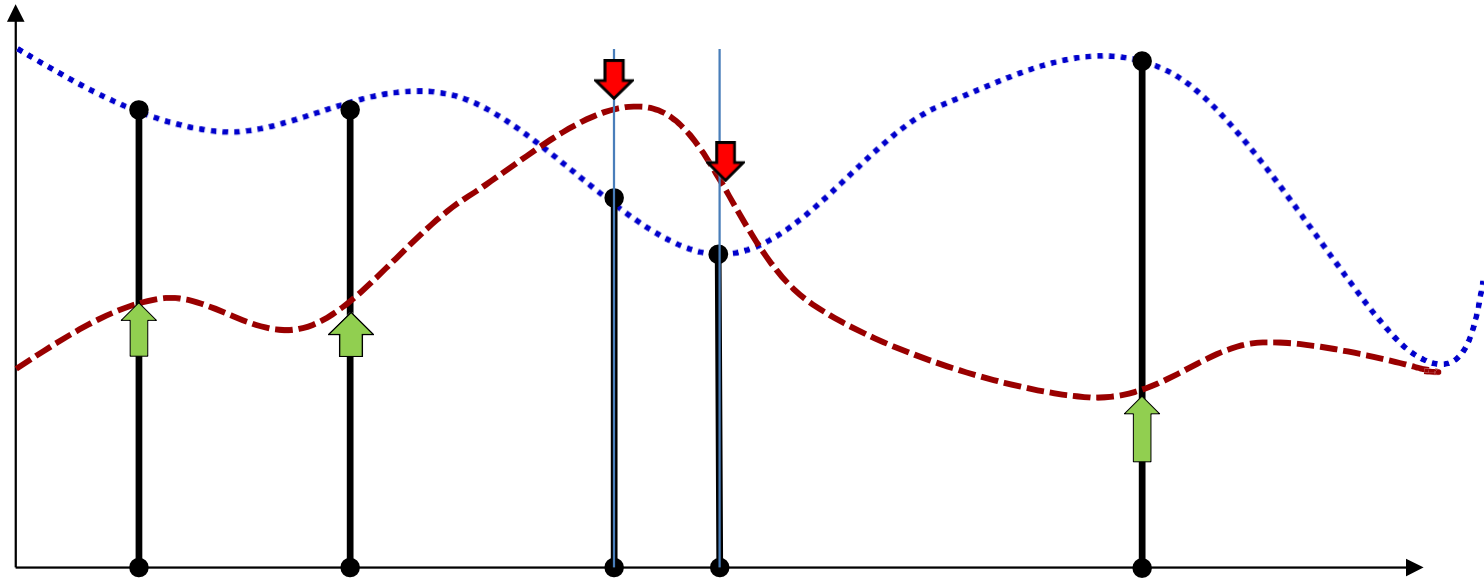
- Given input output pairs at a number of locations, estimate the entire function

Gradient descent



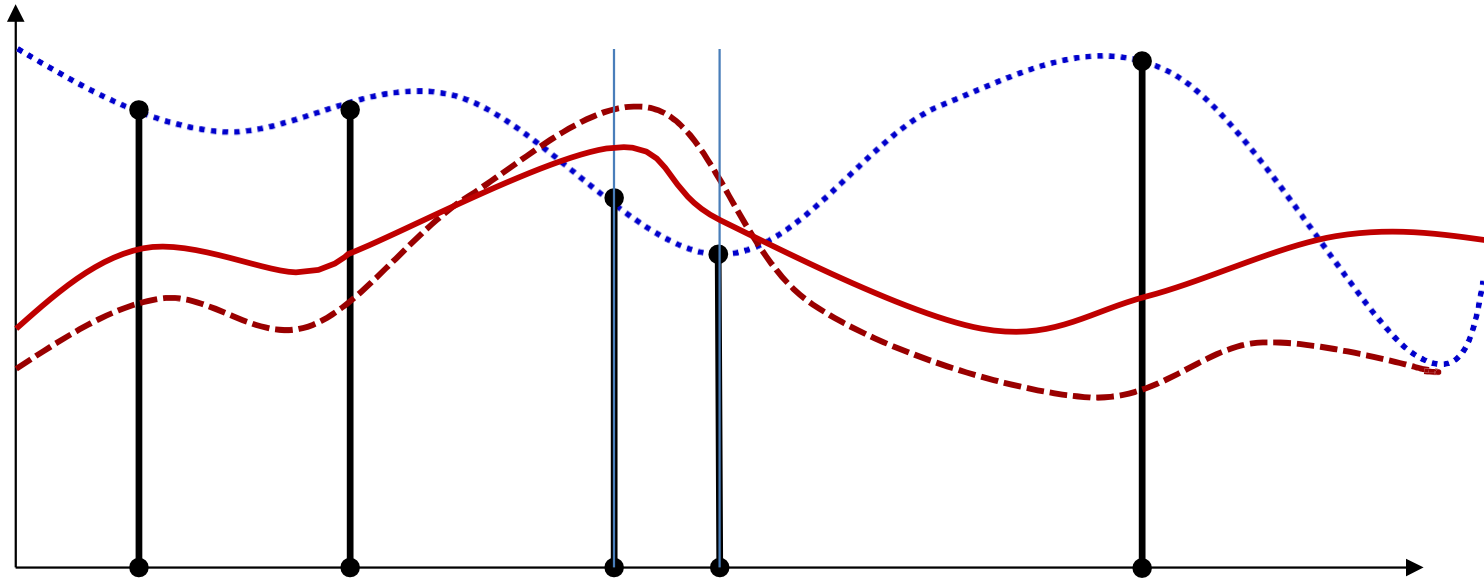
- Start with an initial function

Gradient descent



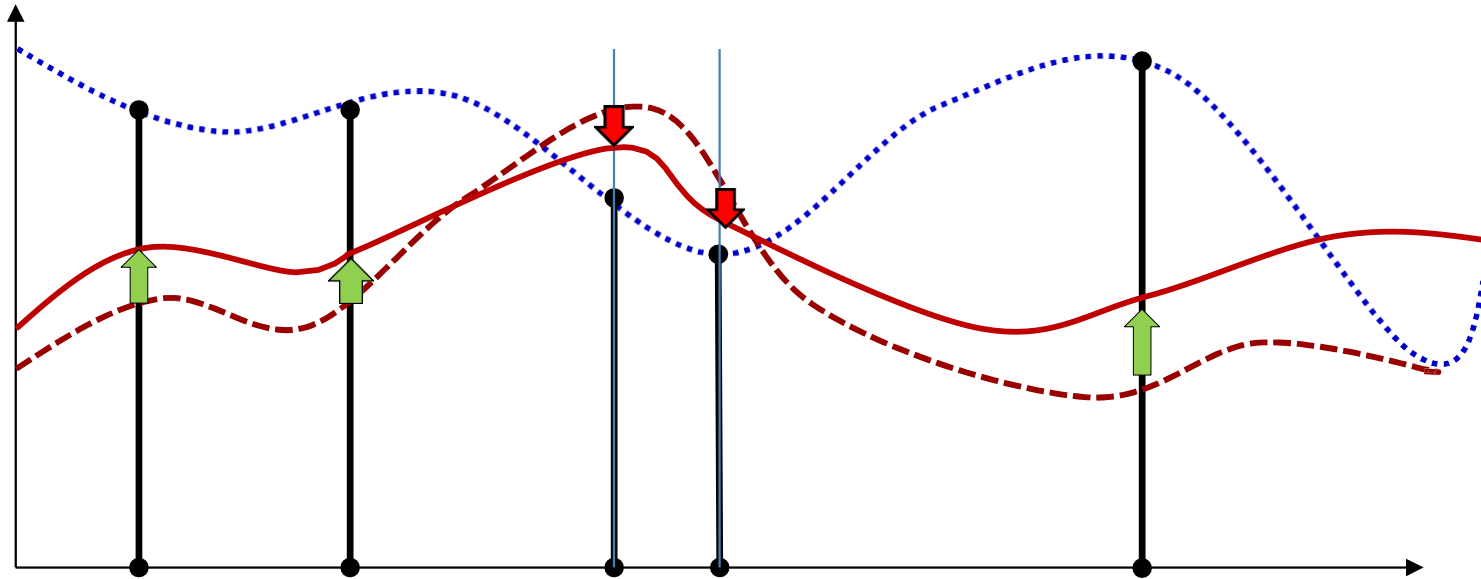
- Start with an initial function
- Adjust its value at *all* points to make the outputs closer to the required value
 - Gradient descent adjusts parameters to adjust the function value at *all* points
 - Repeat this iteratively until we get arbitrarily close to the target function at the training points

Gradient descent



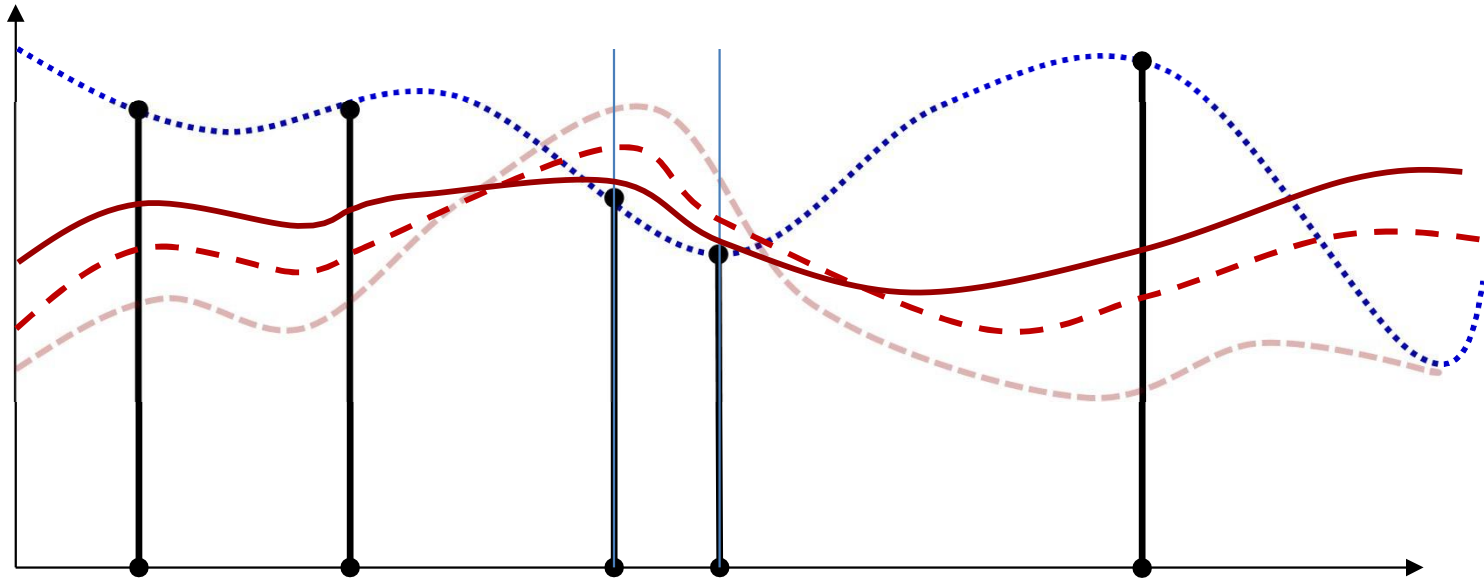
- Start with an initial function
- Adjust its value at *all* points to make the outputs closer to the required value
 - Gradient descent adjusts parameters to adjust the function value at *all* points
 - Repeat this iteratively until we get arbitrarily close to the target function at the training points

Gradient descent



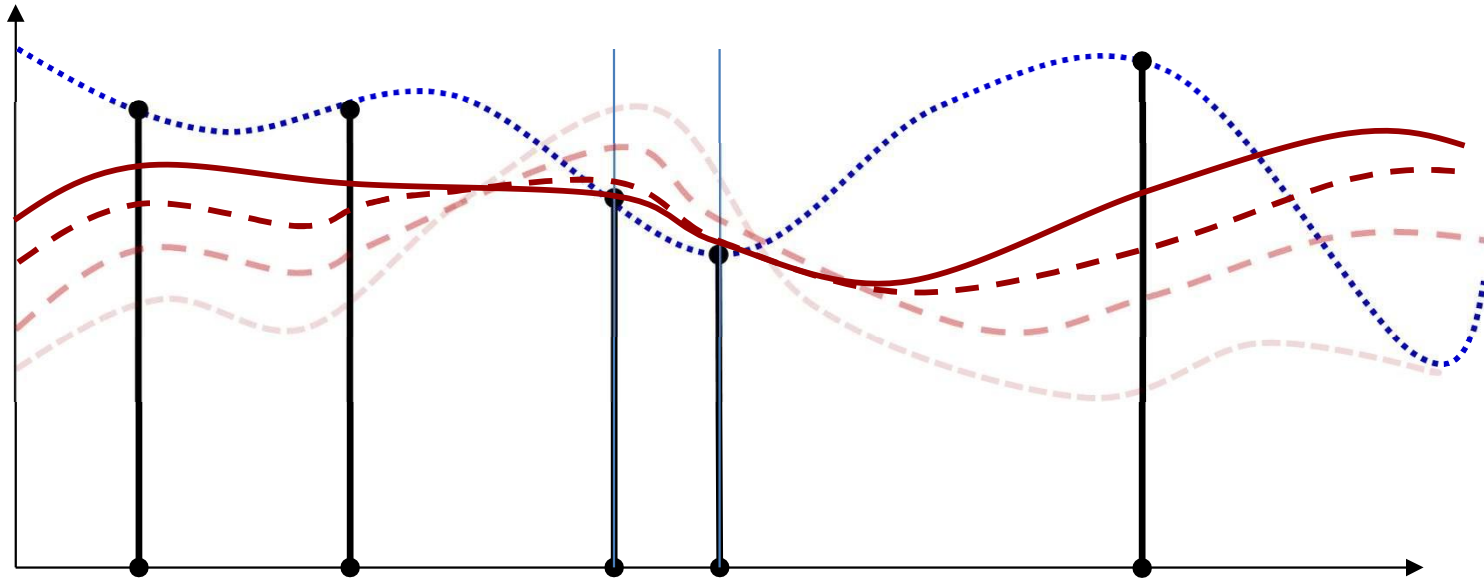
- Start with an initial function
- Adjust its value at *all* points to make the outputs closer to the required value
 - Gradient descent adjusts parameters to adjust the function value at *all* points
 - Repeat this iteratively until we get arbitrarily close to the target function at the training points

Gradient descent



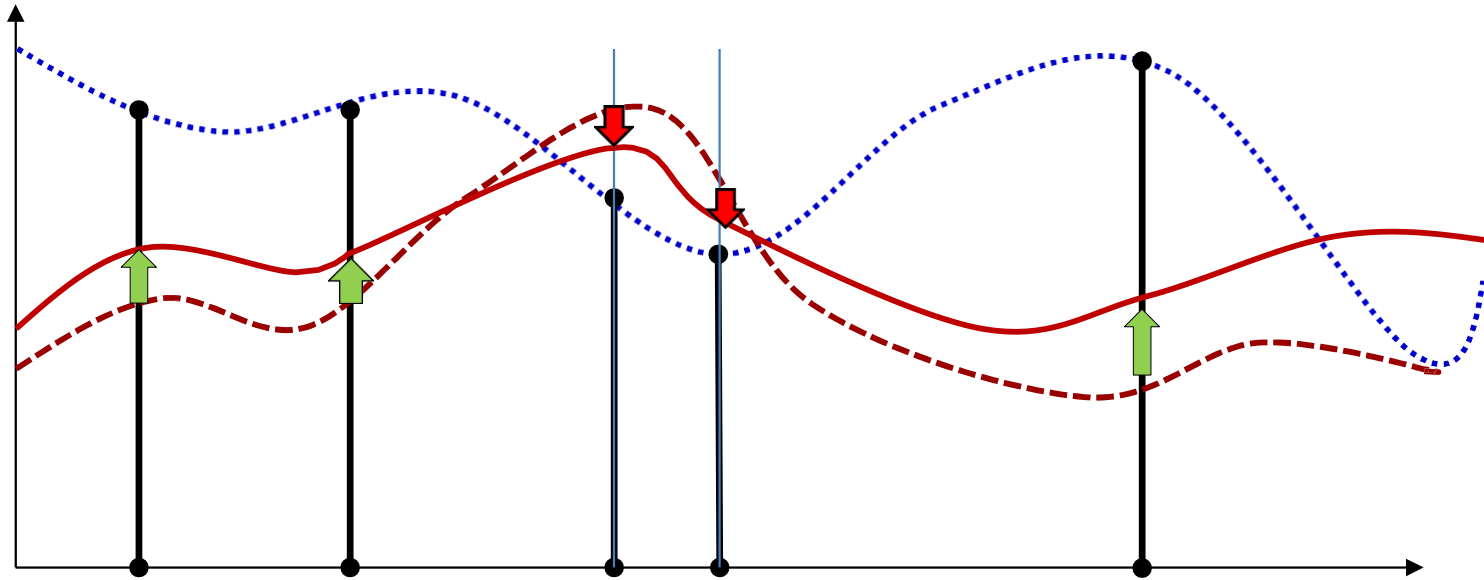
- Start with an initial function
- Adjust its value at *all* points to make the outputs closer to the required value
 - Gradient descent adjusts parameters to adjust the function value at *all* points
 - Repeat this iteratively until we get arbitrarily close to the target function at the training points

Gradient descent



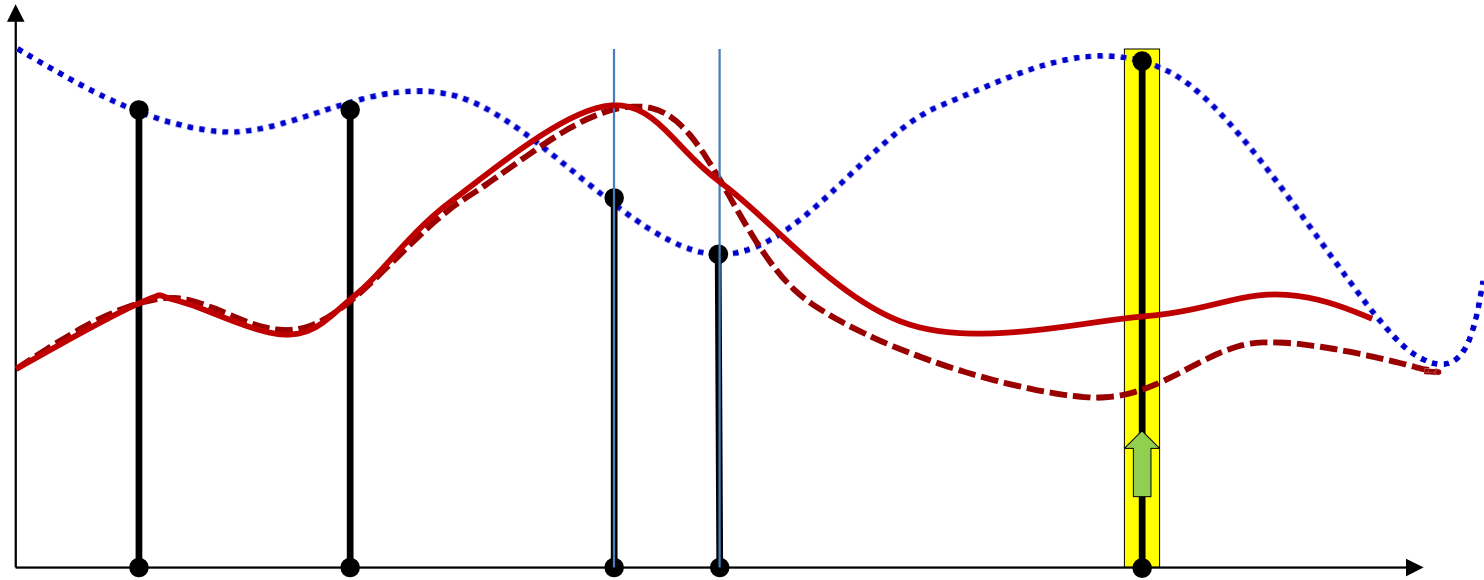
- Start with an initial function
- Adjust its value at *all* points to make the outputs closer to the required value
 - Gradient descent adjusts parameters to adjust the function value at *all* points
 - Repeat this iteratively until we get arbitrarily close to the target function at the training points

Effect of number of samples



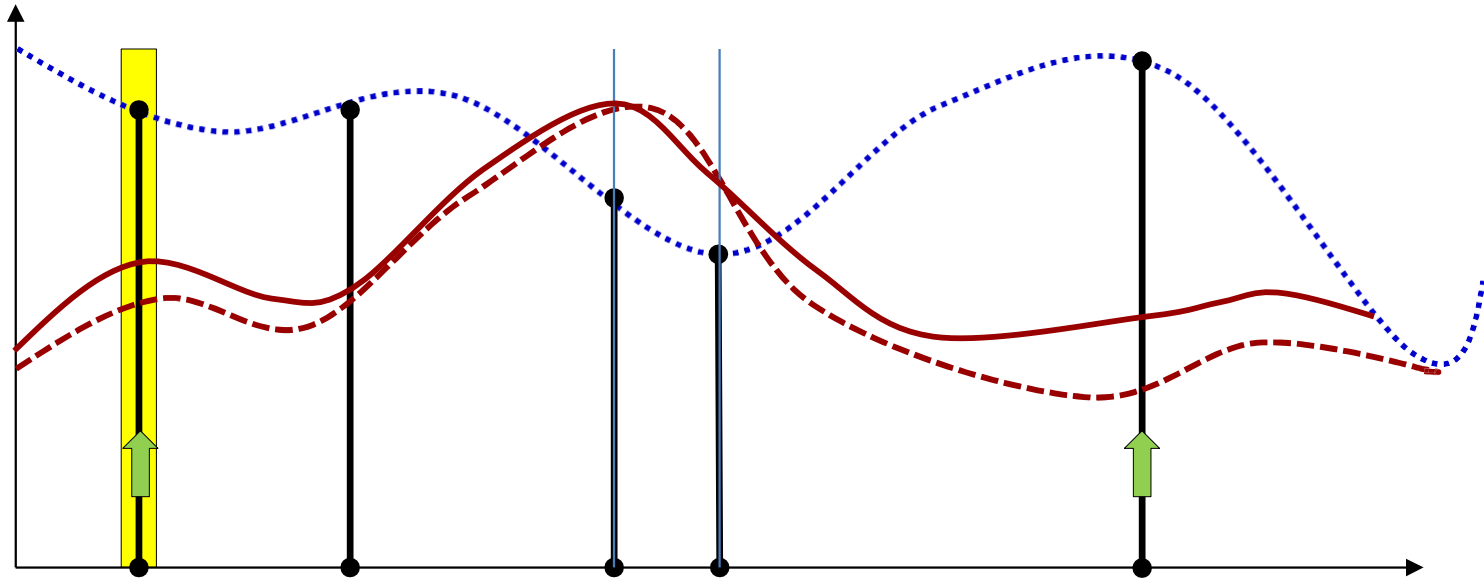
- Problem with conventional gradient descent: we try to simultaneously adjust the function at *all* training points
 - We must process *all* training points before making a single adjustment
 - **“Batch”** update

Alternative: Incremental update



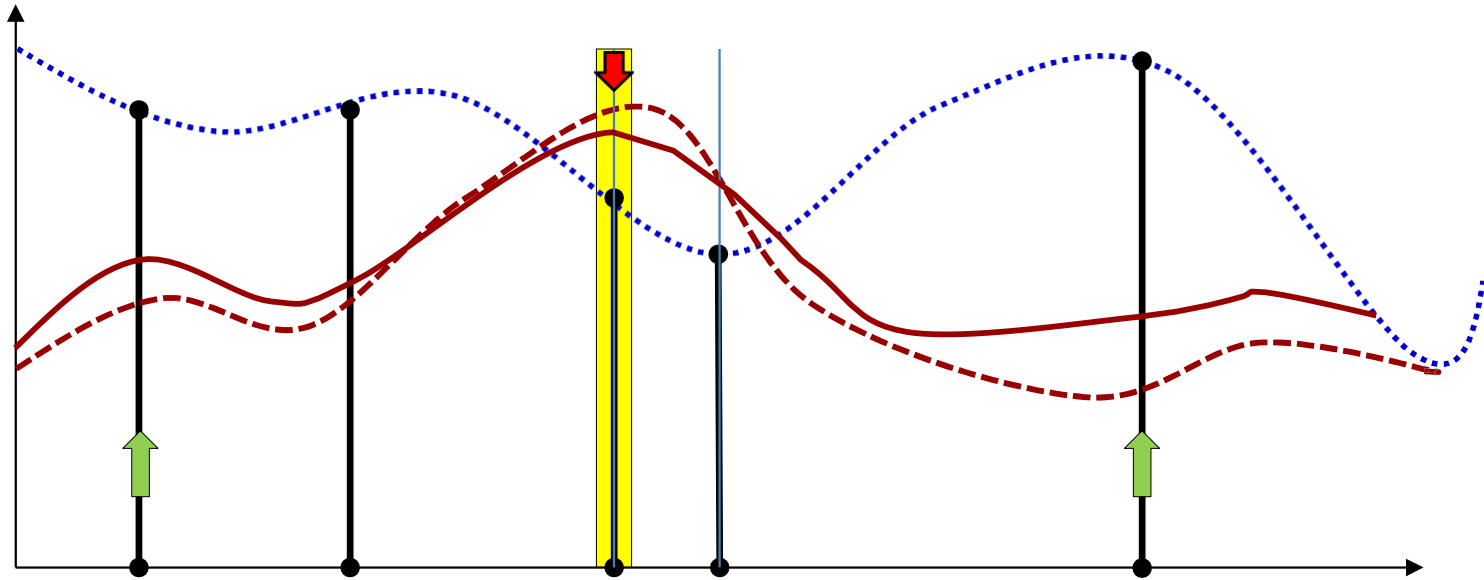
- Alternative: adjust the function at one training point at a time
 - Keep adjustments small

Alternative: Incremental update



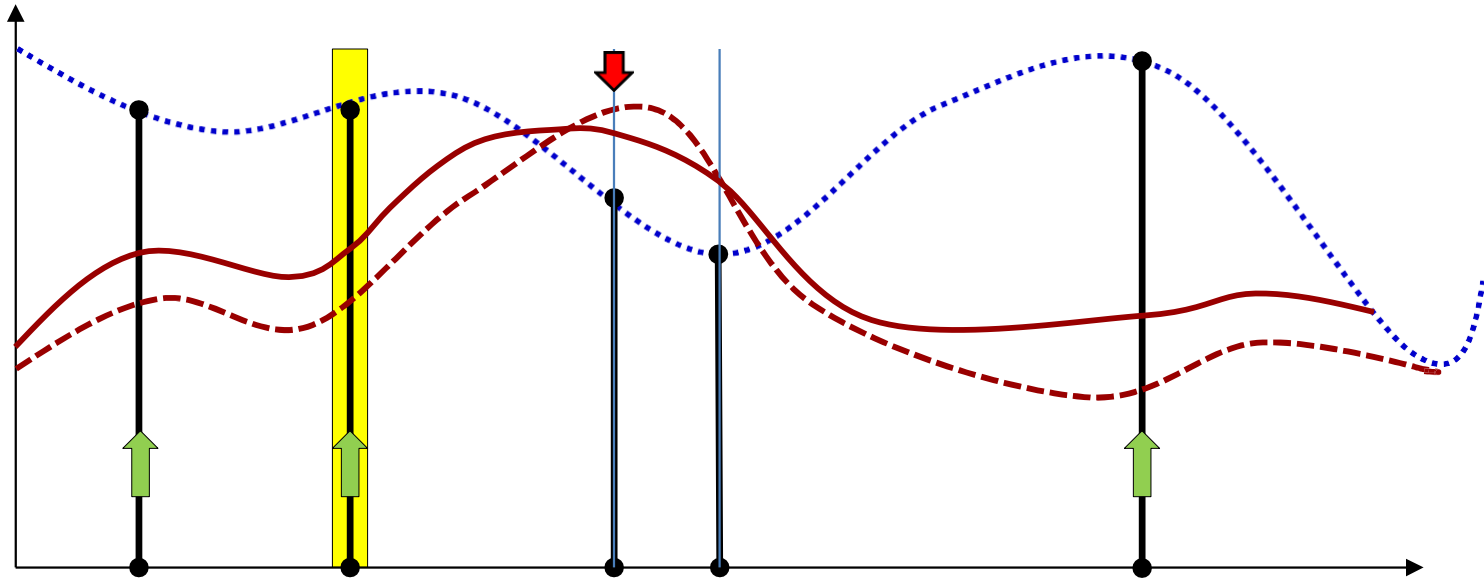
- Alternative: adjust the function at one training point at a time
 - Keep adjustments small

Alternative: Incremental update



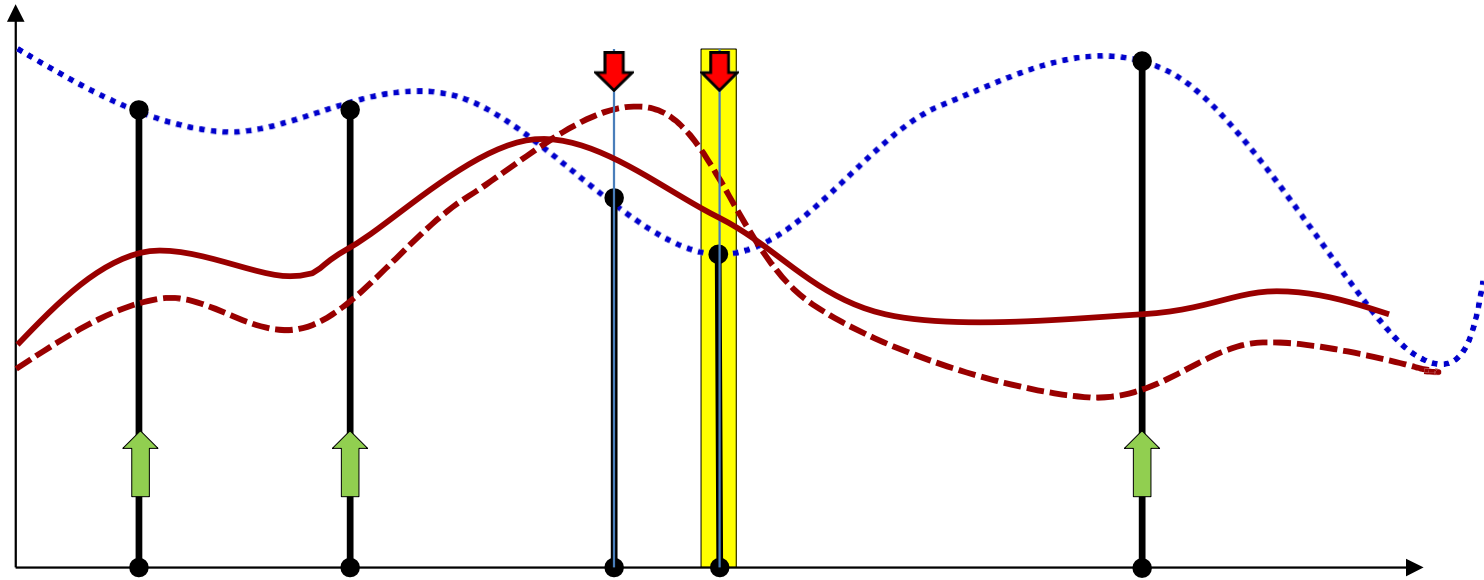
- Alternative: adjust the function at one training point at a time
 - Keep adjustments small

Alternative: Incremental update



- Alternative: adjust the function at one training point at a time
 - Keep adjustments small

Alternative: Incremental update



- Alternative: adjust the function at one training point at a time
 - Keep adjustments small
 - Eventually, when we have processed all the training points, we will have adjusted the entire function
 - With *greater* overall adjustment than we would if we made a single “Batch” update

Incremental Update

- Given $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$
- Initialize all weights $W_1, W_2, \dots, W_K$
- Do:
 - For all $t = 1:T$
 - For every layer k :
 - Compute $\nabla_{W_k} \text{Div}(Y_t, d_t)$
 - Update
$$W_k = W_k - \eta \nabla_{W_k} \text{Div}(Y_t, d_t)^T$$
- Until *Loss* has converged

Incremental Updates

- The iterations can make multiple passes over the data
- A single pass through the entire training data is called an “epoch”
 - An epoch over a training set with T samples results in T updates of parameters

Incremental Update

- Given $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$
- Initialize all weights $W_1, W_2, \dots, W_K$

- Do:  

One epoch



– For all $t = 1:T$

- For every layer k :

– Compute $\nabla_{W_k} \text{Div}(Y_t, d_t)$

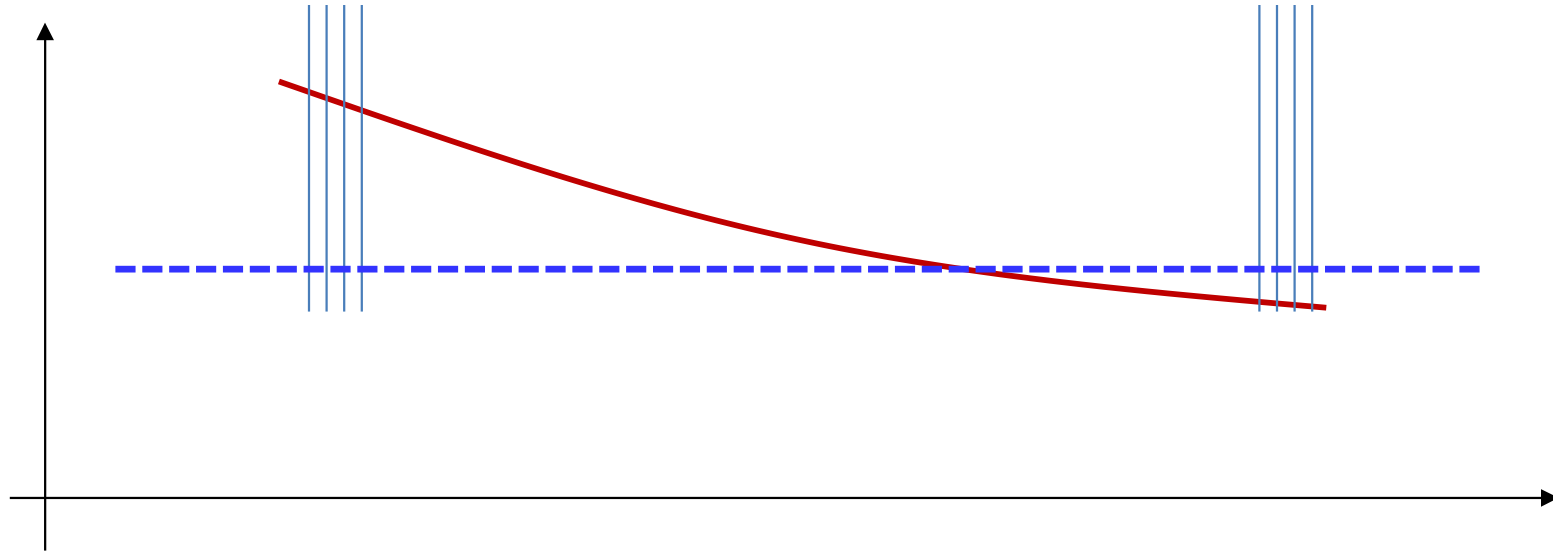
– Update

$$W_k = W_k - \eta \nabla_{W_k} \text{Div}(Y_t, d_t)^T$$

One update

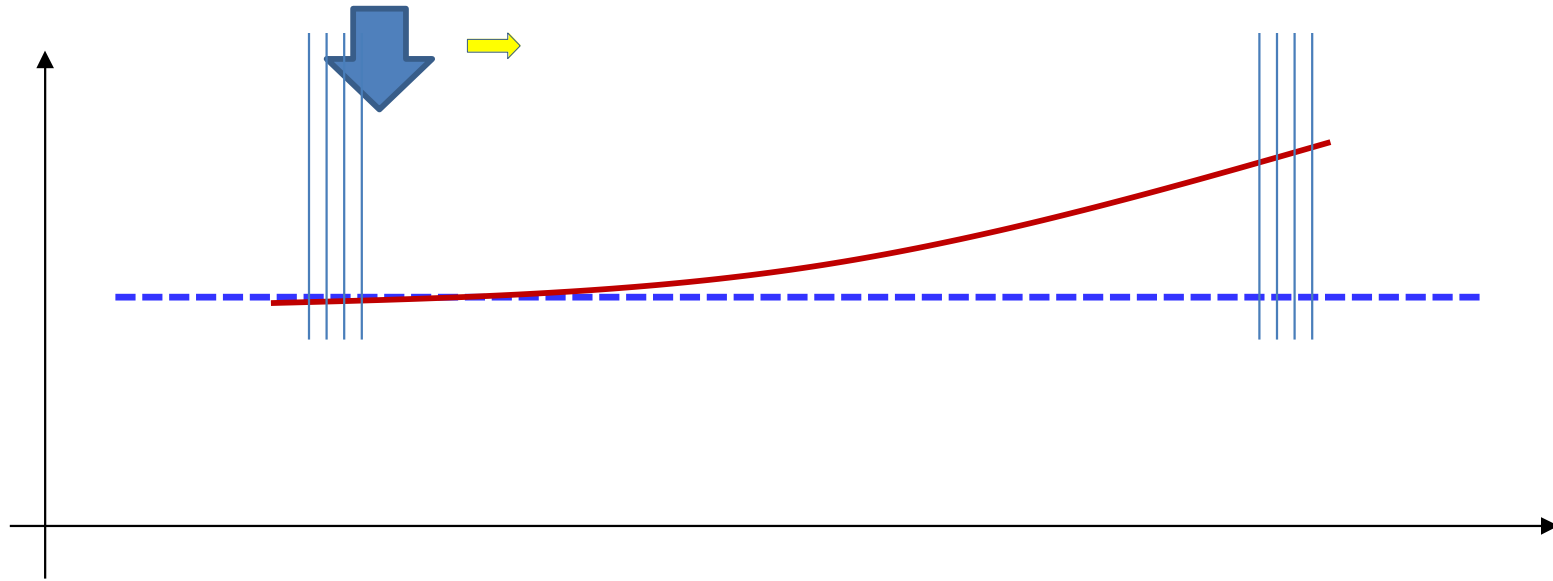
- Until *Loss* has converged

Caveats: order of presentation



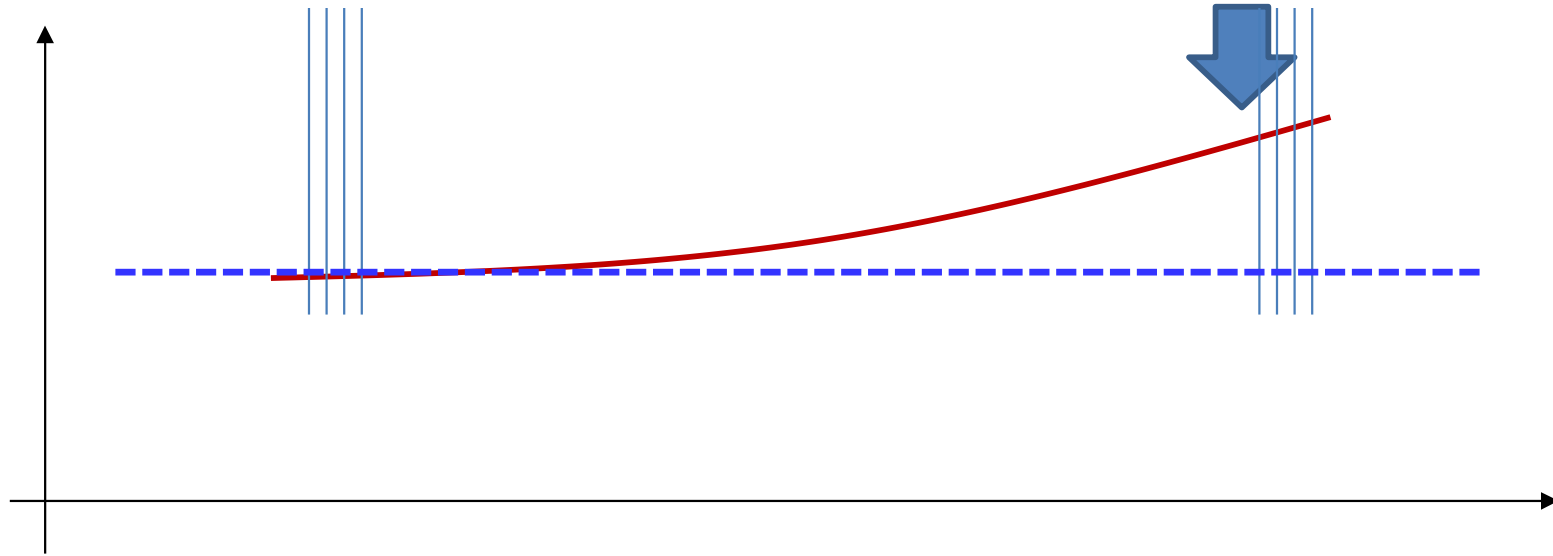
- If we loop through the samples in the same order, we may get *cyclic* behavior

Caveats: order of presentation



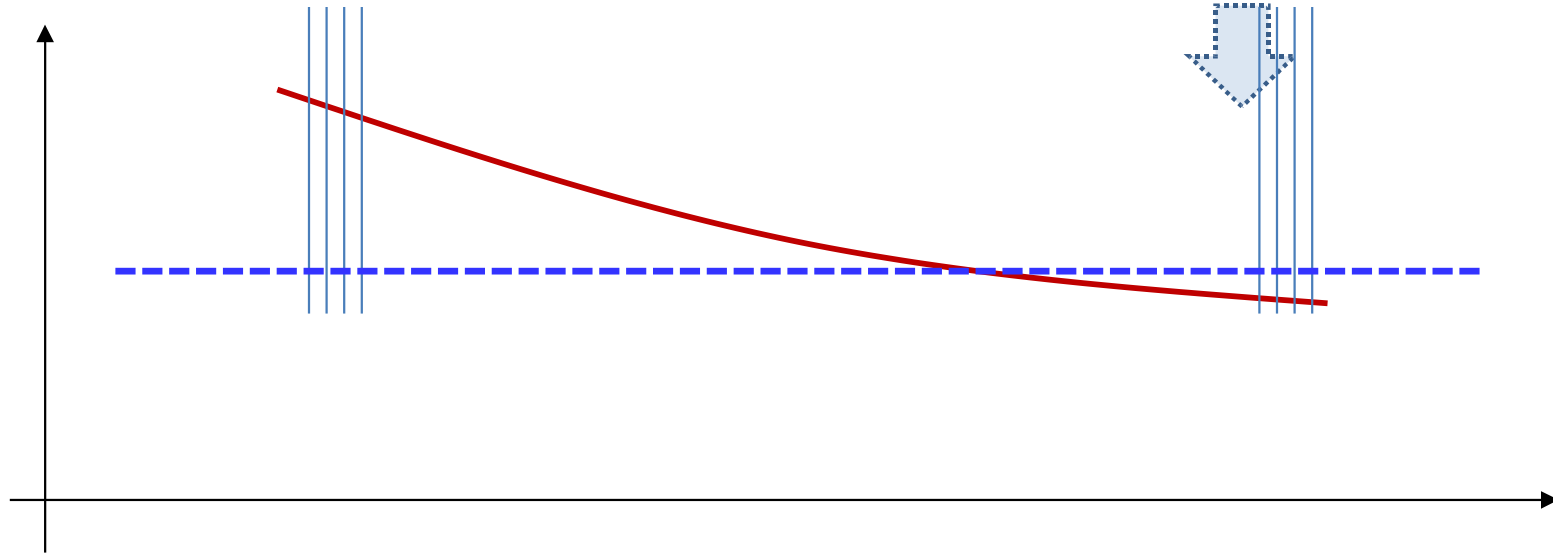
- If we loop through the samples in the same order, we may get *cyclic* behavior

Caveats: order of presentation



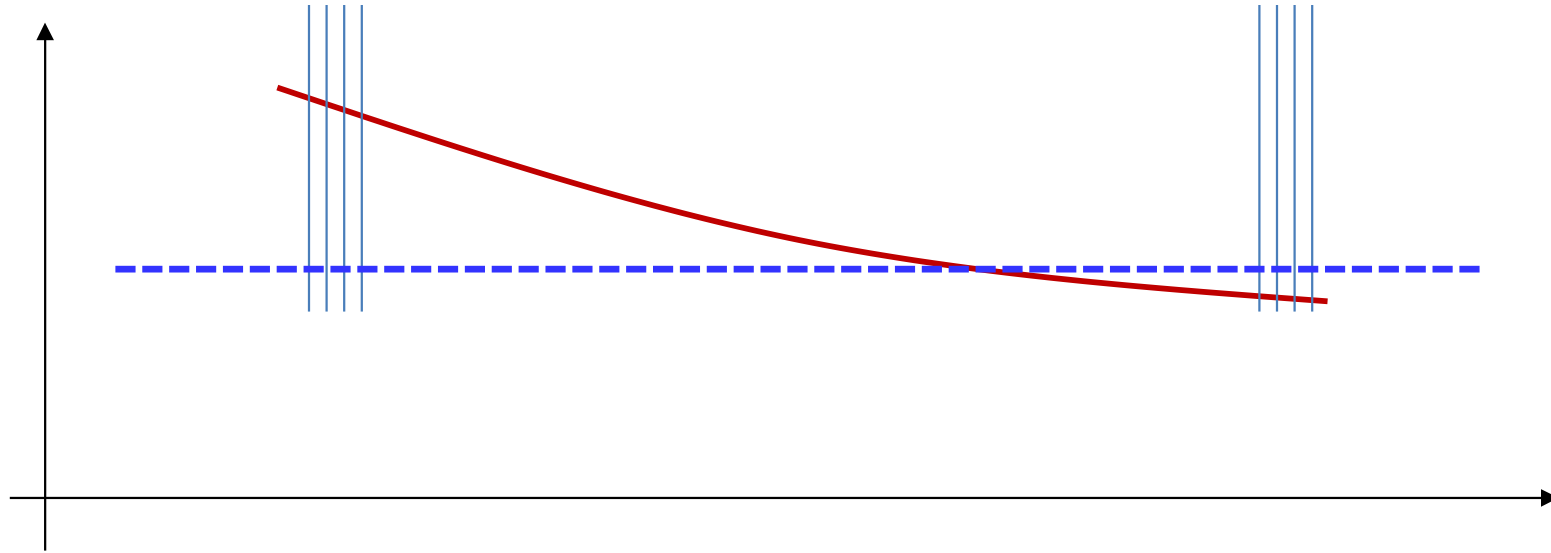
- If we loop through the samples in the same order, we may get *cyclic* behavior

Caveats: order of presentation



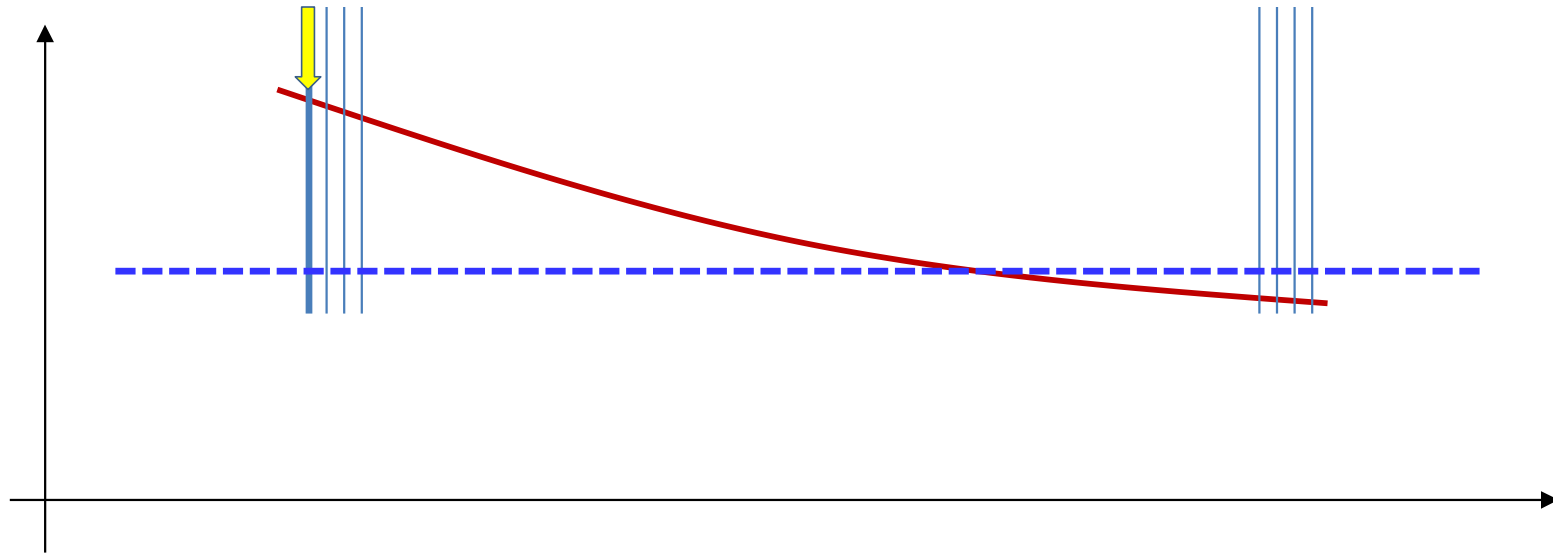
- If we loop through the samples in the same order, we may get *cyclic* behavior

Caveats: order of presentation



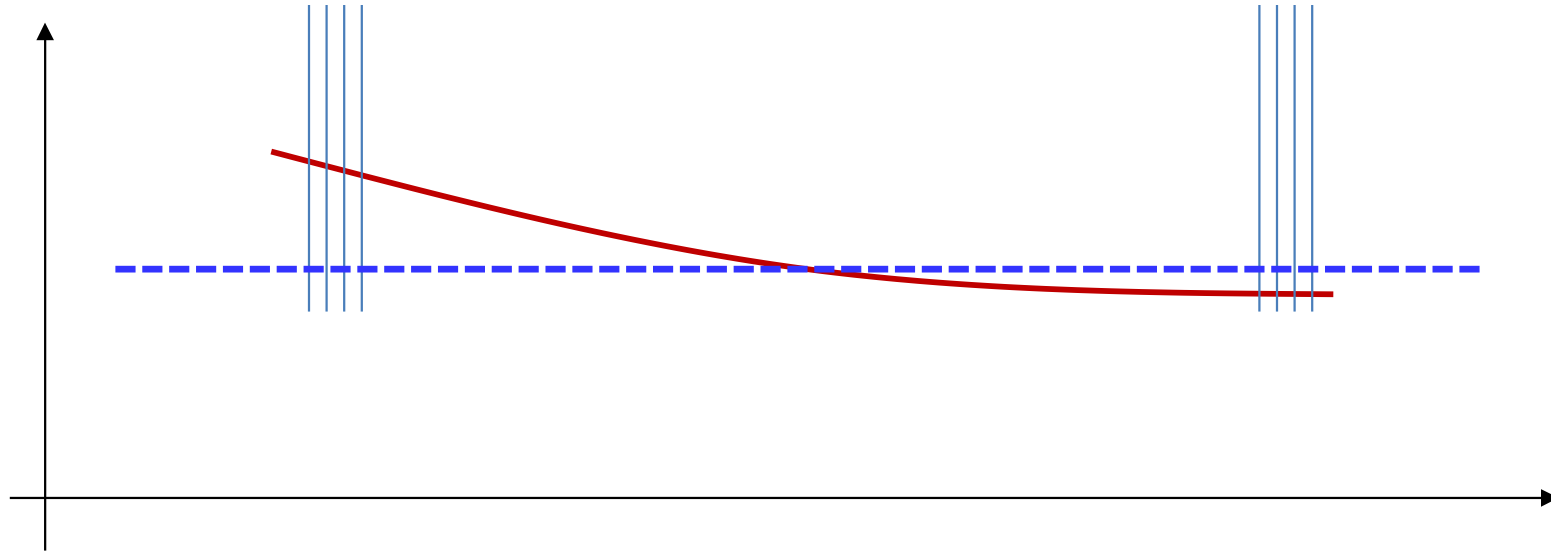
- If we loop through the samples in the same order, we may get *cyclic* behavior
- We must go through them *randomly* to get more convergent behavior

Caveats: order of presentation



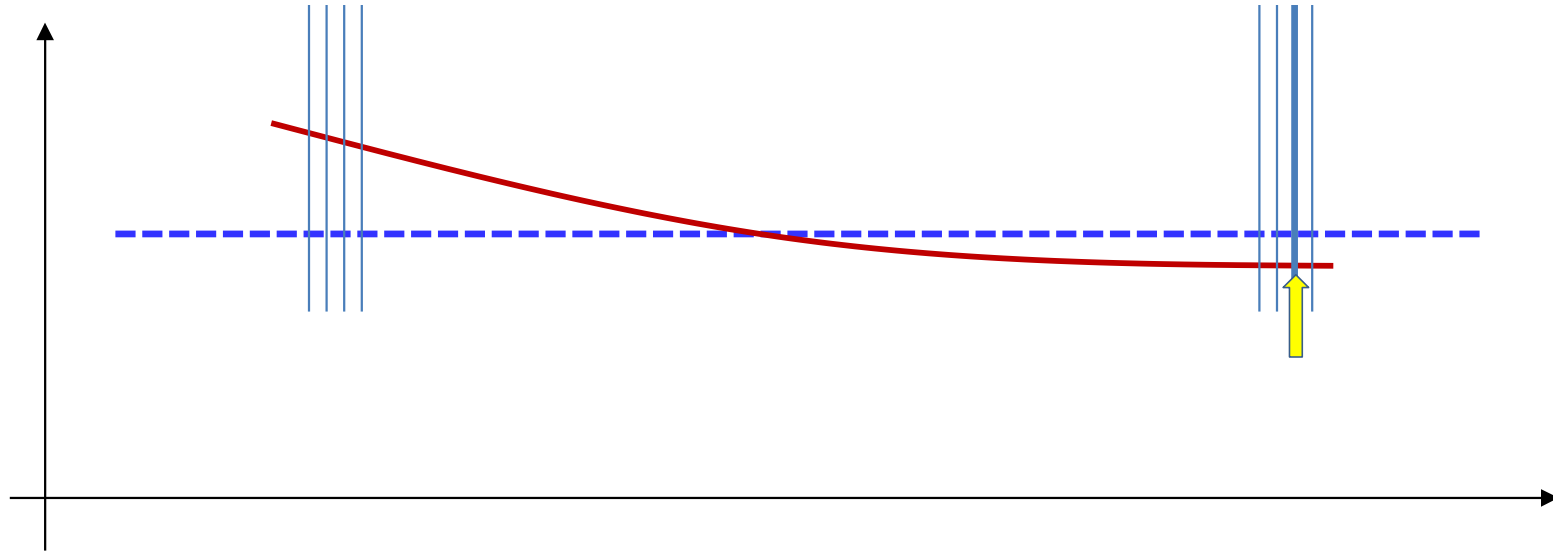
- If we loop through the samples in the same order, we may get *cyclic* behavior
- We must go through them *randomly* to get more convergent behavior

Caveats: order of presentation



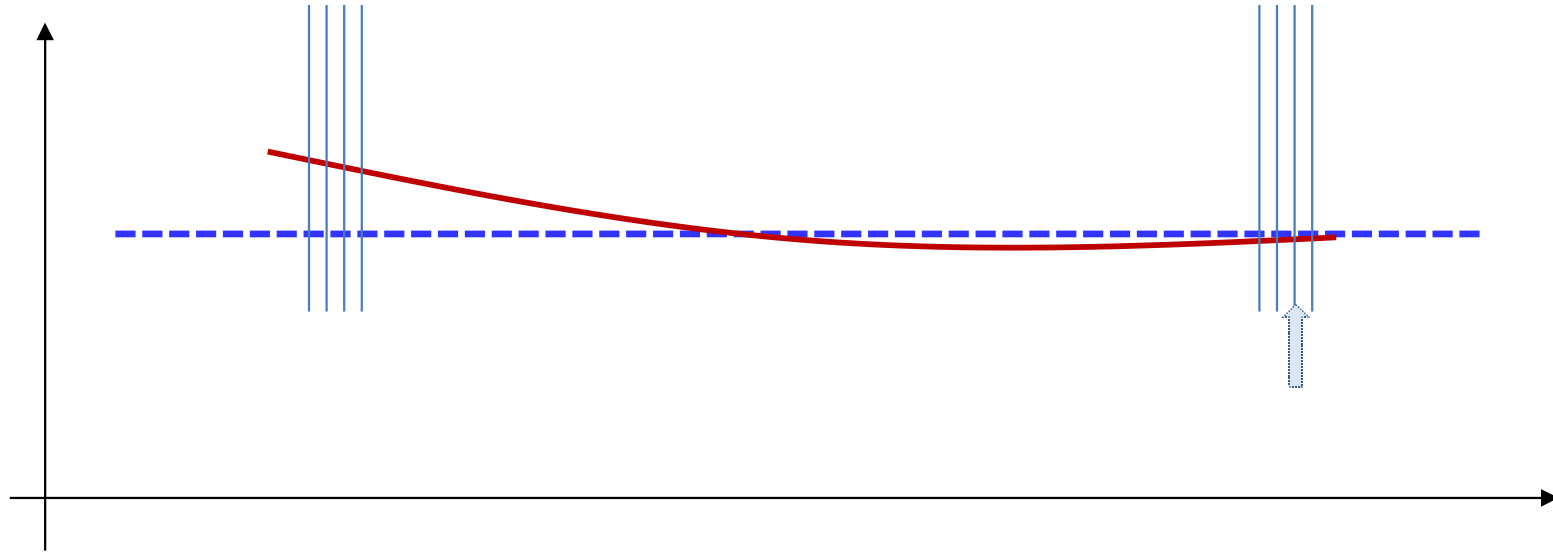
- If we loop through the samples in the same order, we may get *cyclic* behavior
- We must go through them *randomly* to get more convergent behavior

Caveats: order of presentation



- If we loop through the samples in the same order, we may get *cyclic* behavior
- We must go through them *randomly* to get more convergent behavior

Caveats: order of presentation



- If we loop through the samples in the same order, we may get *cyclic* behavior
- We must go through them *randomly* to get more convergent behavior

Incremental Update: Stochastic Gradient Descent

- Given $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$
- Initialize all weights $W_1, W_2, \dots, W_K$
- Do:
 - Randomly permute $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$
 - For all $t = 1:T$
 - For every layer k :
 - Compute $\nabla_{W_k} \text{Div}(\mathbf{Y}_t, \mathbf{d}_t)$
 - Update
$$W_k = W_k - \eta \nabla_{W_k} \text{Div}(\mathbf{Y}_t, \mathbf{d}_t)^T$$
- Until *Loss* has converged

Story so far

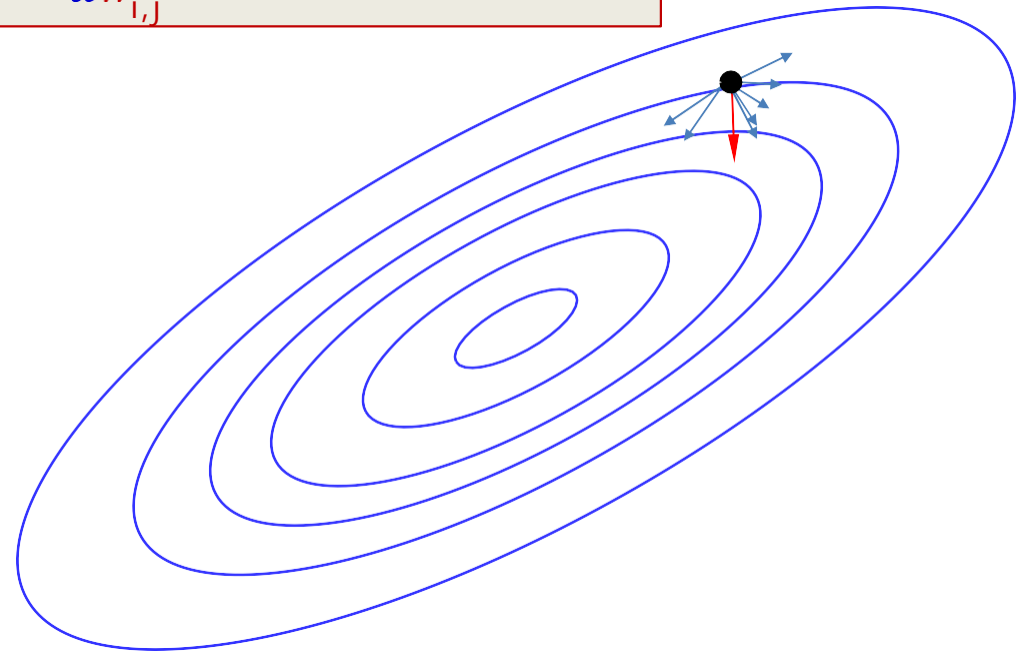
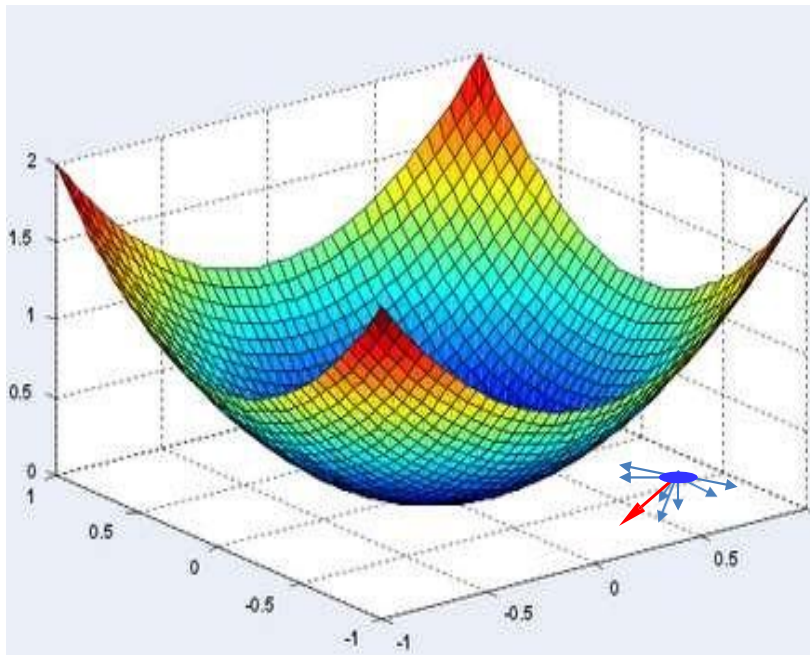
- In any gradient descent optimization problem, presenting training instances incrementally can be more effective than presenting them all at once
 - Provided training instances are provided in random order
 - “Stochastic Gradient Descent”
- This also holds for training neural networks

Explanations and restrictions

- So why does this process of incremental updates work?
- Under what conditions?
- For “why”: first consider a simplistic explanation that’s often given
 - Look at an extreme example

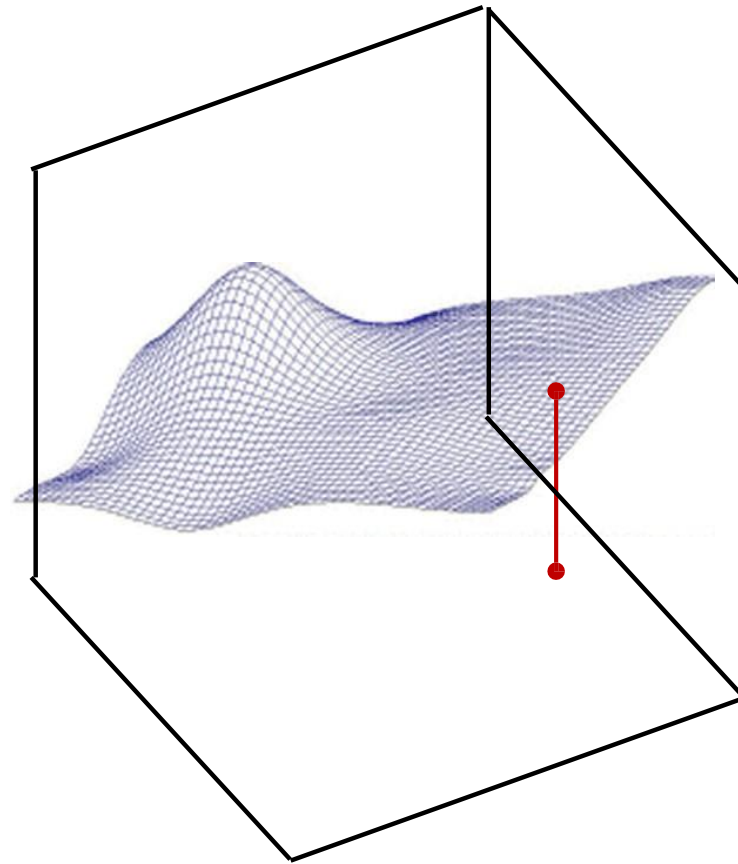
The expected behavior of the gradient

$$\frac{dE(W^{(1)}, W^{(2)}, \dots, W^{(K)})}{dw_{i,j}^{(k)}} = \frac{1}{T} \sum_i \frac{d\text{Div}(Y(X_i), d_i; W^{(1)}, W^{(2)}, \dots, W^{(K)})}{dw_{i,j}^{(k)}}$$



- The individual training instances contribute different directions to the overall gradient
 - The final gradient points is the average of individual gradients
 - It points towards the *net* direction

Extreme example

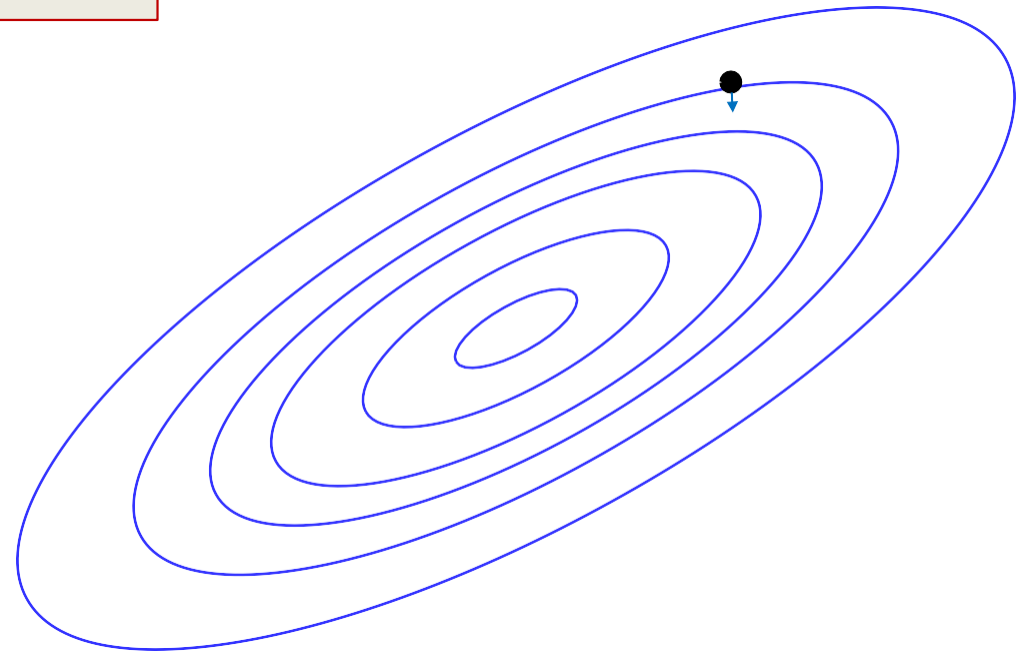
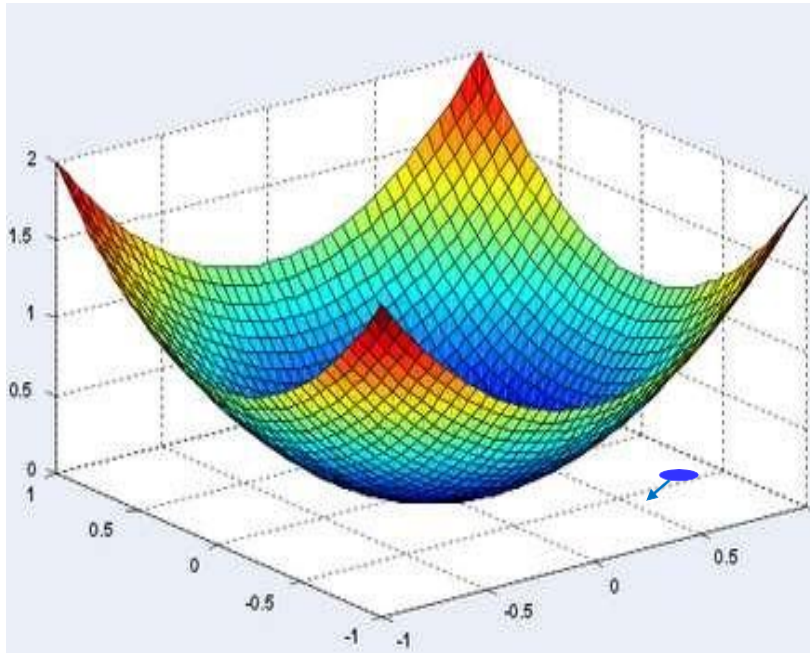


$$X_1 = X_2 = \dots = X_T$$

- Extreme instance of data clotting: all the training instances are exactly the same

The expected behavior of the gradient

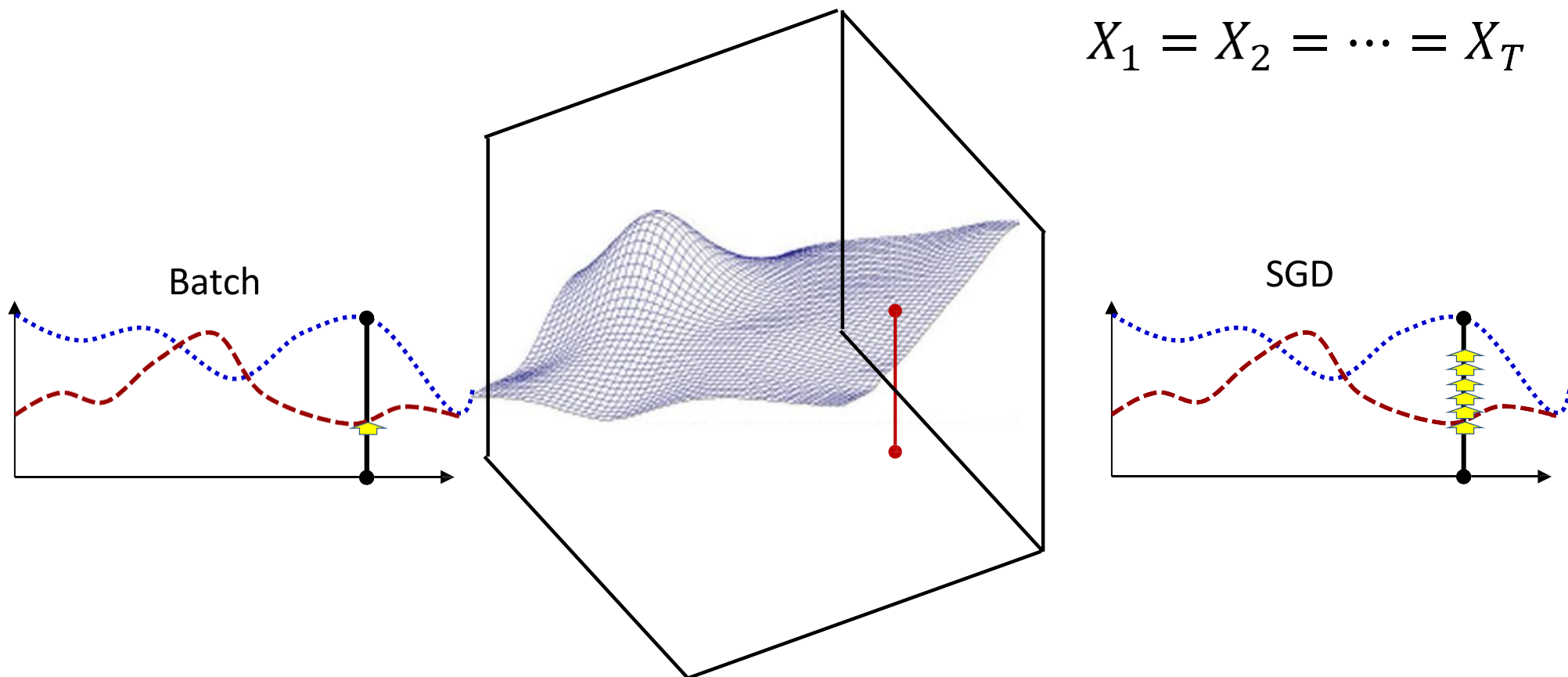
$$\frac{dE}{dw_{i,j}^{(k)}} = \frac{1}{T} \sum_i \frac{d\text{Div}(Y(X_i), d_i)}{dw_{i,j}^{(k)}} = \frac{d\text{Div}(Y(X_i), d_i)}{dw_{i,j}^{(k)}}$$



- The individual training instance contribute identical directions to the overall gradient
 - The final gradient points is simply the gradient for an individual instance

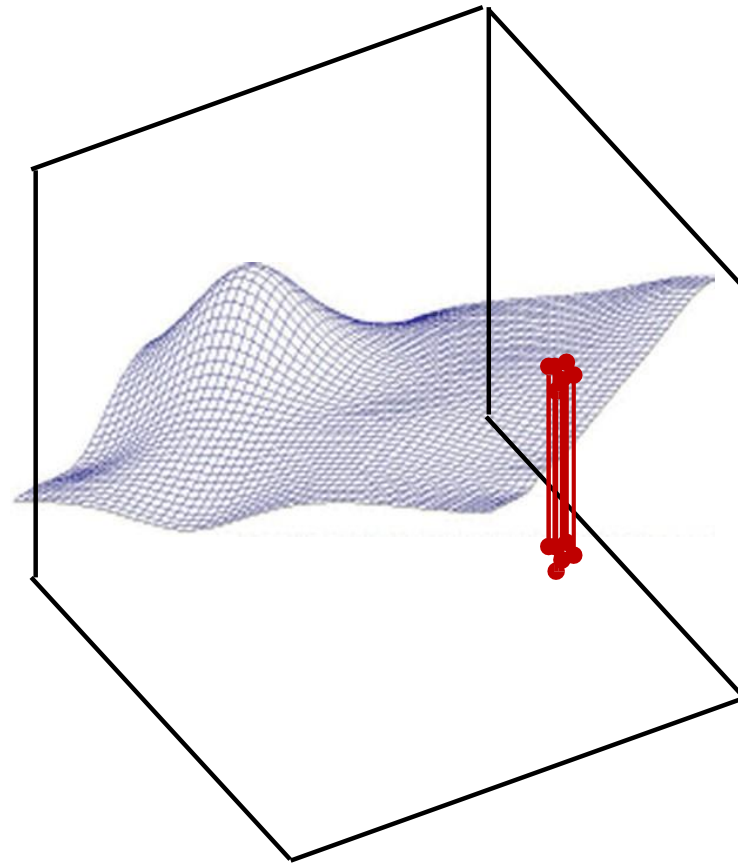
Batch vs SGD

$$X_1 = X_2 = \dots = X_T$$



- Batch gradient descent operates over T training instances to get a *single* update
- SGD gets T updates for the same computation

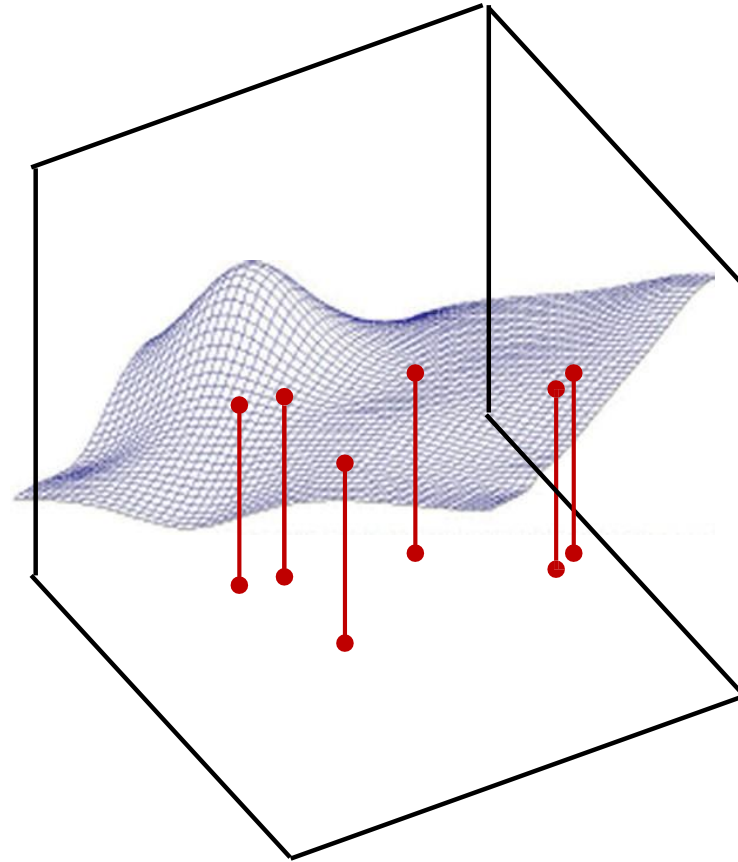
Clumpy data..



$$X_1 \approx X_2 \approx \dots \approx X_T$$

- Also holds if all the data are not identical, but are tightly clumped together

Clumpy data..

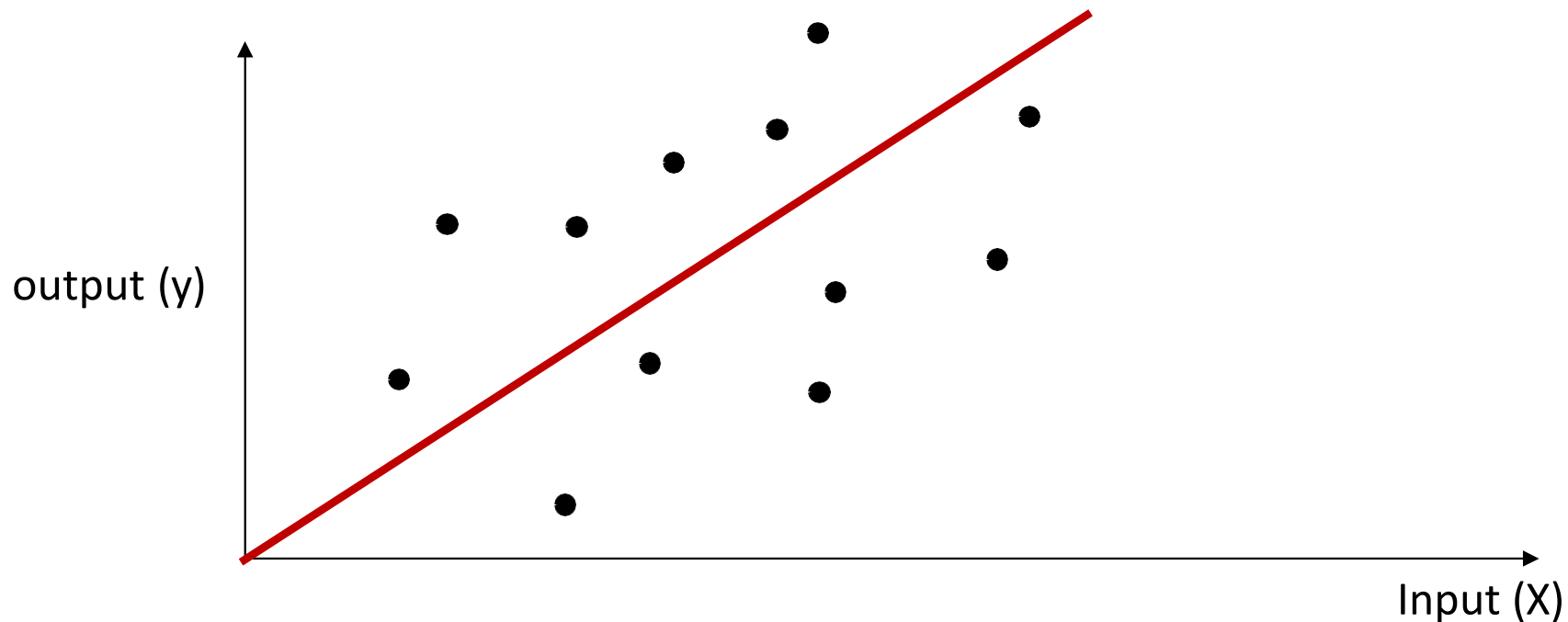


- As data get increasingly diverse, the benefits of incremental updates decrease, but do not entirely vanish

When does it work

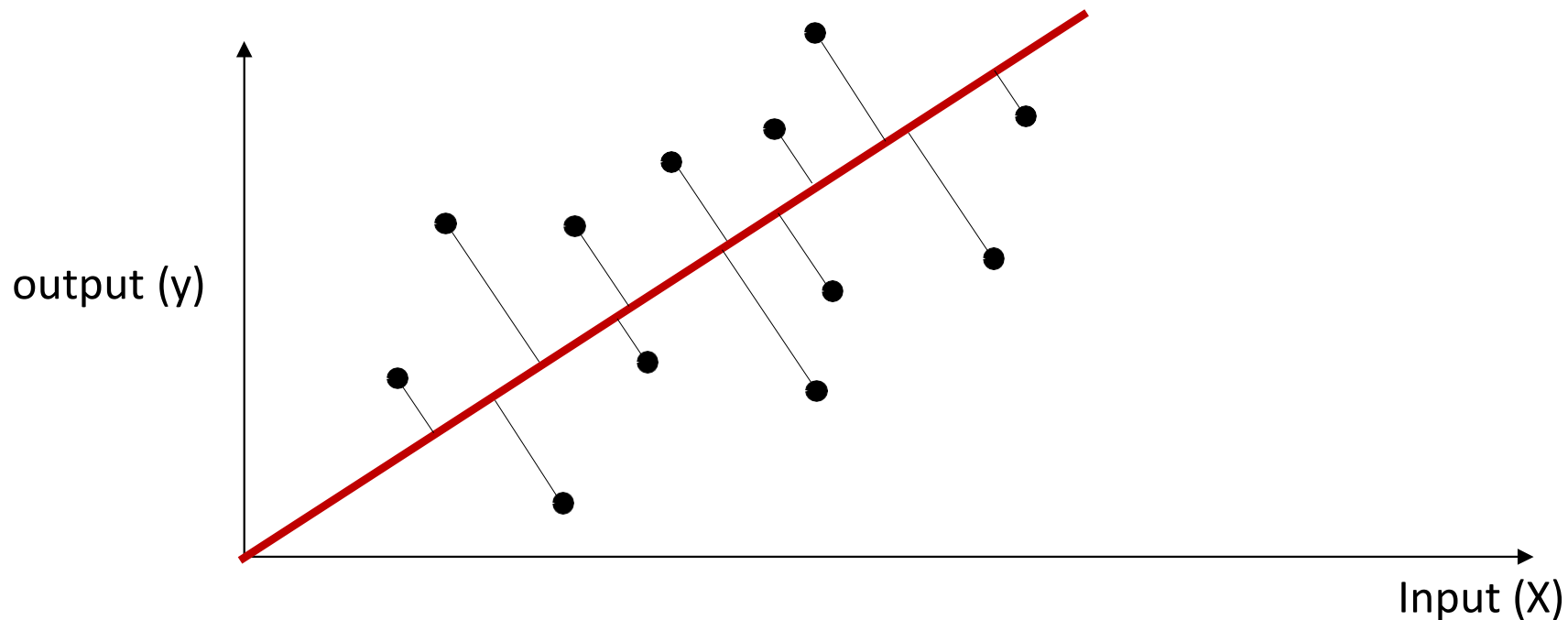
- What are the considerations?
- And how well does it work?

Incremental learning runs the risk of always chasing the latest input



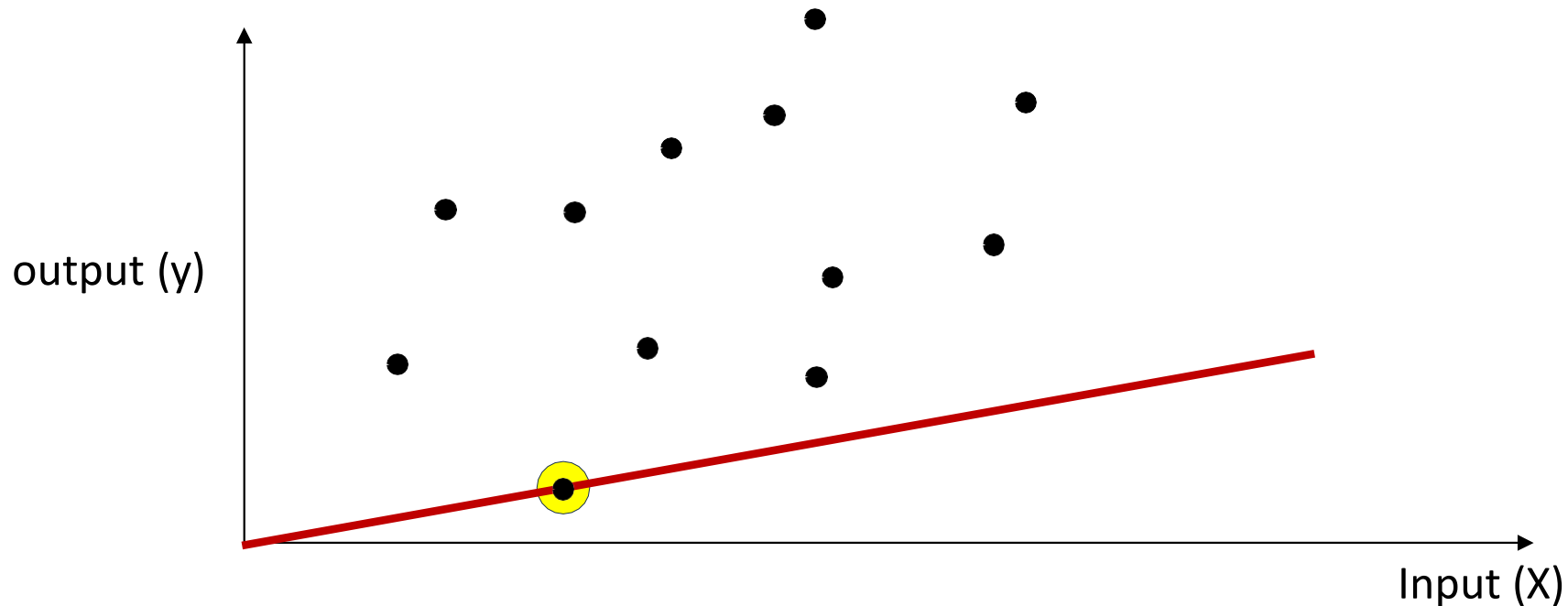
- **Modelling problem:** Find a linear regression line (through origin) to model the data

Incremental learning runs the risk of always chasing the latest input



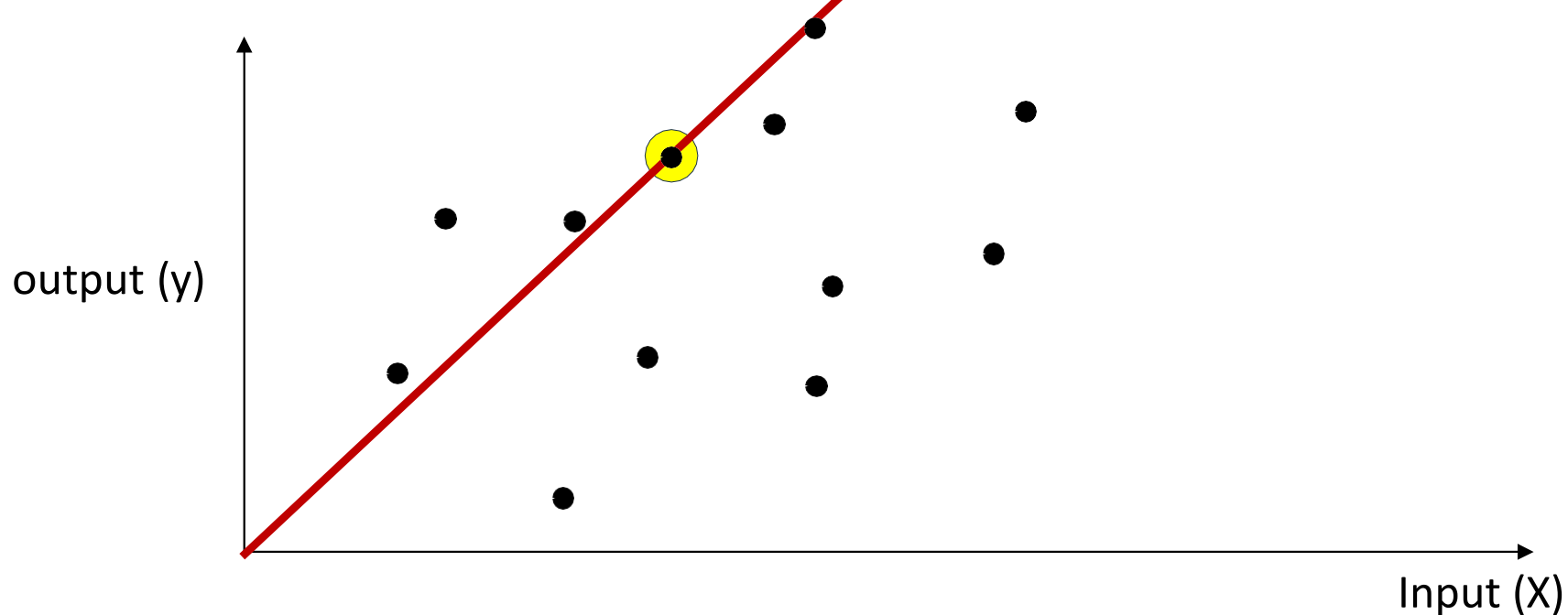
- **Modelling problem:** Find a linear regression line (through origin) to model the data
 - **Batch processing:** Find the line through origin that has the lowest overall squared projection error w.r.t. data

Incremental learning runs the risk of always chasing the latest input



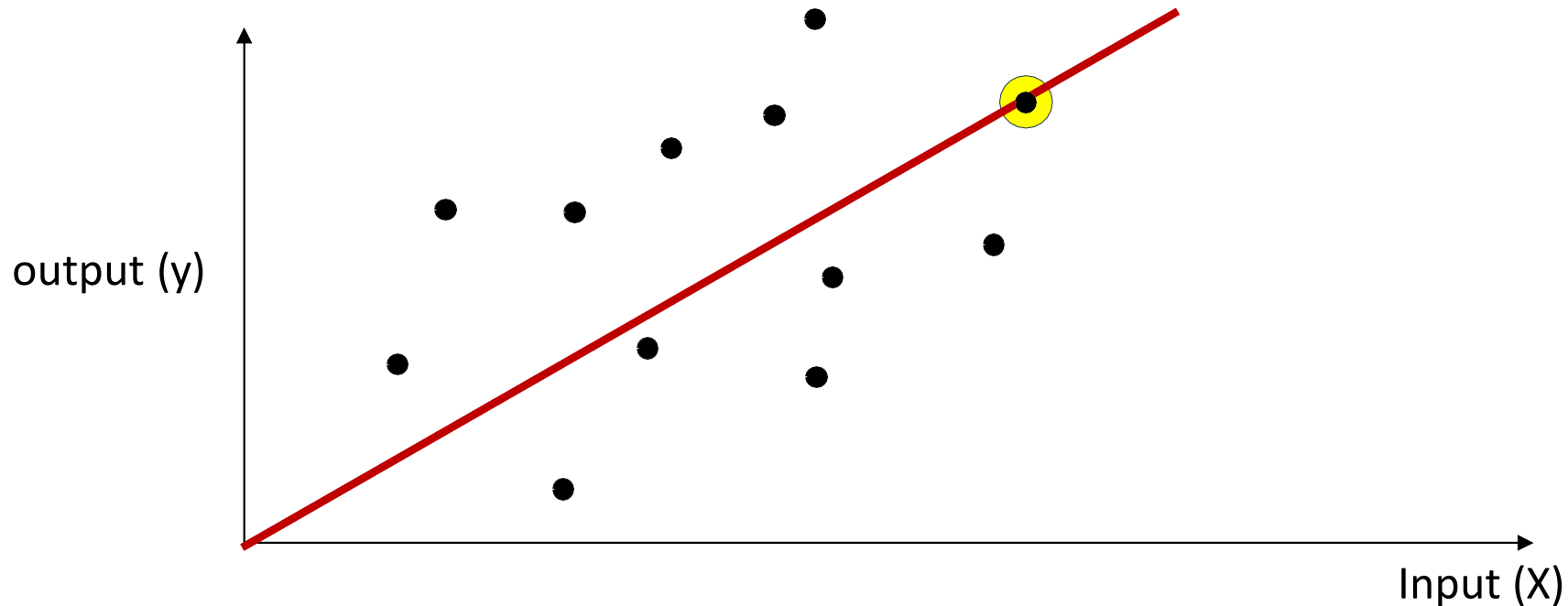
- **Incremental learning:** Update the model to always minimize the error on the latest instance
 - It will never converge

Incremental learning runs the risk of always chasing the latest input



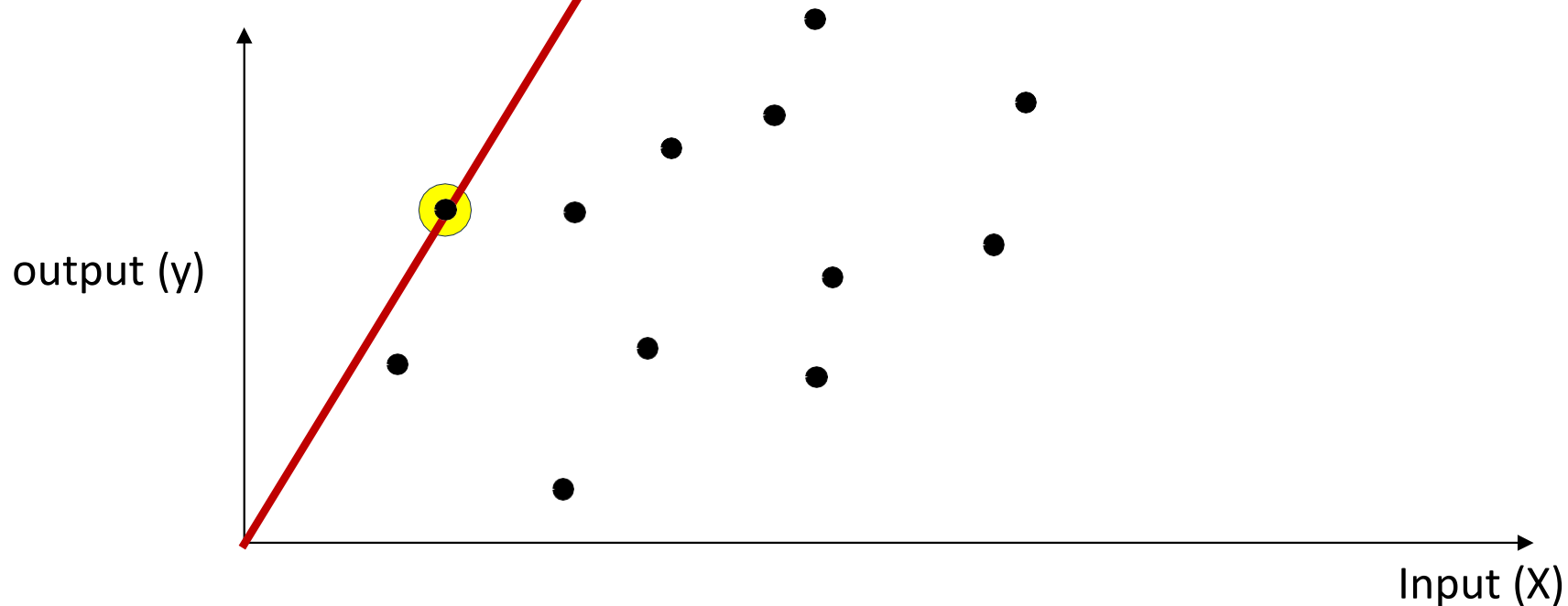
- **Incremental learning:** Update the model to always minimize the error on the latest instance
 - It will never converge

Incremental learning runs the risk of always chasing the latest input



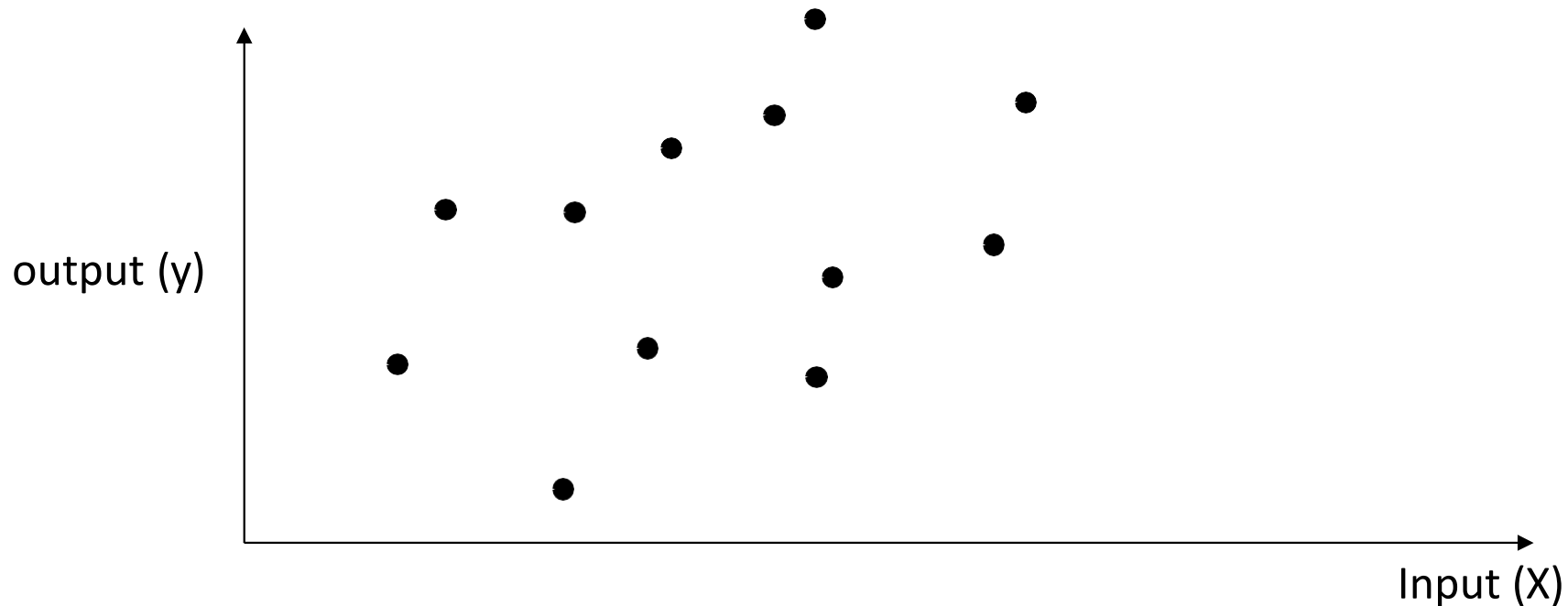
- **Incremental learning:** Update the model to always minimize the error on the latest instance
 - It will never converge

Incremental learning runs the risk of always chasing the latest input



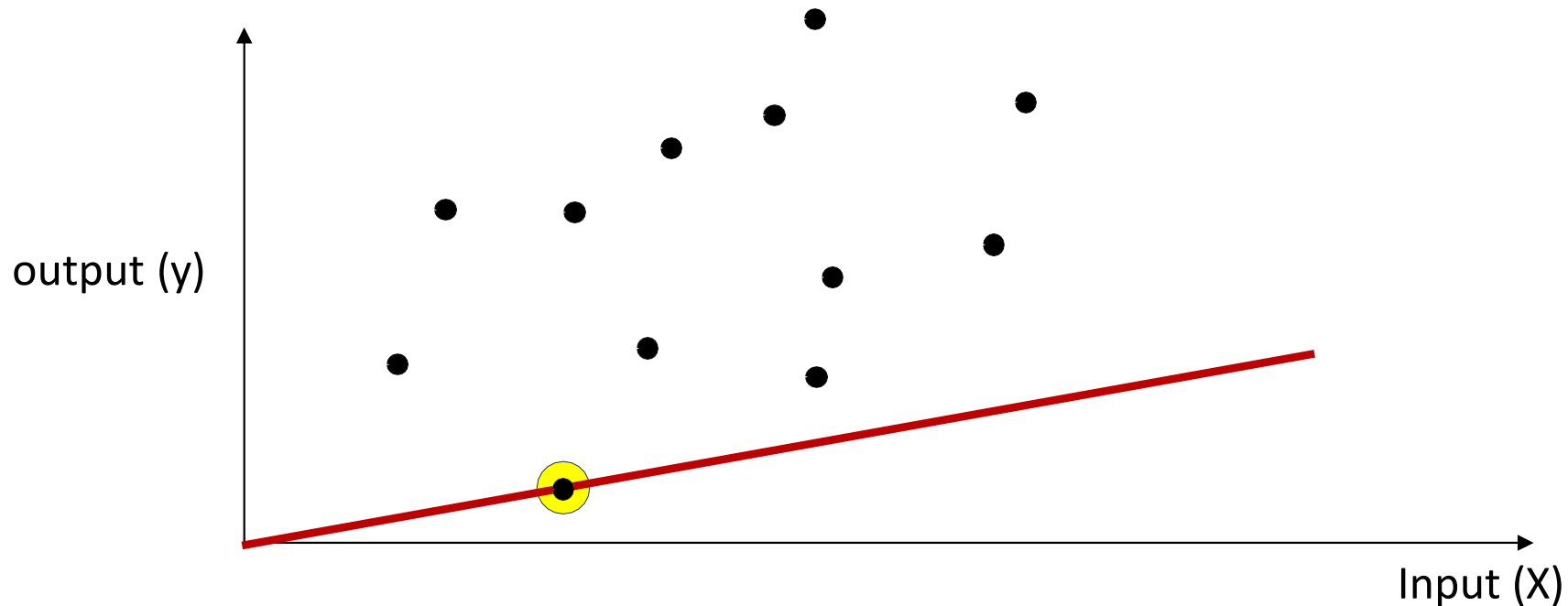
- **Incremental learning:** Update the model to always minimize the error on the latest instance
 - It will never converge
 - **Solution?**

Incremental learning runs the risk of always chasing the latest input



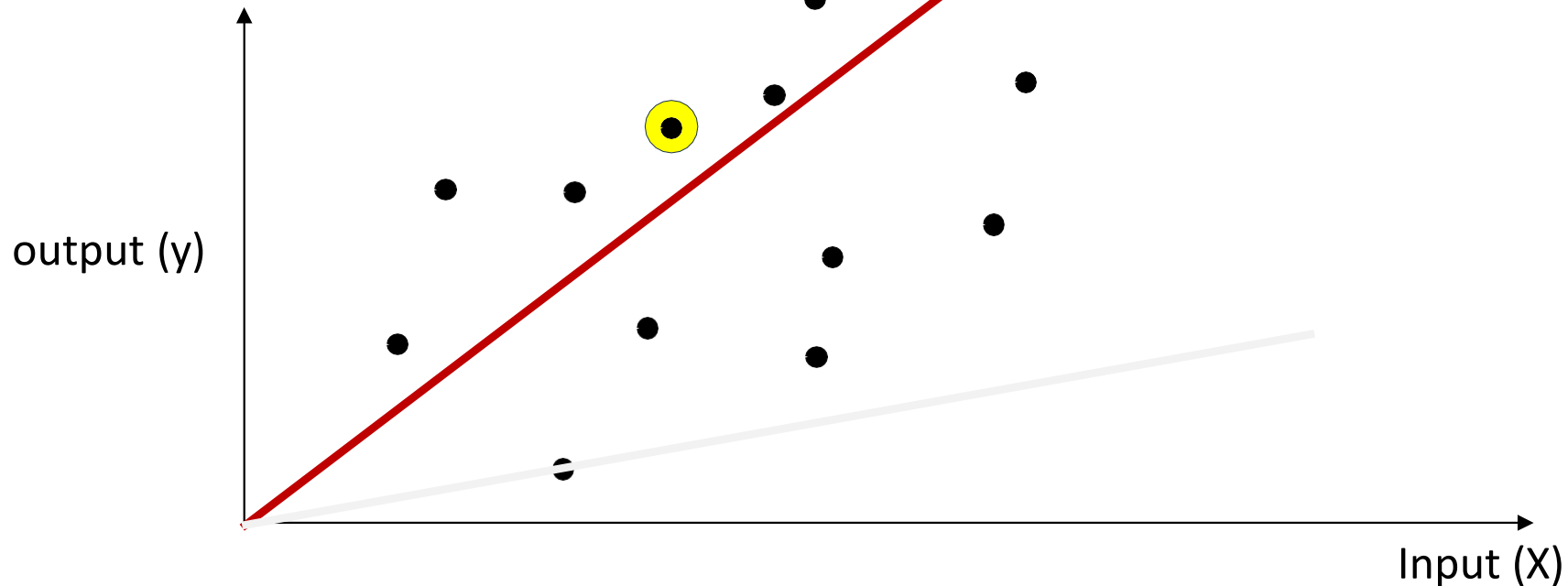
- **Incremental learning:** Update the model to always minimize the error on the latest instance
 - Shrink the learning rate with iterations
 - With increasing iterations, it will swing less and less towards the new point

Incremental learning runs the risk of always chasing the latest input



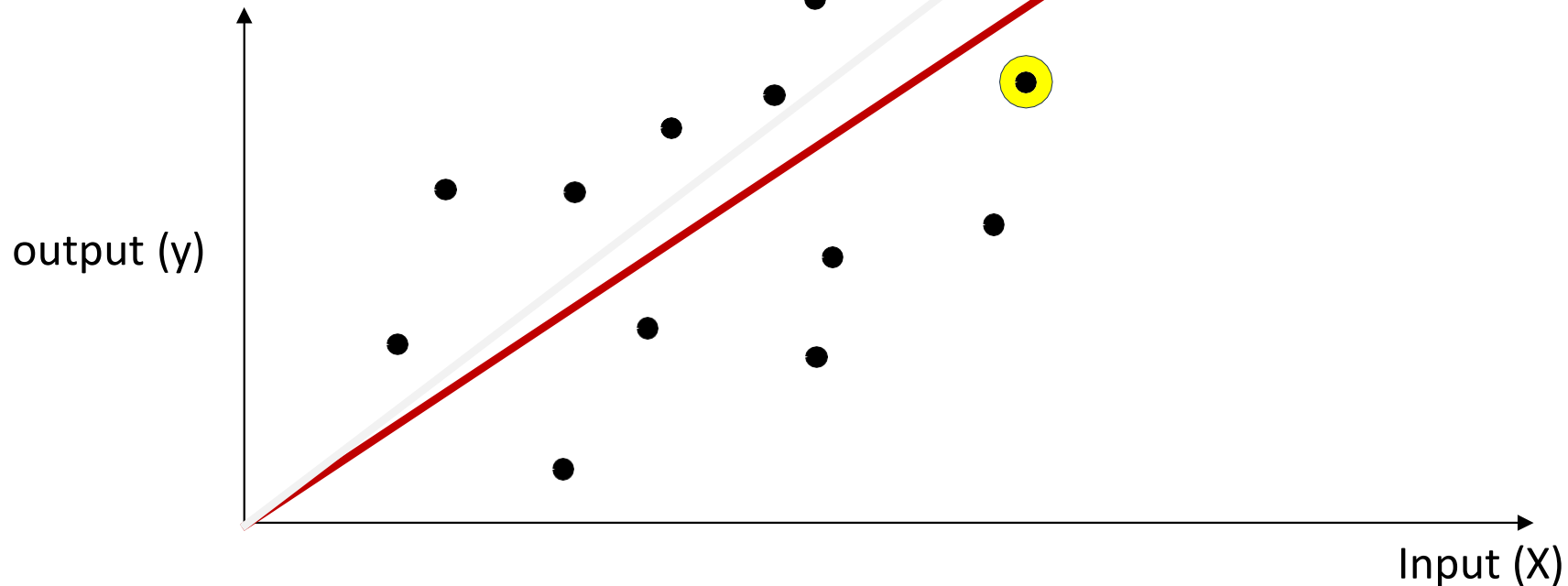
- **Incremental learning:** Update the model to always minimize the error on the latest instance
 - Shrink the learning rate with iterations
 - With increasing iterations, it will swing less and less towards the new point

Incremental learning runs the risk of always chasing the latest input



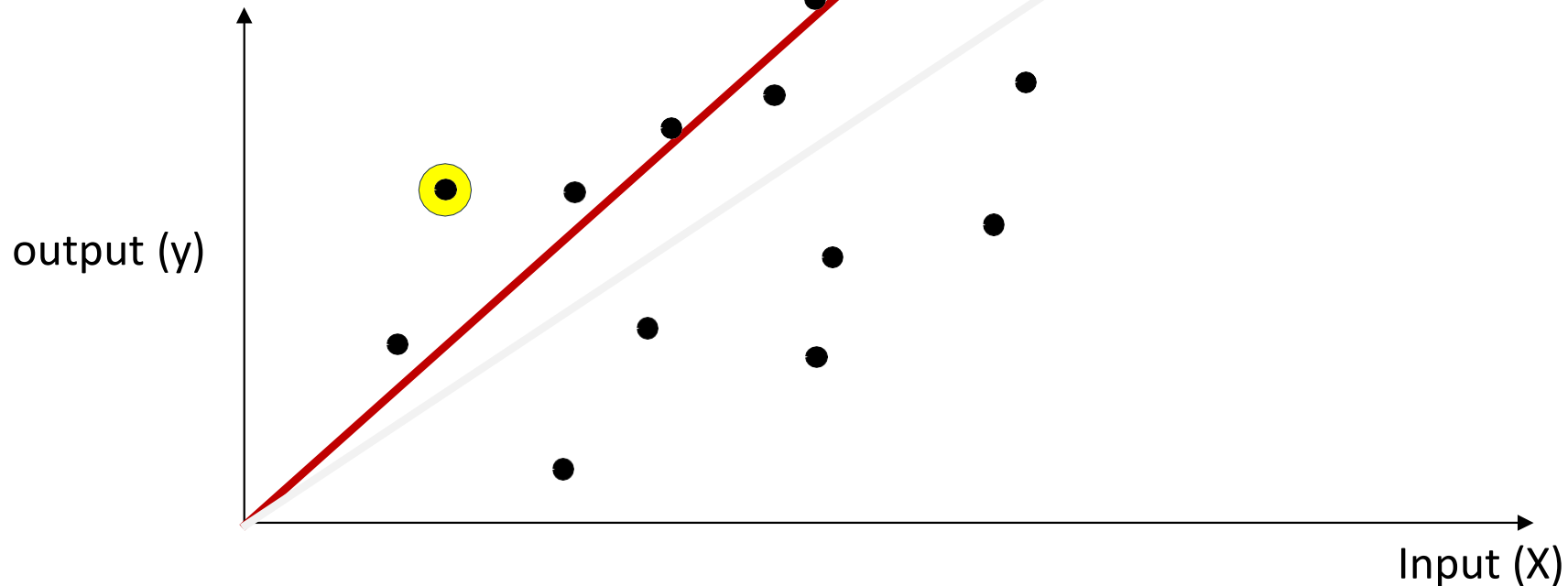
- **Incremental learning:** Update the model to always minimize the error on the latest instance
 - Shrink the learning rate with iterations
 - With increasing iterations, it will swing less and less towards the new point

Incremental learning runs the risk of always chasing the latest input



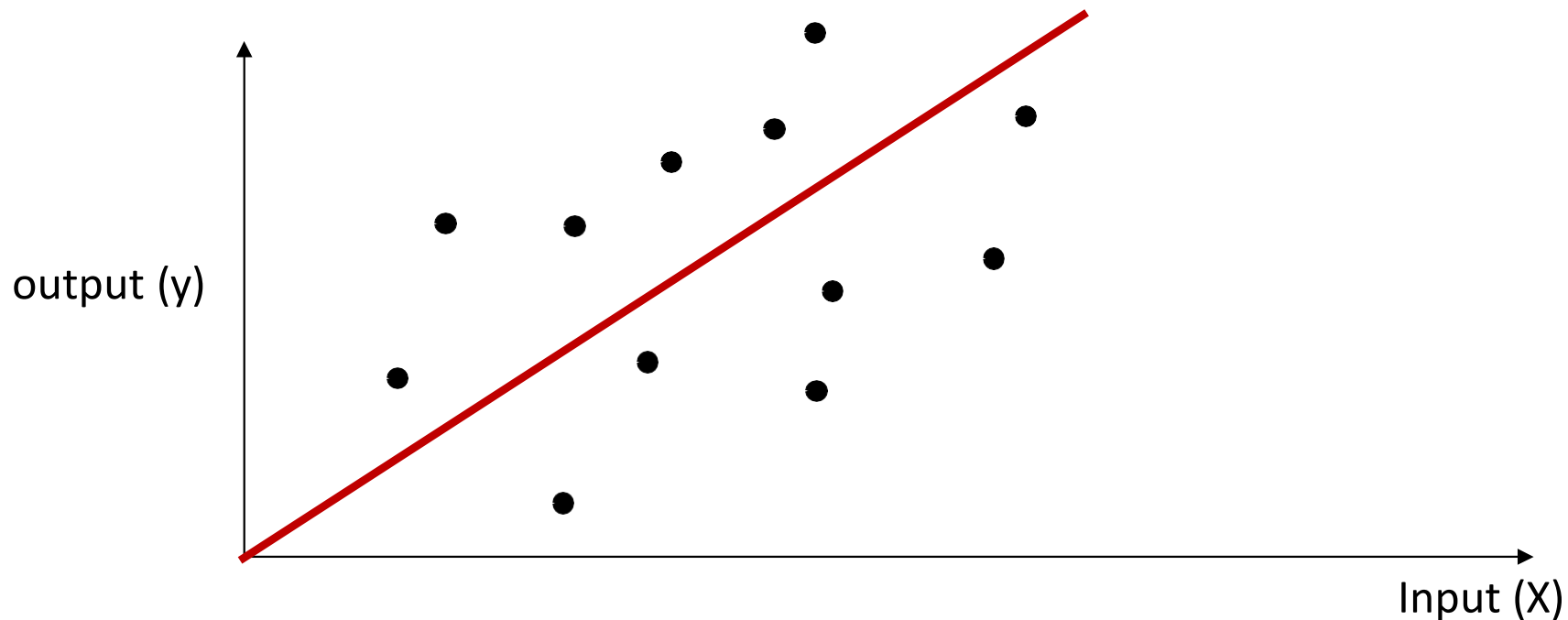
- **Incremental learning:** Update the model to always minimize the error on the latest instance
 - Shrink the learning rate with iterations
 - With increasing iterations, it will swing less and less towards the new point

Incremental learning runs the risk of always chasing the latest input



- **Incremental learning:** Update the model to always minimize the error on the latest instance
 - Shrink the learning rate with iterations
 - With increasing iterations, it will swing less and less towards the new point

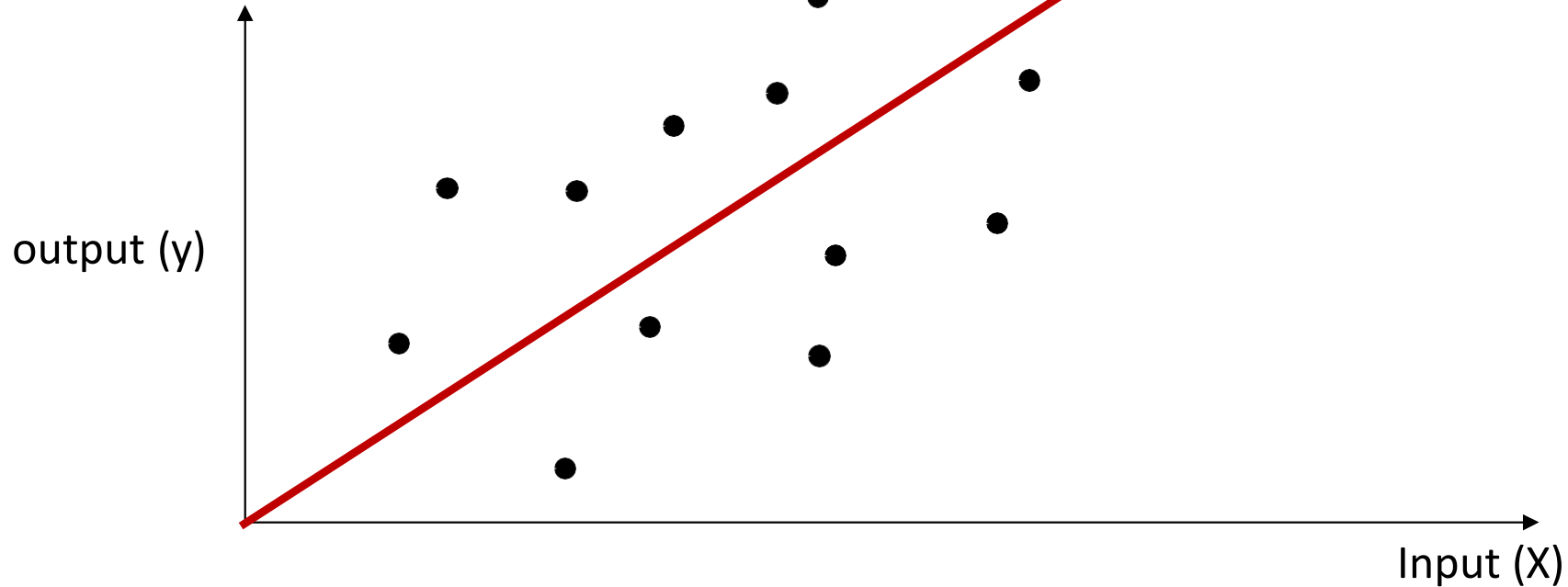
Incremental learning runs the risk of always chasing the latest input



- **Incremental learning:** Update the model to always minimize the error on the latest instance
 - Shrink the learning rate with iterations
 - With increasing iterations, it will swing less and less towards the new point
 - Eventually arriving at the correct solution and not moving much from it further because the step sizes are now too small...

Incremental learning caveat: **learning**

rate



- **Incremental learning:** Update the model to always minimize the error on the latest instance
 - **Caveat:** We must *shrink* the learning rate with iterations for convergence
 - Correction for individual instances with the eventual miniscule learning rates will not modify the function

Incremental Update: Stochastic Gradient Descent

- Given $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$
- Initialize all weights $W_1, W_2, \dots, W_K; j = 0$
- Do:
 - Randomly permute $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$
 - For all $t = 1:T$
 - $j = j + 1$
 - For every layer k :
 - Compute $\nabla_{W_k} \text{Div}(\mathbf{Y}_t, \mathbf{d}_t)$
 - Update
$$W_k = W_k - \eta_j \nabla_{W_k} \text{Div}(\mathbf{Y}_t, \mathbf{d}_t)^T$$
- Until *Loss* has converged

Incremental Update: Stochastic Gradient Descent

- Given $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$
- Initialize all weights $W_1, W_2, \dots, W_K; j = 0$
- Do:

– Randomly permute $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$

– For all $t = 1:T$

• $j = j + 1$

• For every layer k :

– Compute $\nabla_{W_k} \text{Div}(\mathbf{Y}_t, \mathbf{d}_t)$

– Update

$$W_k = W_k - \eta_j \nabla_{W_k} \text{Div}(\mathbf{Y}_t, \mathbf{d}_t)^T$$

- Until *Loss* has converged

Randomize input order

Learning rate reduces with j

SGD convergence

- SGD converges “almost surely” to a global or local minimum for most functions
 - Sufficient condition: step sizes follow the following conditions (Robbins and Munro 1951)

$$\sum_k \eta_k = \infty$$

- Eventually the entire parameter space can be searched

$$\sum_k \eta_k^2 < \infty$$

- The steps shrink
 - The fastest converging series that satisfies both above requirements is

$$\eta_k \propto \frac{1}{k}$$

- This is the optimal rate of shrinking the step size for strongly convex functions
 - More generally, the learning rates are heuristically determined
- If the loss is convex, SGD converges to the optimal solution
- For non-convex losses SGD converges to a local minimum

SGD convergence

- We will define convergence in terms of the number of iterations taken to get within ϵ of the optimal solution

- $|f(W^{(k)}) - f(W^*)| < \epsilon$

- Note: $f(W)$ here is the optimization objective on the *entire* training data, although SGD itself updates after every training instance

- Using the optimal learning rate $1/k$, for *strongly convex* functions,

$$|f(W^{(k)}) - f(W^*)| < \frac{1}{k} |f(W^{(0)}) - f(W^*)|$$

- Strongly convex $\rightarrow$ Can be placed inside a quadratic bowl, touching at any point
 - Giving us the iterations to ϵ convergence as $O\left(\frac{1}{\epsilon}\right)$

- For generically convex (but not strongly convex) function, various proofs report an ϵ convergence of $\frac{1}{\sqrt{k}}$ using a learning rate of $\frac{1}{\sqrt{k}}$.

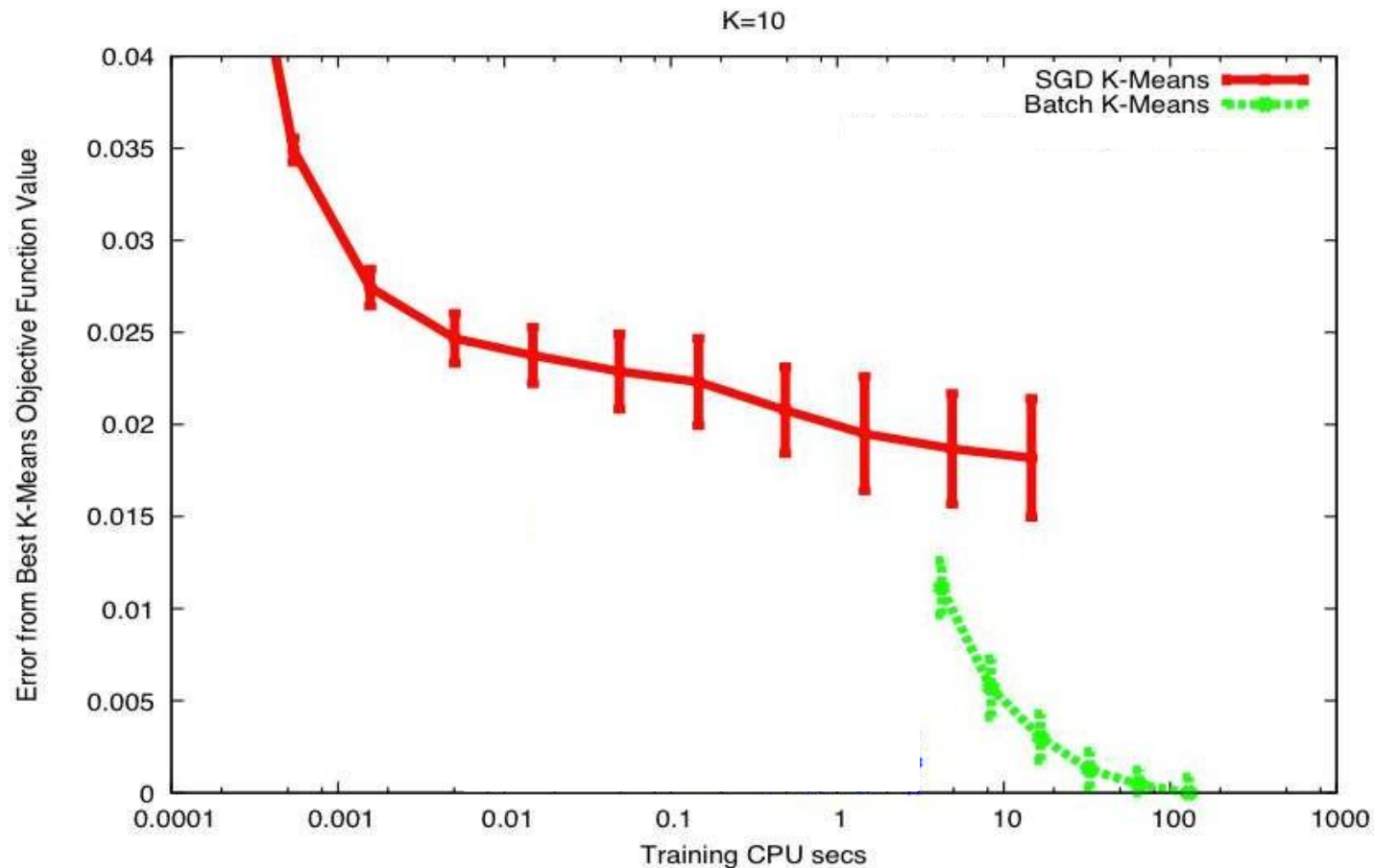
Batch gradient convergence

- In contrast, using the batch update method, for *strongly convex* functions,

$$|f(W^{(k)}) - f(W^*)| < c^k |f(W^{(0)}) - f(W^*)|$$

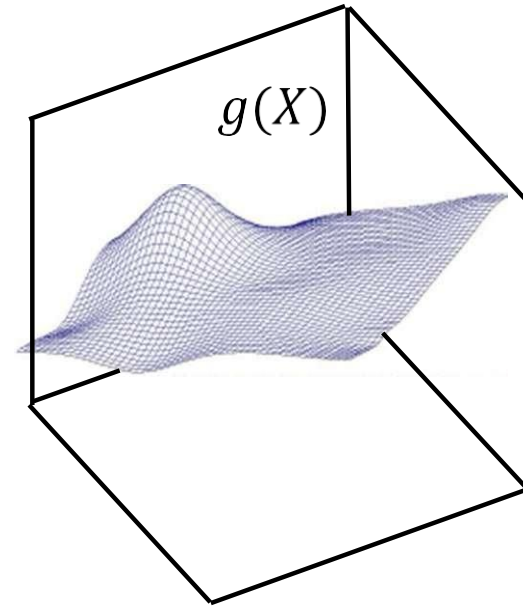
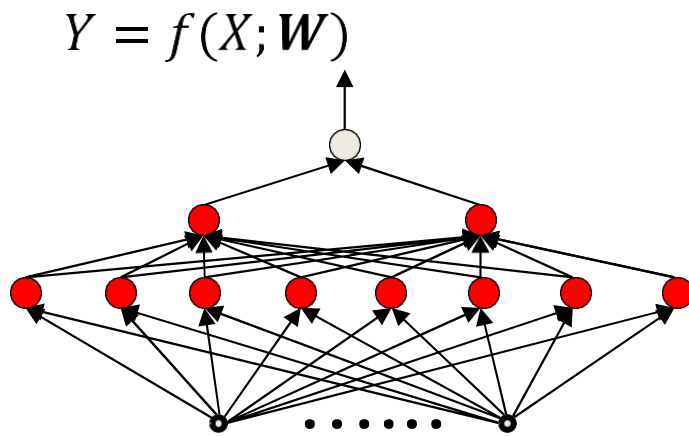
- Giving us the iterations to ϵ convergence as $O\left(\log\left(\frac{1}{\epsilon}\right)\right)$
- For generic convex functions, iterations to ϵ convergence is $O\left(\frac{1}{\epsilon}\right)$
- Batch gradients converge “faster”
 - But SGD performs T updates for every batch update

SGD example



- A simpler problem: K-means
- Note: SGD converges faster
 - But to a poorer minimum
- Also note the rather large variation between runs
 - Let's try to understand these results..

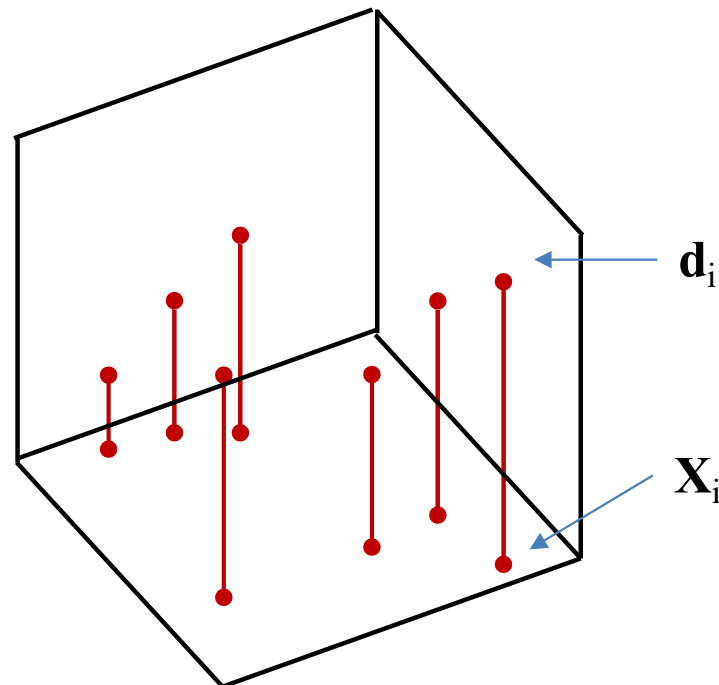
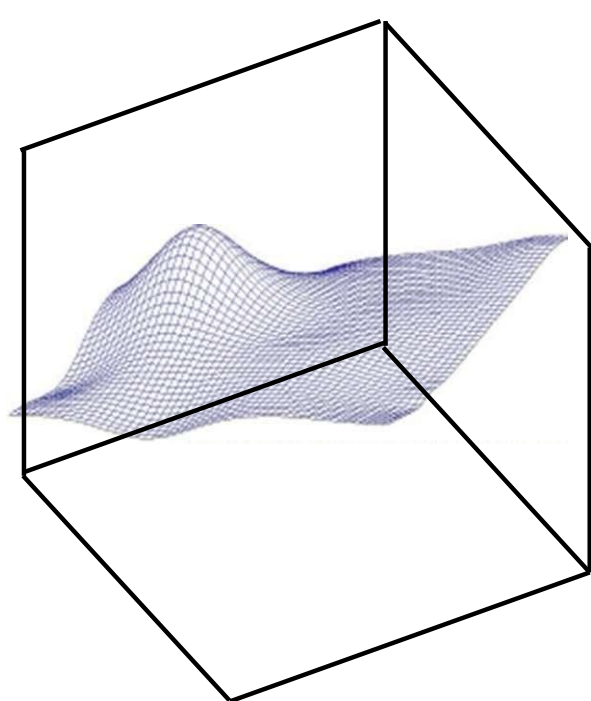
Recall: Modelling a function



- To learn a network $f(X; W)$ to model a function $g(X)$ we minimize the *expected divergence*

$$\begin{aligned}\widehat{W} &= \operatorname{argmin}_W \int_X \operatorname{div}(f(X; W), g(X)) P(X) dX \\ &= \operatorname{argmin}_W E[\operatorname{div}(f(X; W), g(X))]\end{aligned}$$

Recall: The *Empirical* risk



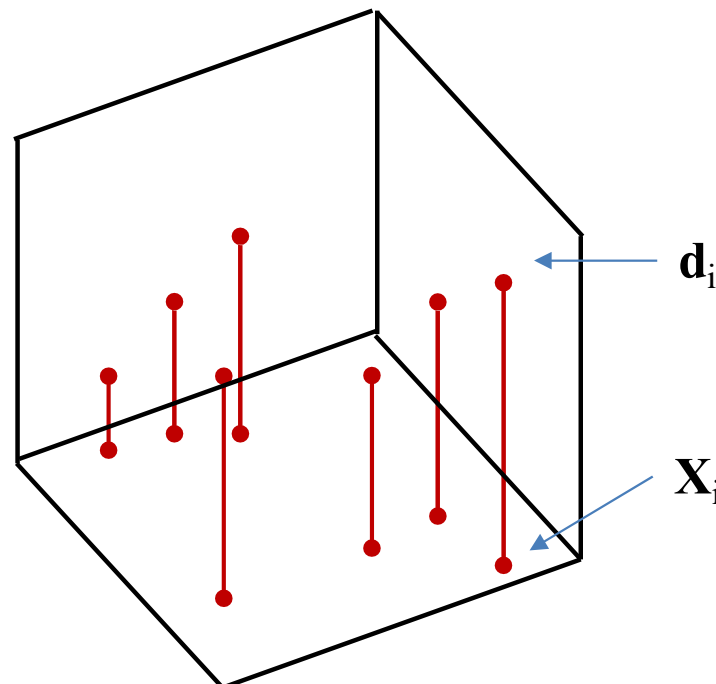
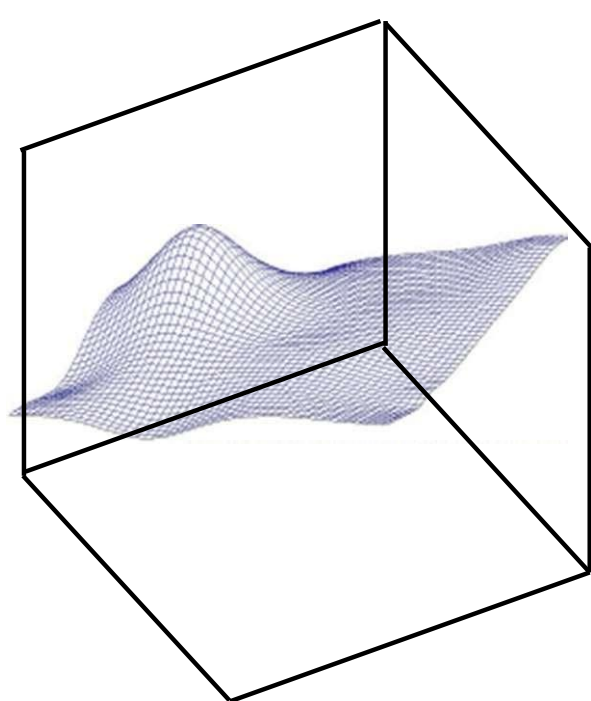
- In practice, we minimize the *empirical risk* (or *loss*)

$$Loss(W) = \frac{1}{N} \sum_{i=1}^N \text{div}(f(X_i; W), d_i)$$
$$\hat{W} = \underset{W}{\operatorname{argmin}} Loss(W)$$

- The *expected value* of the *empirical risk* is actually the *expected divergence*

$$E[Loss(W)] = E[\text{div}(f(X; W), g(X))]$$

Recall: The *Empirical* risk



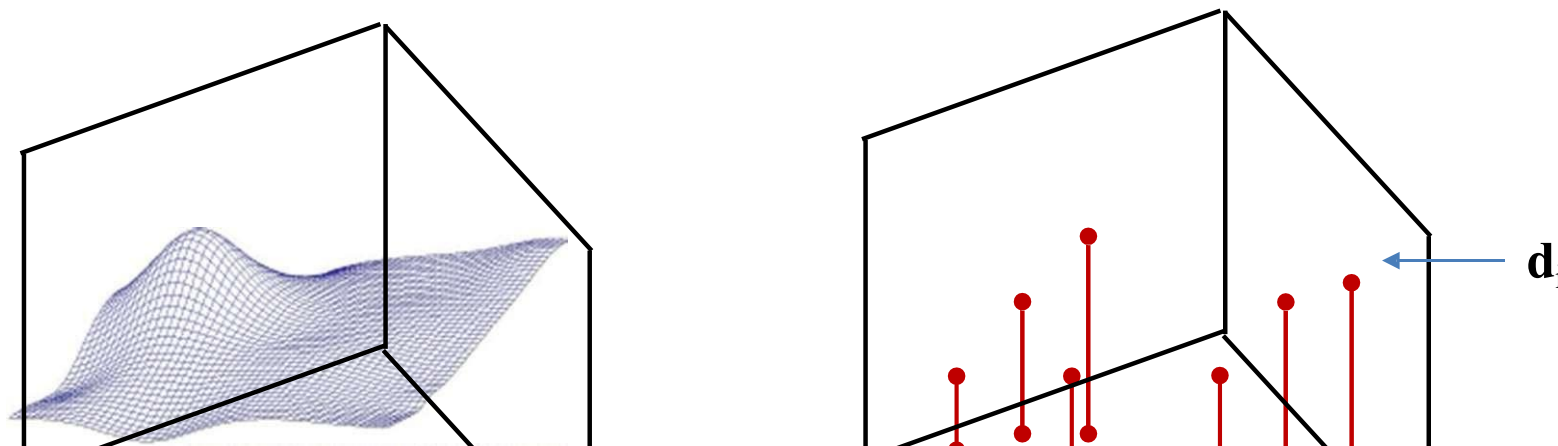
- In practice, we minimize the *empirical risk* (or *loss*)

$$Loss(W) = \frac{1}{N} \sum_{i=1}^N div(f(X_i; W), d_i)$$

The empirical risk is an *unbiased* estimate of the expected divergence
Though there is no guarantee that minimizing it will minimize the
expected divergence

$$E[Loss(W)] = E[div(f(X; W), g(X))]$$

Recall: The *Empirical* risk



The variance of the empirical risk: $\text{var}(\text{Loss}) = 1/N \text{ var}(\text{div})$

The variance of the estimator is proportional to $1/N$

The larger this variance, the greater the likelihood that the W that minimizes the empirical risk will differ significantly from the W that minimizes the expected divergence

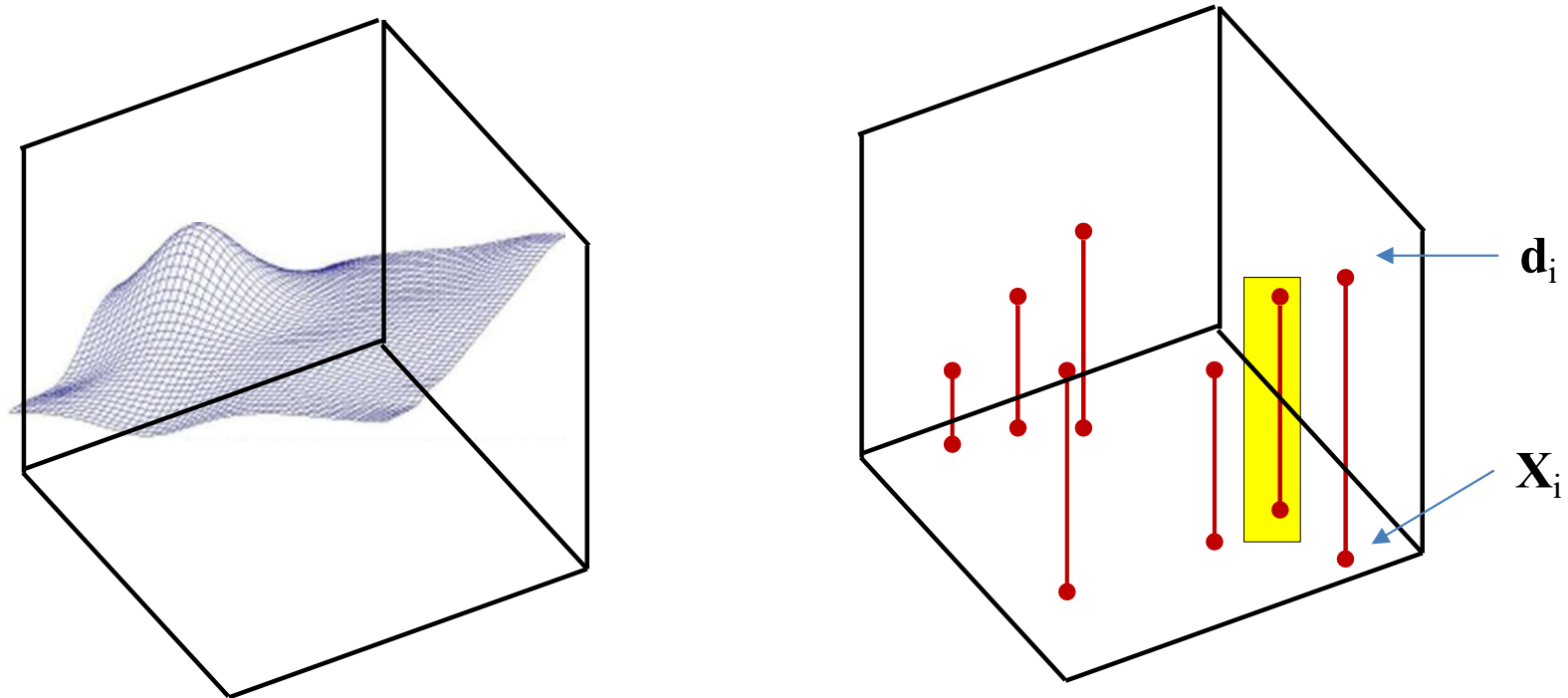
$$\text{Loss}(W) = \frac{1}{N} \sum_{i=1}^N \text{div}(f(X_i; W), d_i)$$

The empirical risk is an *unbiased* estimate of the expected divergence

Though there is no guarantee that minimizing it will minimize the expected divergence

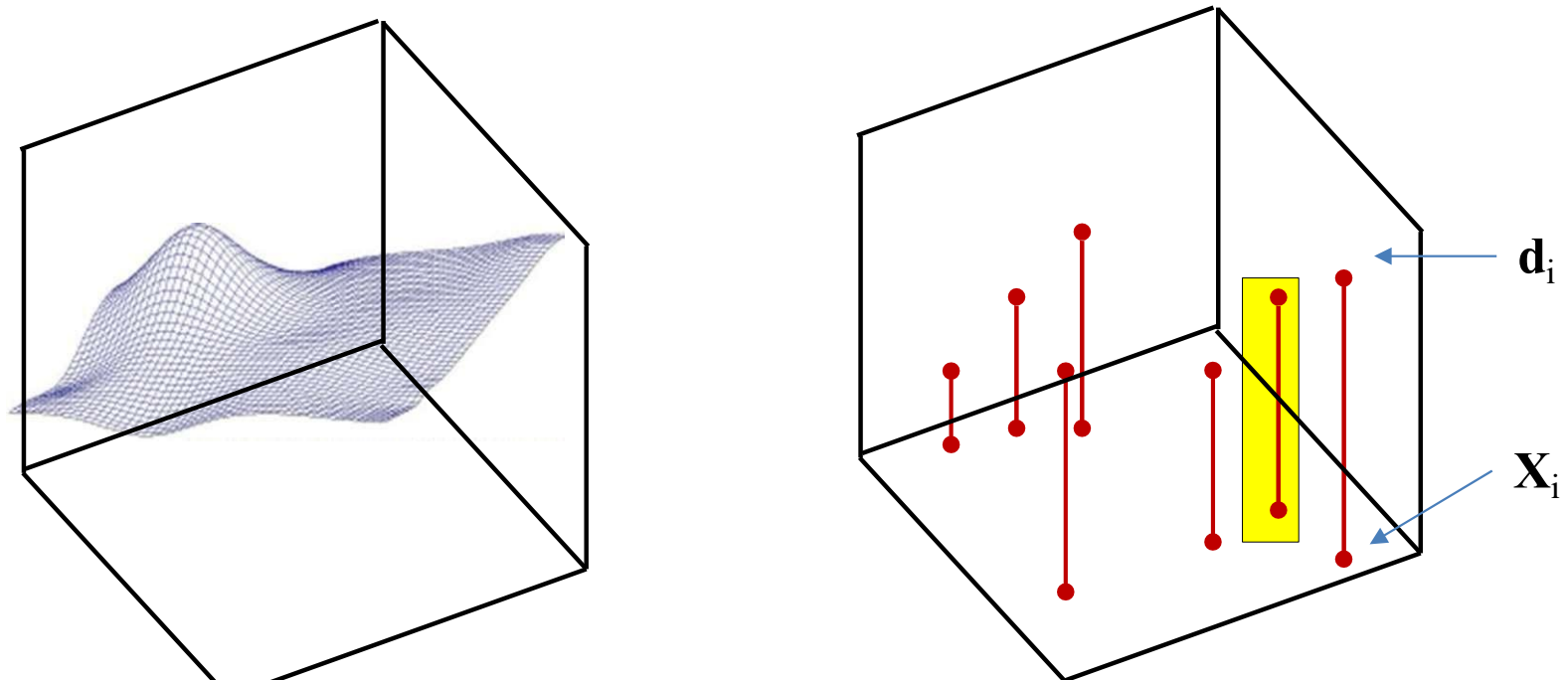
$$E[\text{Loss}(W)] = E[\text{div}(f(X; W), g(X))]$$

SGD



- At each iteration, **SGD** focuses on the divergence of a **single** sample $div(f(X_i; W), d_i)$
- The *expected value* of the *sample error* is **still** the *expected divergence* $E[div(f(X; W), g(X))]$

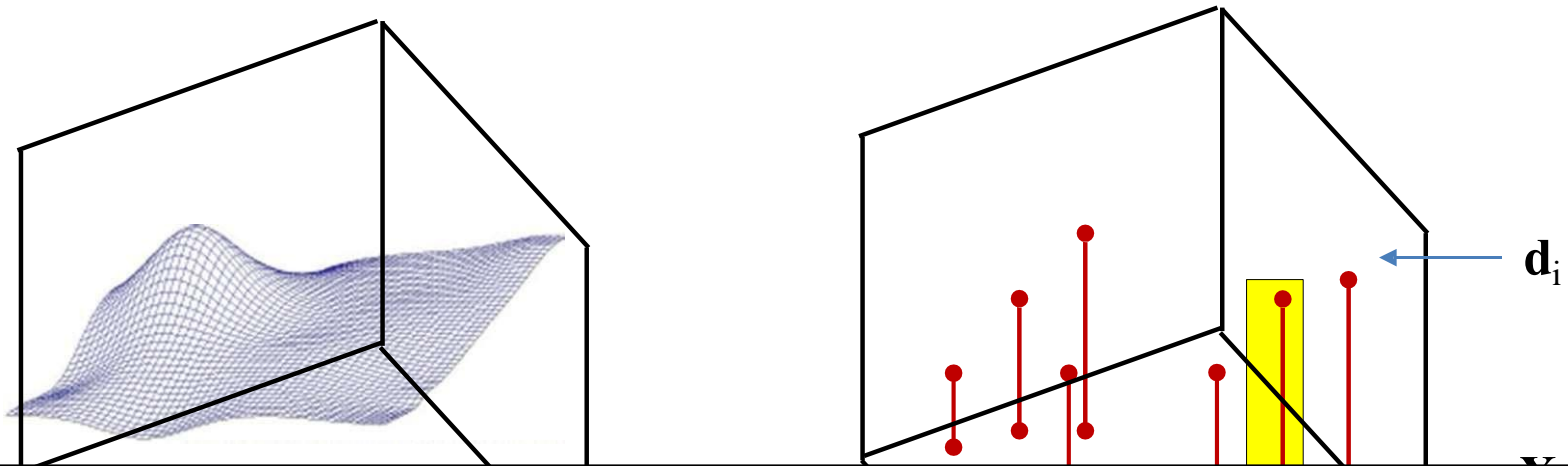
SGD



The sample divergence is also an *unbiased* estimate of the expected error

- At each iteration, **SGD** focuses on the divergence of a *single* sample $div(f(X_i; W), d_i)$
- The *expected value* of the *sample error* is *still* the *expected divergence* $E[div(f(X; W), g(X))]$

SGD

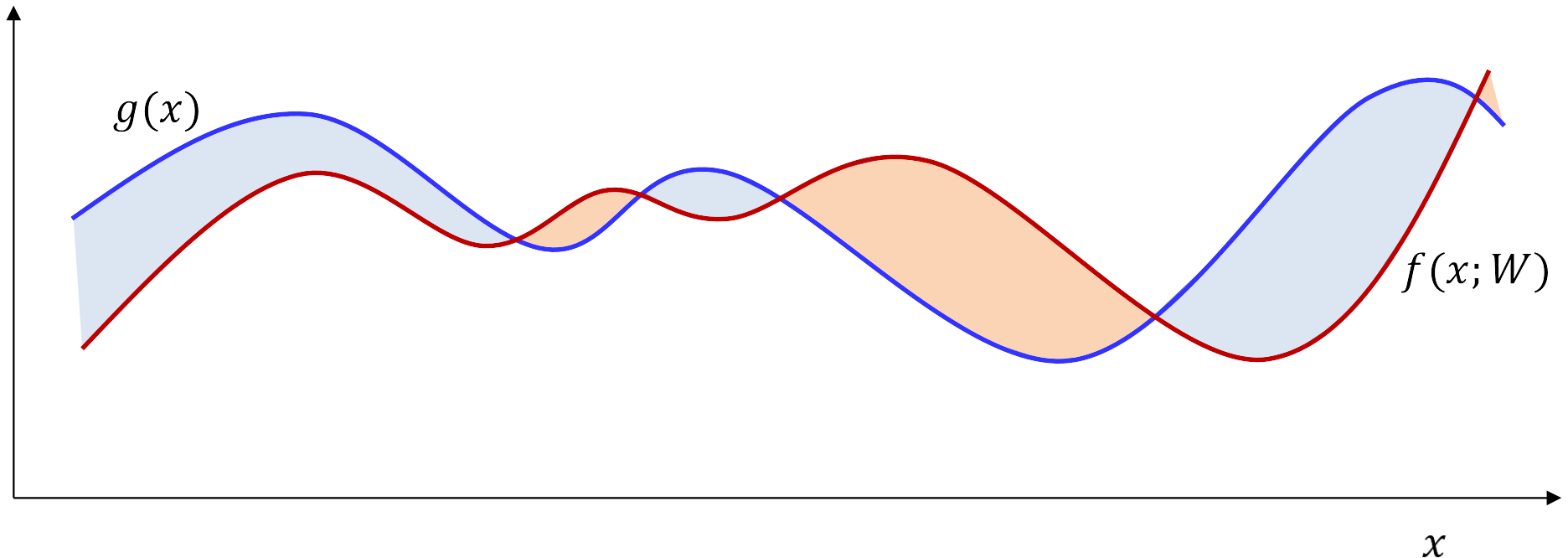


The variance of the sample divergence is the variance of the divergence itself: $\text{var}(\text{div})$. This is N times the variance of the empirical average minimized by batch update

The sample divergence is also an *unbiased* estimate of the expected error

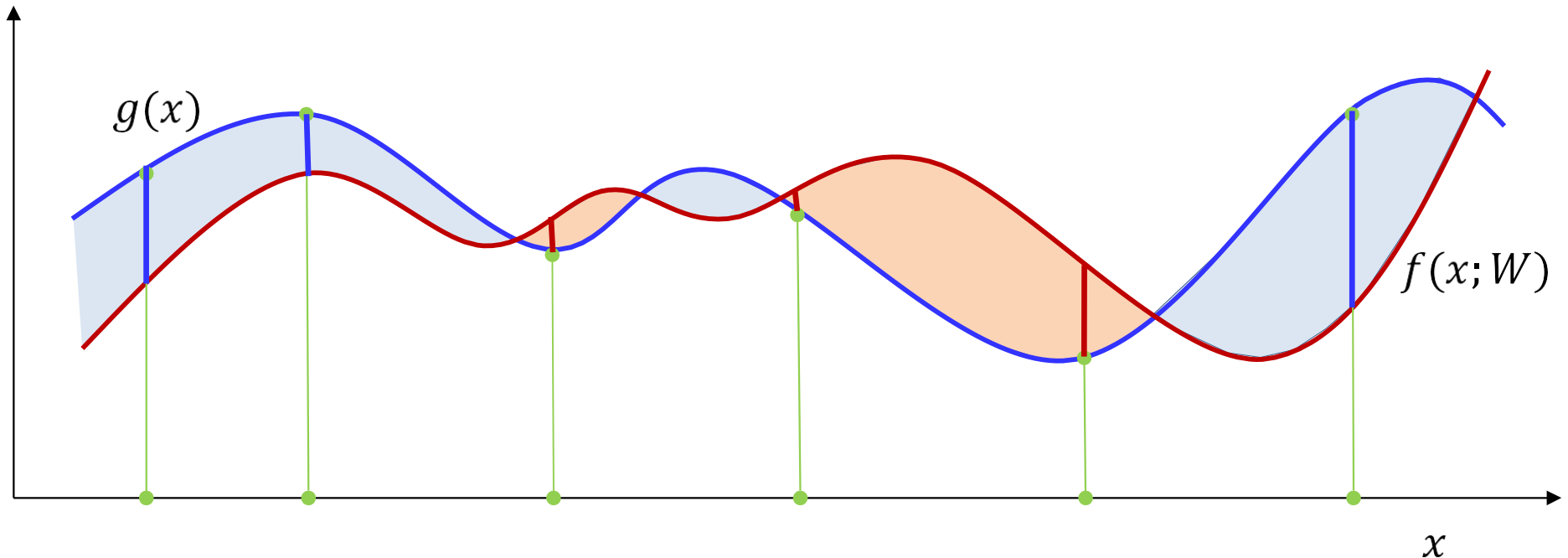
- At each iteration, **SGD** focuses on the divergence of a **single** sample $\text{div}(f(X_i; W), d_i)$
- The *expected value* of the *sample error* is **still** the *expected divergence* $E[\text{div}(f(X; W), g(X))]$

Explaining the variance



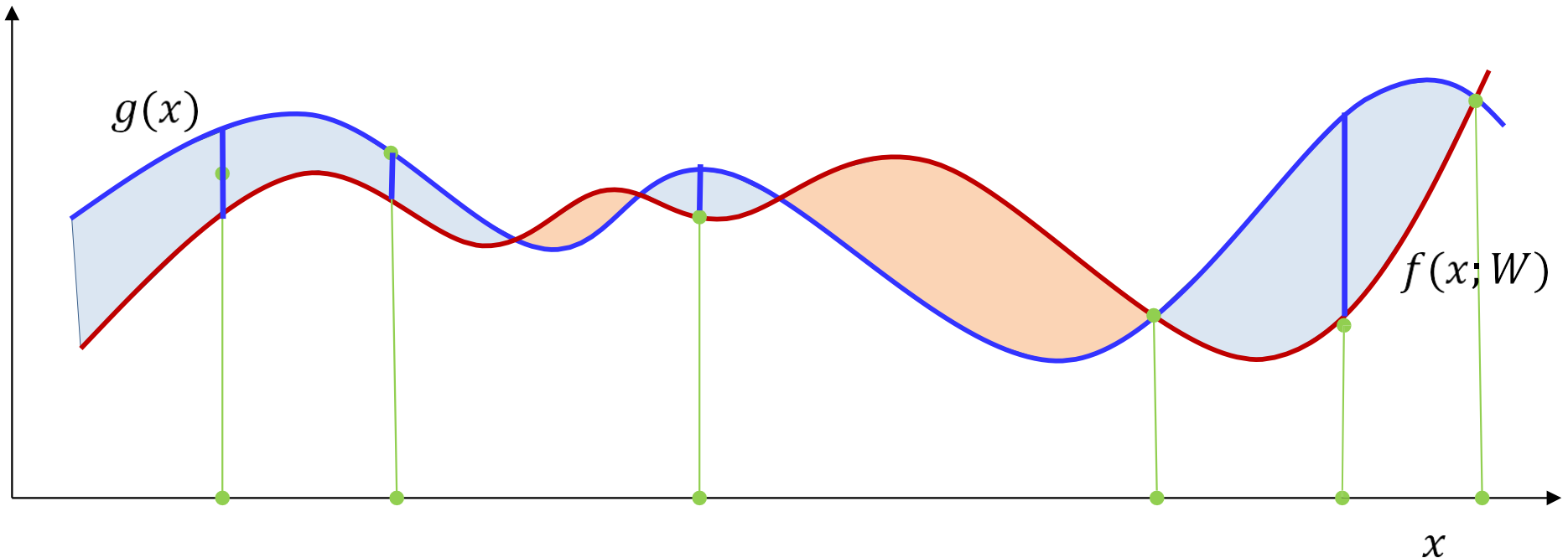
- The blue curve is the function being approximated
- The red curve is the approximation by the model at a given W
- The heights of the shaded regions represent the point-by-point error
 - The divergence is a function of the error
 - We want to find the W that minimizes the average divergence

Explaining the variance



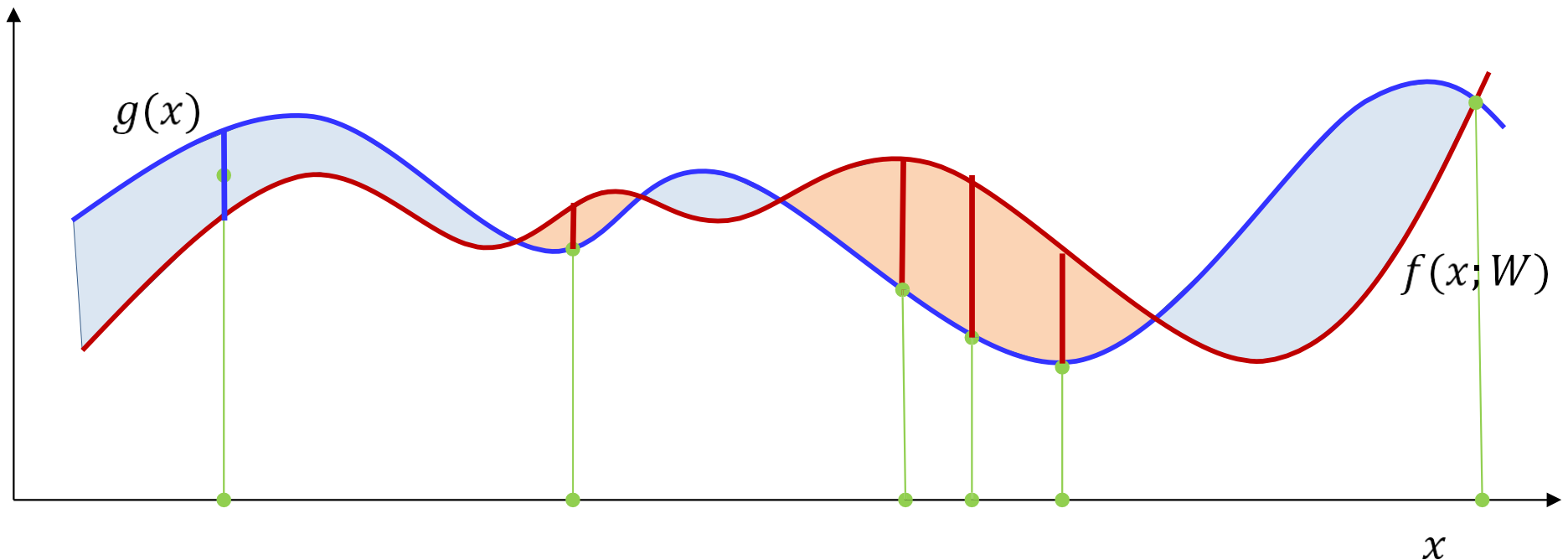
- Sample estimate approximates the shaded area with the average length of the error lines

Explaining the variance



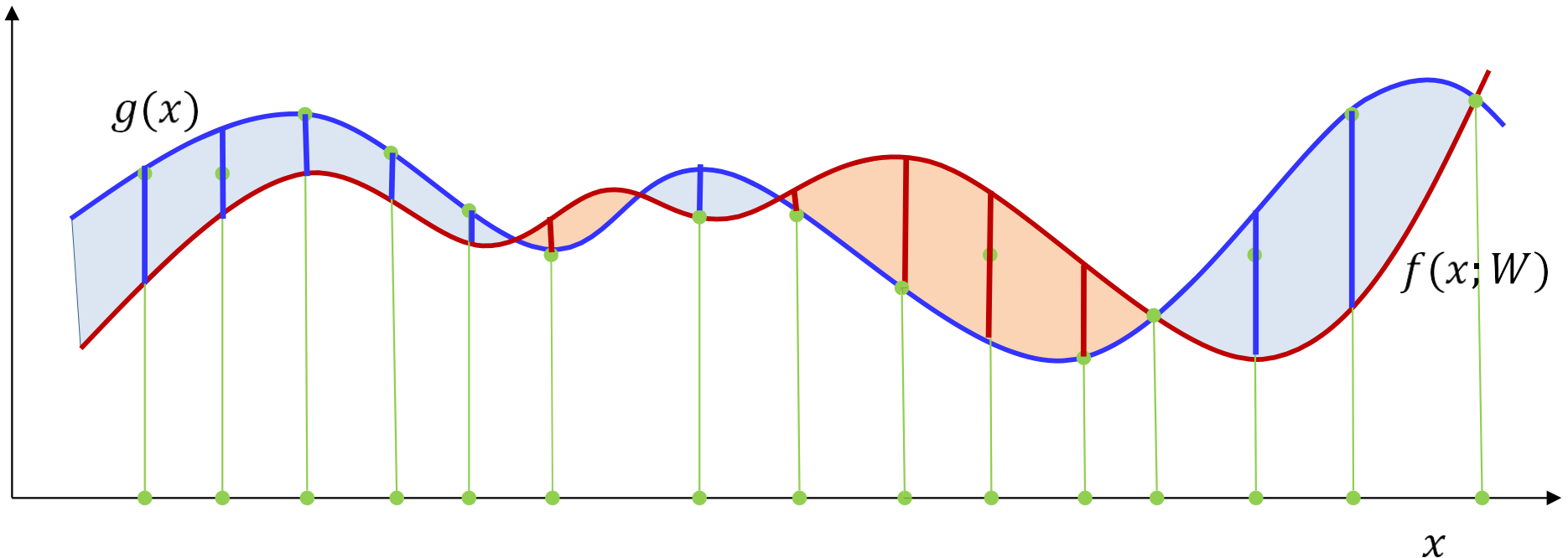
- Sample estimate approximates the shaded area with the average length of the error lines
- This average length will change with position of the samples

Explaining the variance



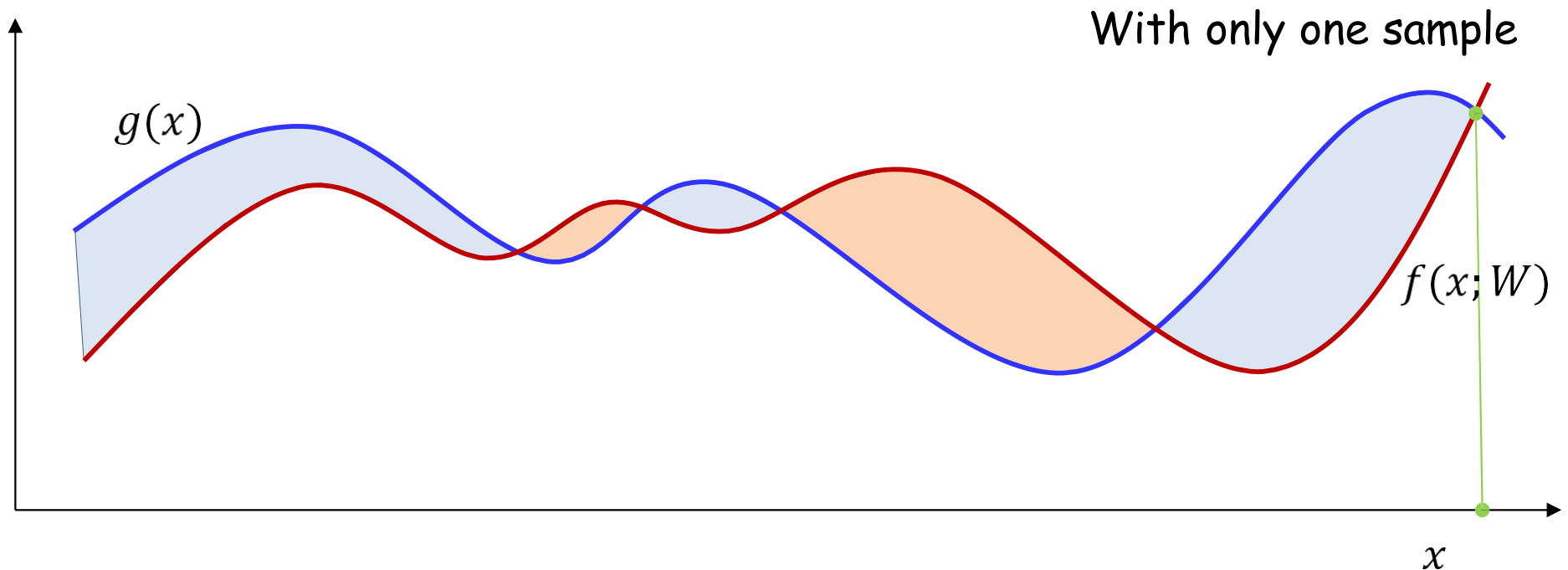
- Sample estimate approximates the shaded area with the average length of the error lines
- This average length will change with position of the samples

Explaining the variance



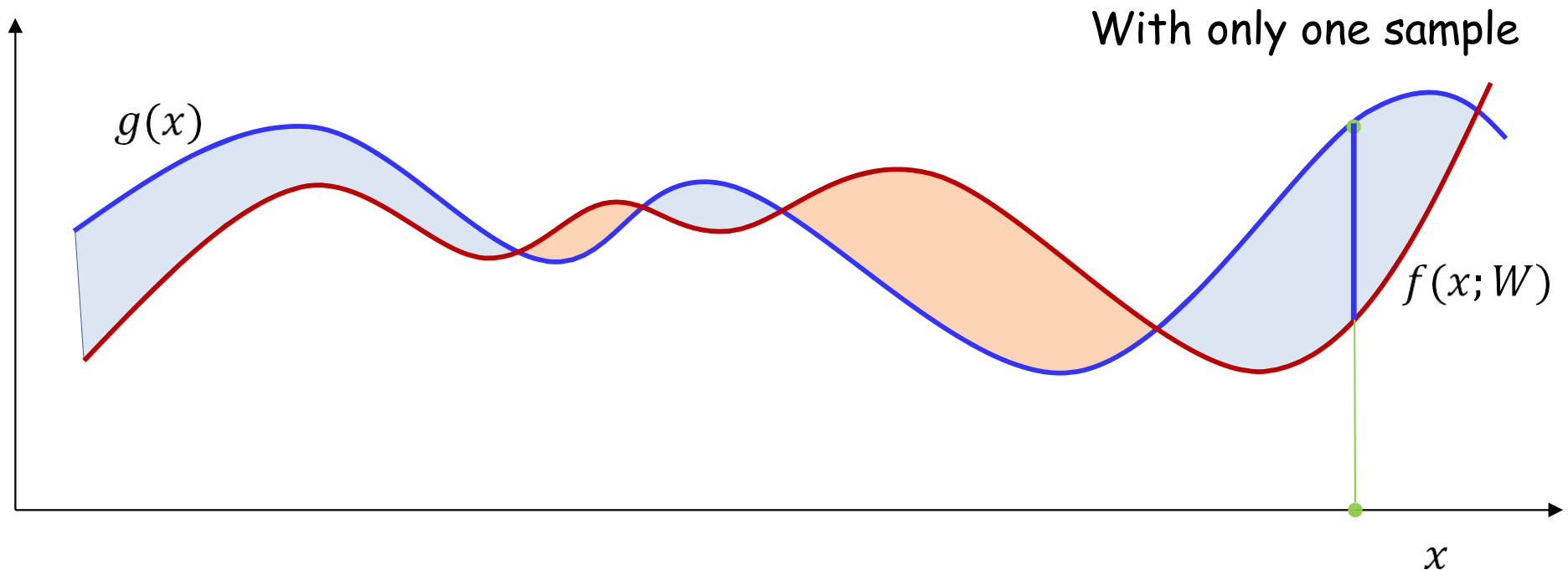
- Having more samples makes the estimate more robust to changes in the position of samples
 - The variance of the estimate is smaller

Explaining the variance



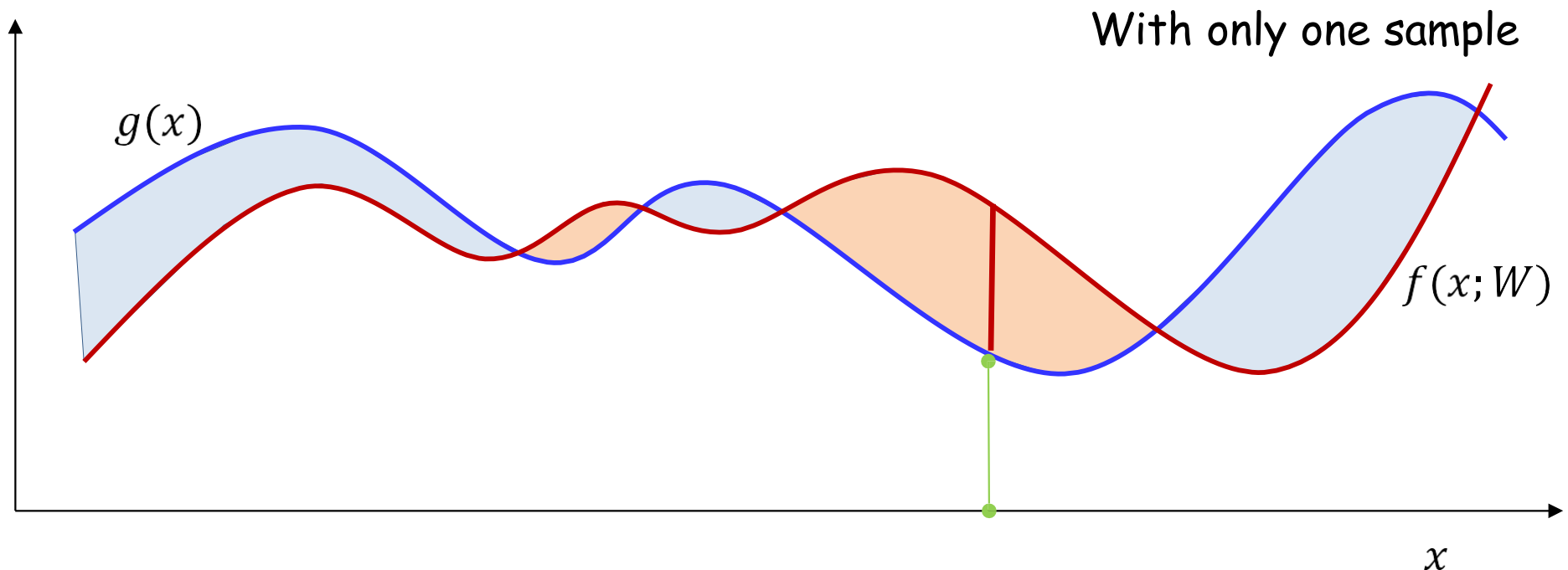
- Having very few samples makes the estimate swing wildly with the sample position
 - Since our estimator learns the W to minimize this estimate, the learned W too can swing wildly

Explaining the variance



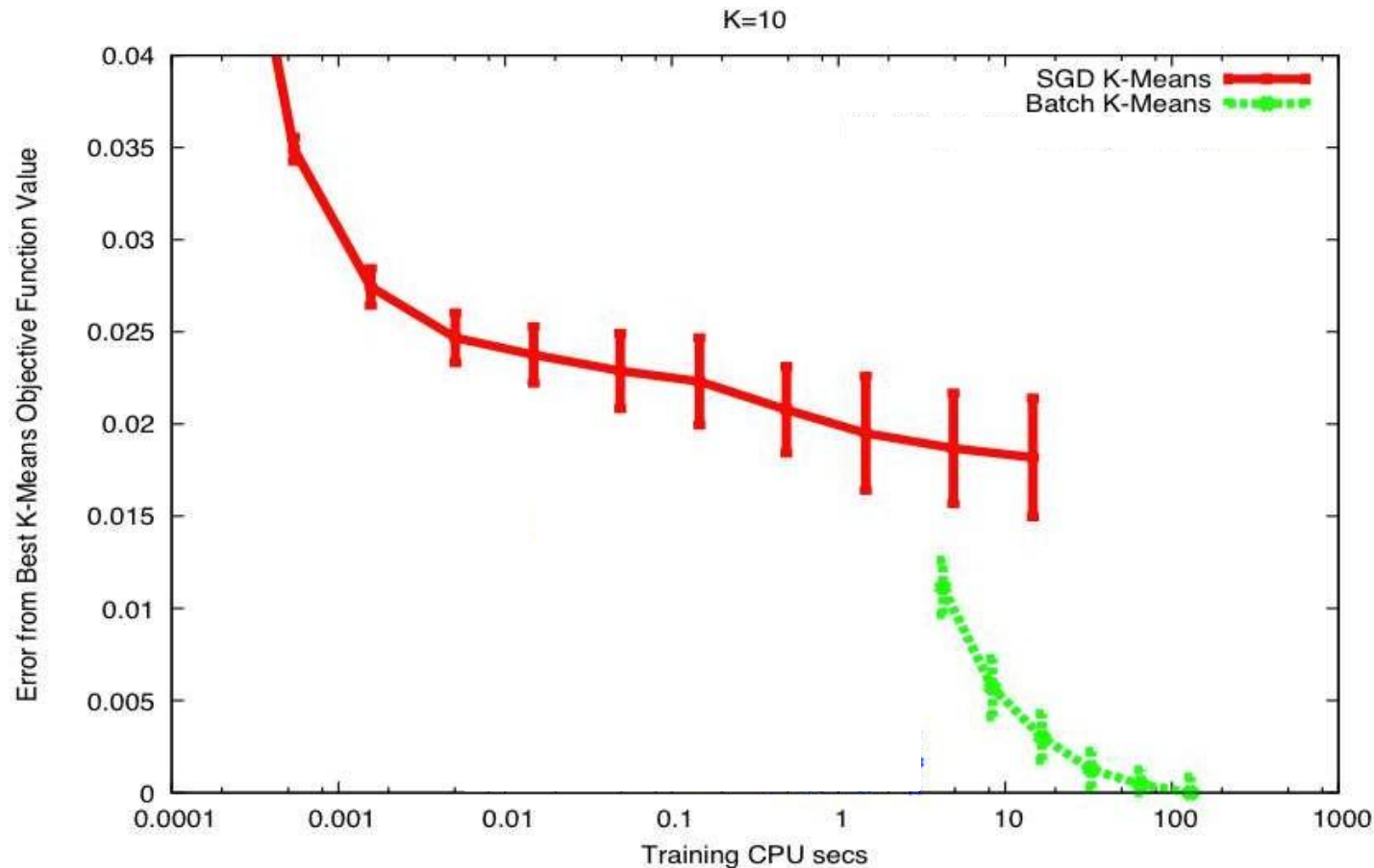
- Having very few samples makes the estimate swing wildly with the sample position
 - Since our estimator learns the W to minimize this estimate, the learned W too can swing wildly

Explaining the variance



- Having very few samples makes the estimate swing wildly with the sample position
 - Since our estimator learns the W to minimize this estimate, the learned W too can swing wildly

SGD example

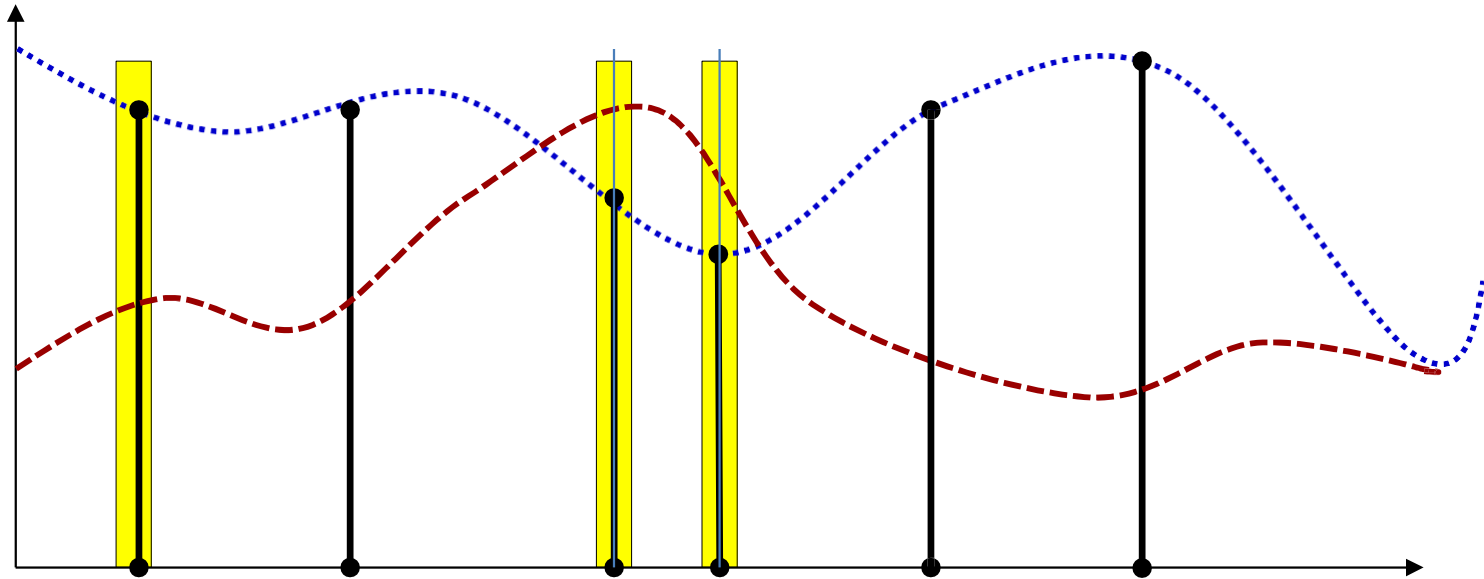


- A simpler problem: K-means
- Note: SGD converges faster
- But also has large variation between runs

SGD vs batch

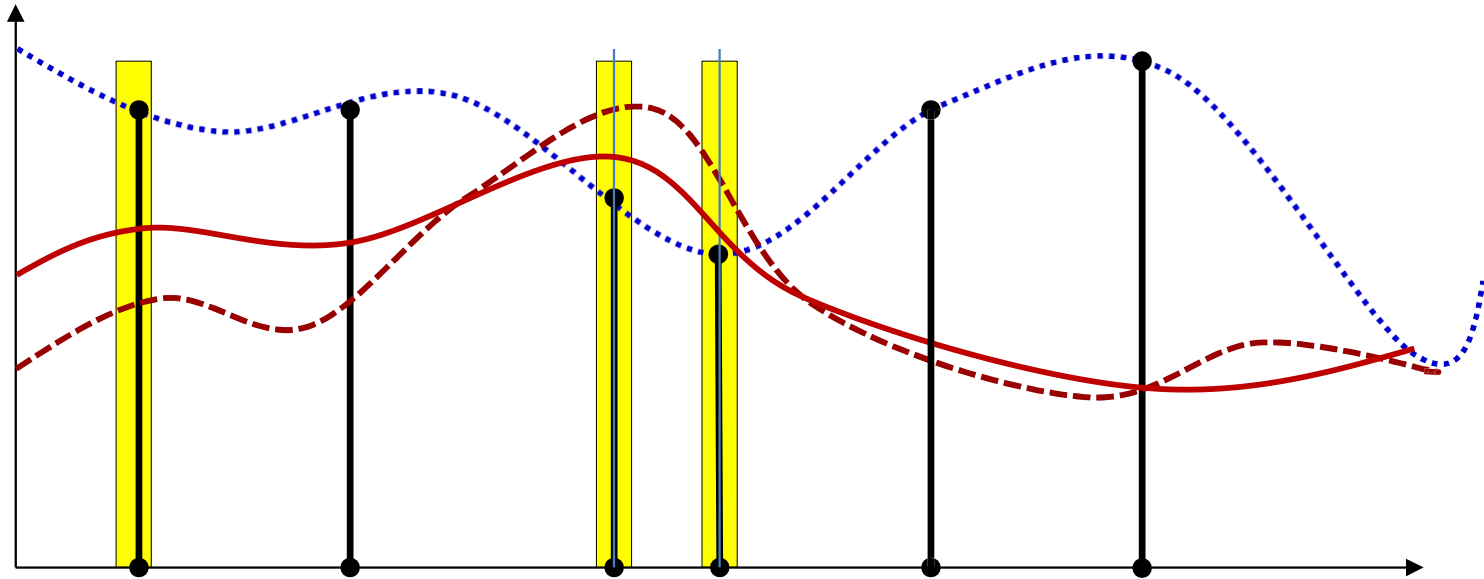
- SGD uses the gradient from only one sample at a time, and is consequently high variance
- But also provides significantly quicker updates than batch
- Is there a good medium?

Alternative: Mini-batch update



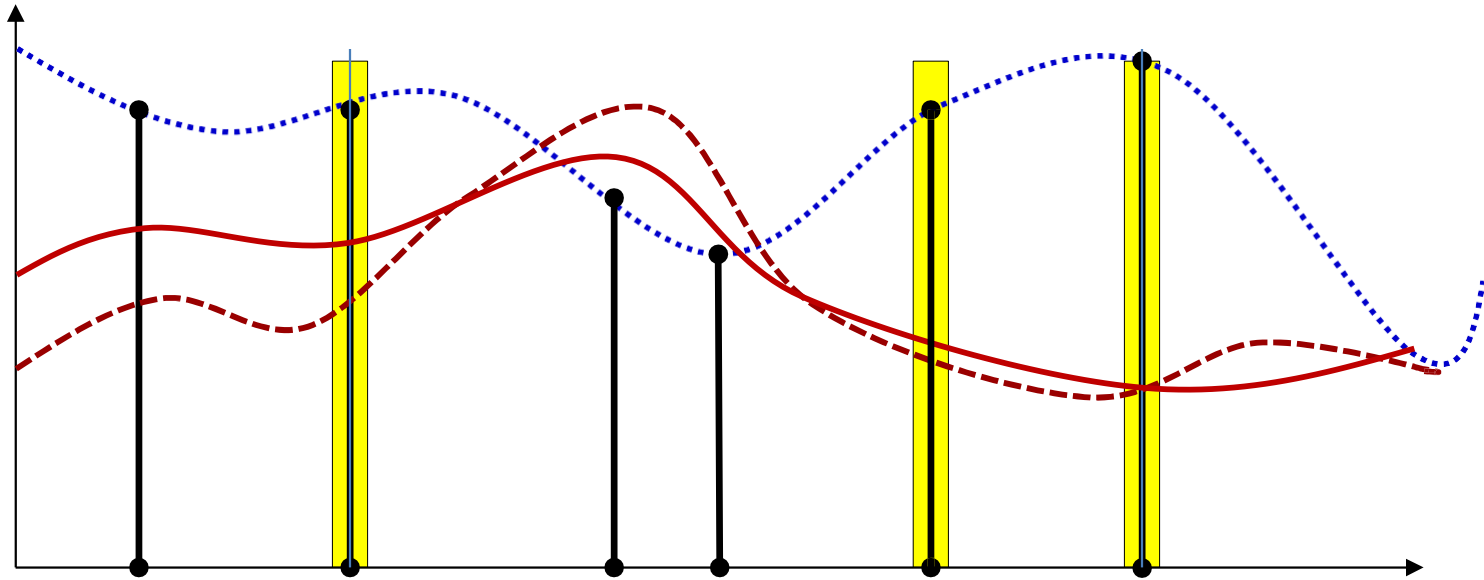
- Alternative: adjust the function at a small, randomly chosen subset of points
 - Keep adjustments small
 - If the subsets cover the training set, we will have adjusted the entire function
- As before, vary the subsets randomly in different passes through the training data

Alternative: Mini-batch update



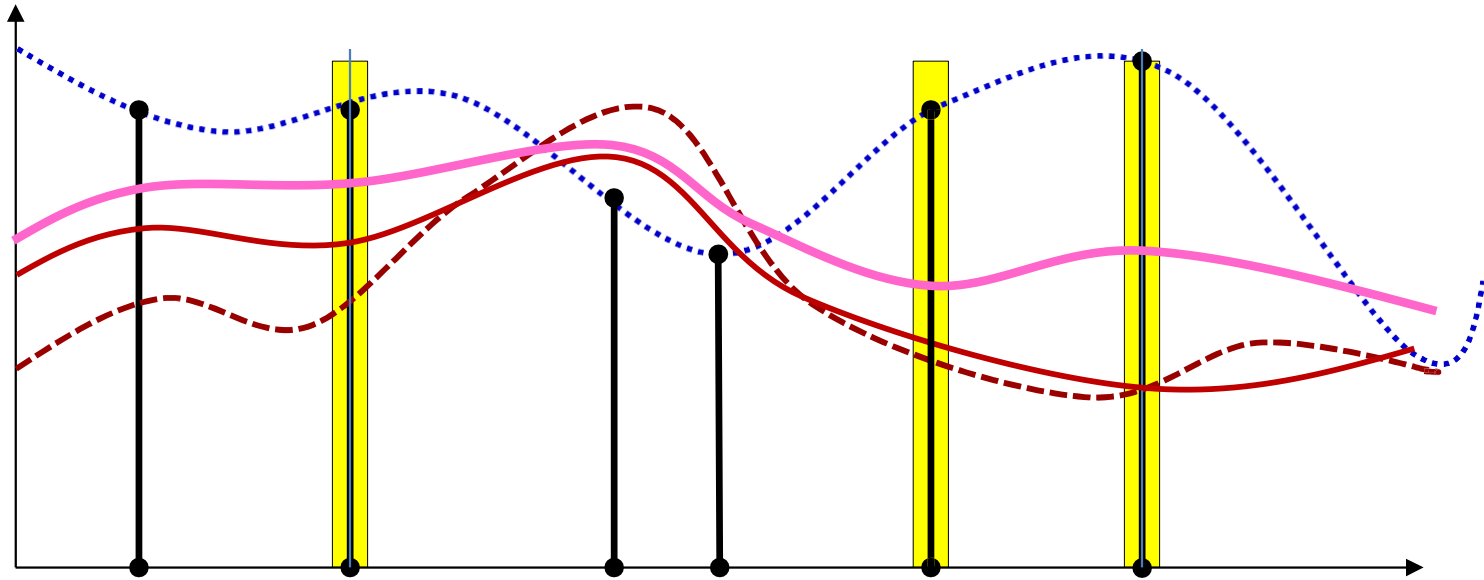
- Alternative: adjust the function at a small, randomly chosen subset of points
 - Keep adjustments small
 - If the subsets cover the training set, we will have adjusted the entire function
- As before, vary the subsets randomly in different passes through the training data

Alternative: Mini-batch update



- Alternative: adjust the function at a small, randomly chosen subset of points
 - Keep adjustments small
 - If the subsets cover the training set, we will have adjusted the entire function
- As before, vary the subsets randomly in different passes through the training data

Alternative: Mini-batch update



- Alternative: adjust the function at a small, randomly chosen subset of points
 - Keep adjustments small
 - If the subsets cover the training set, we will have adjusted the entire function
- As before, vary the subsets randomly in different passes through the training data

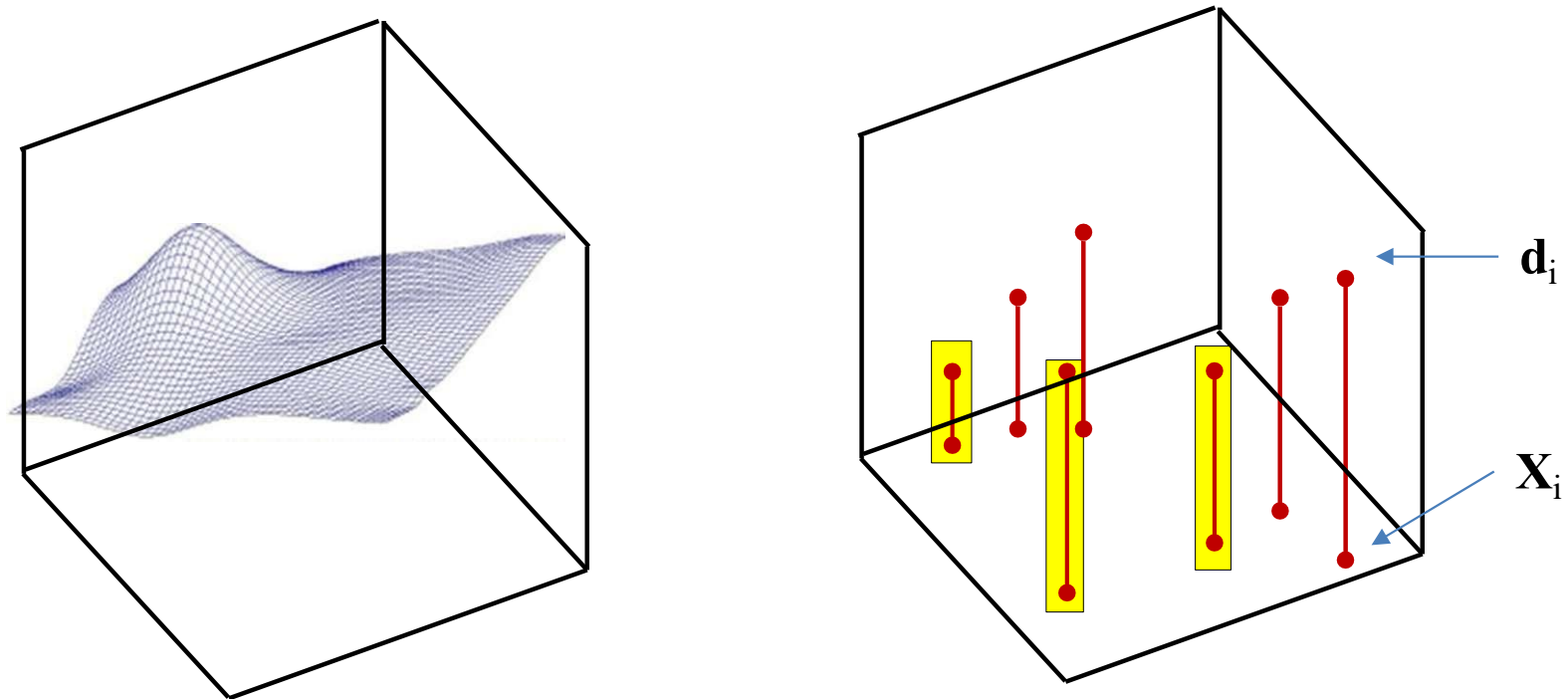
Incremental Update: Mini-batch update

- Given $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$
- Initialize all weights $W_1, W_2, \dots, W_K$; $j = 0$
- Do:
 - Randomly permute $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$
 - For $t = 1:b:T$
 - $j = j + 1$
 - For every layer k :
 - $\Delta W_k = 0$
 - For $t' = t : t+b-1$
 - For every layer k :
 - » Compute $\square_{W_k} \text{Div}(Y_{t'}, d_{t'})$
 - » $\Delta W_k = \Delta W_k + \frac{1}{b} \square_{W_k} \text{Div}(Y_{t'}, d_{t'})^\top$
 - Update
 - For every layer k :
$$W_k = W_k - \eta_j \Delta W_k$$- Until *Err* has converged

Incremental Update: Mini-batch update

- Given $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$
 - Initialize all weights $W_1, W_2, \dots, W_K$; $j = 0$
 - Do:
 - Randomly permute $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$
 - For $t = 1:b:T$
 - $j = j + 1$
 - For every layer k :
 - $\Delta W_k = 0$
 - For $t' = t : t+b-1$
 - For every layer k :
 - » Compute $\square_{W_k} \text{Div}(Y_{t'}, d_{t'})$
 - » $\Delta W_k = \Delta W_k + \frac{1}{b} \square_{W_k} \text{Div}(Y_{t'}, d_{t'})^\top$
 - Update
 - For every layer k :
$$W_k = W_k - \eta_j \Delta W_k$$
 - Until *Err* has converged
- Mini-batch size
- Shrinking step size

Mini Batches



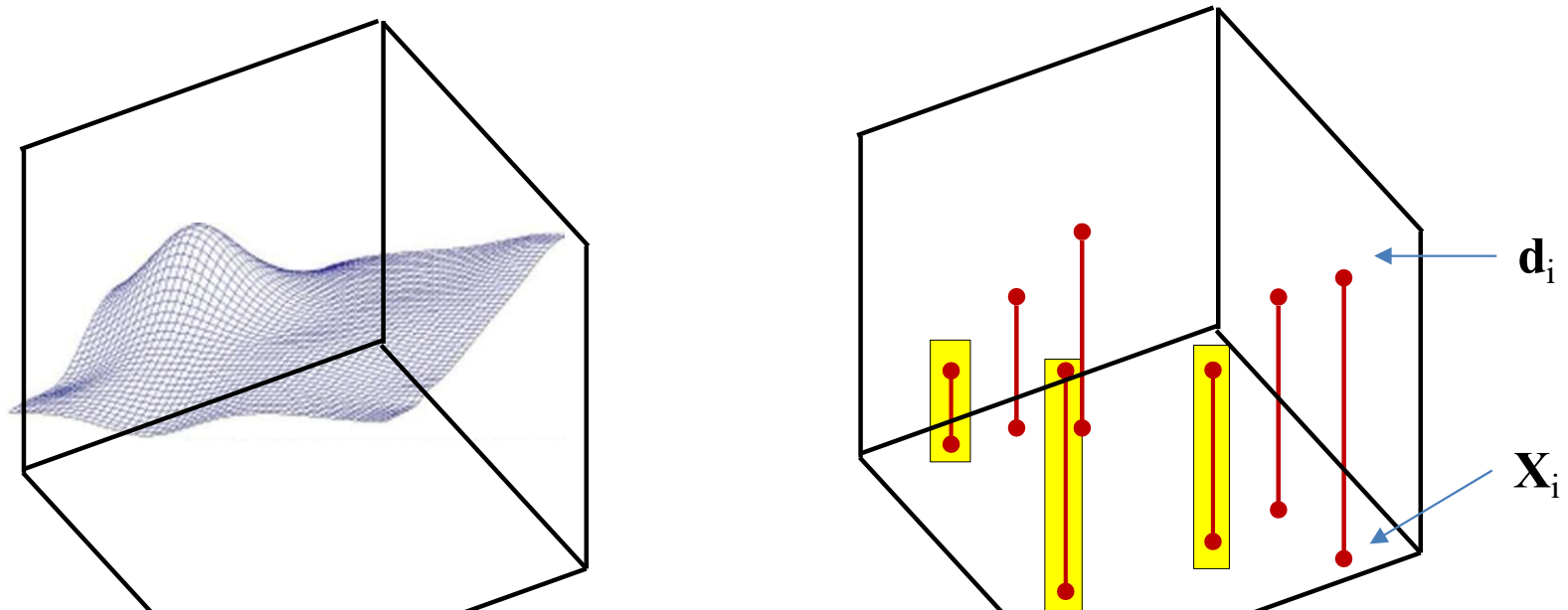
- Mini-batch updates compute and minimize a *batch loss*

$$\text{MiniBatchLoss}(W) = \frac{1}{b} \sum_{i=1}^b \text{div}(f(X_i; W), d_i)$$

- The *expected value* of the *batch loss* is also the *expected divergence*

$$E[\text{MiniBatchLoss}(W)] = E[\text{div}(f(X; W), g(X))]$$

Mini Batches



The minibatch loss is also an unbiased estimate of the expected loss

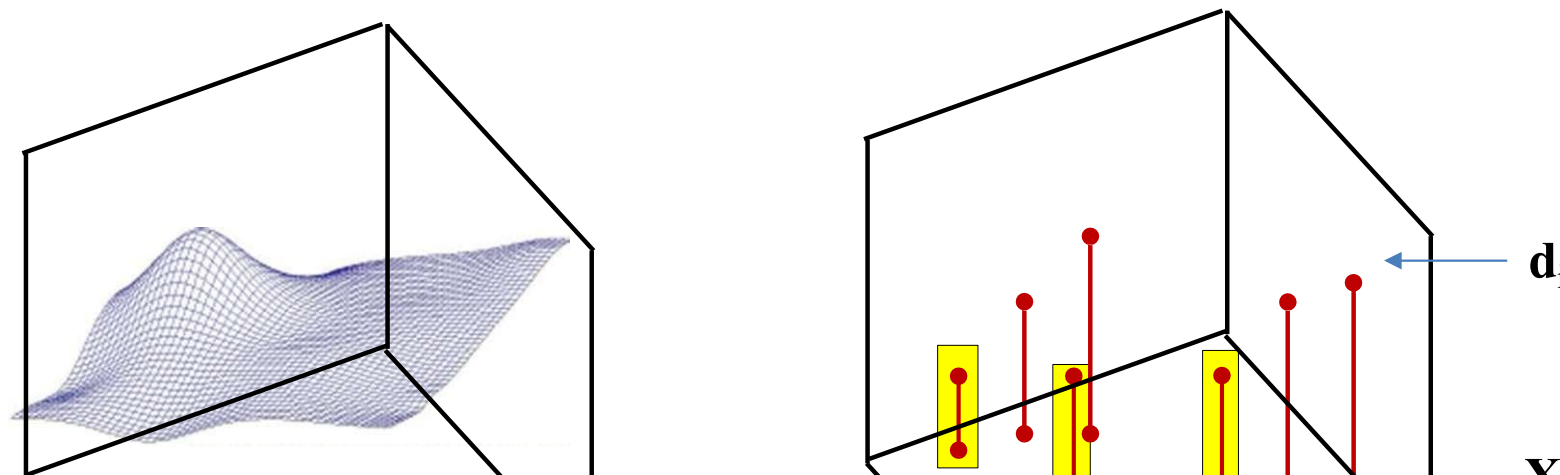
- Mini-batch updates compute and minimize a *batch loss*

$$\text{MiniBatchLoss}(W) = \frac{1}{b} \sum_{i=1}^b \text{div}(f(X_i; W), d_i)$$

- The *expected value* of the *batch loss* is also the *expected divergence*

$$E[\text{MiniBatchLoss}(W)] = E[\text{div}(f(X; W), g(X))]$$

Mini Batches



The variance of the minibatch loss: $\text{var}(\text{BatchLoss}) = 1/b \text{ var}(\text{div})$
This will be much smaller than the variance of the sample error in SGD

The minibatch loss is also an unbiased estimate of the expected error

- Mini-batch updates compute and minimize a *batch loss*

$$\text{MiniBatchLoss}(W) = \frac{1}{b} \sum_{i=1}^b \text{div}(f(X_i; W), d_i)$$

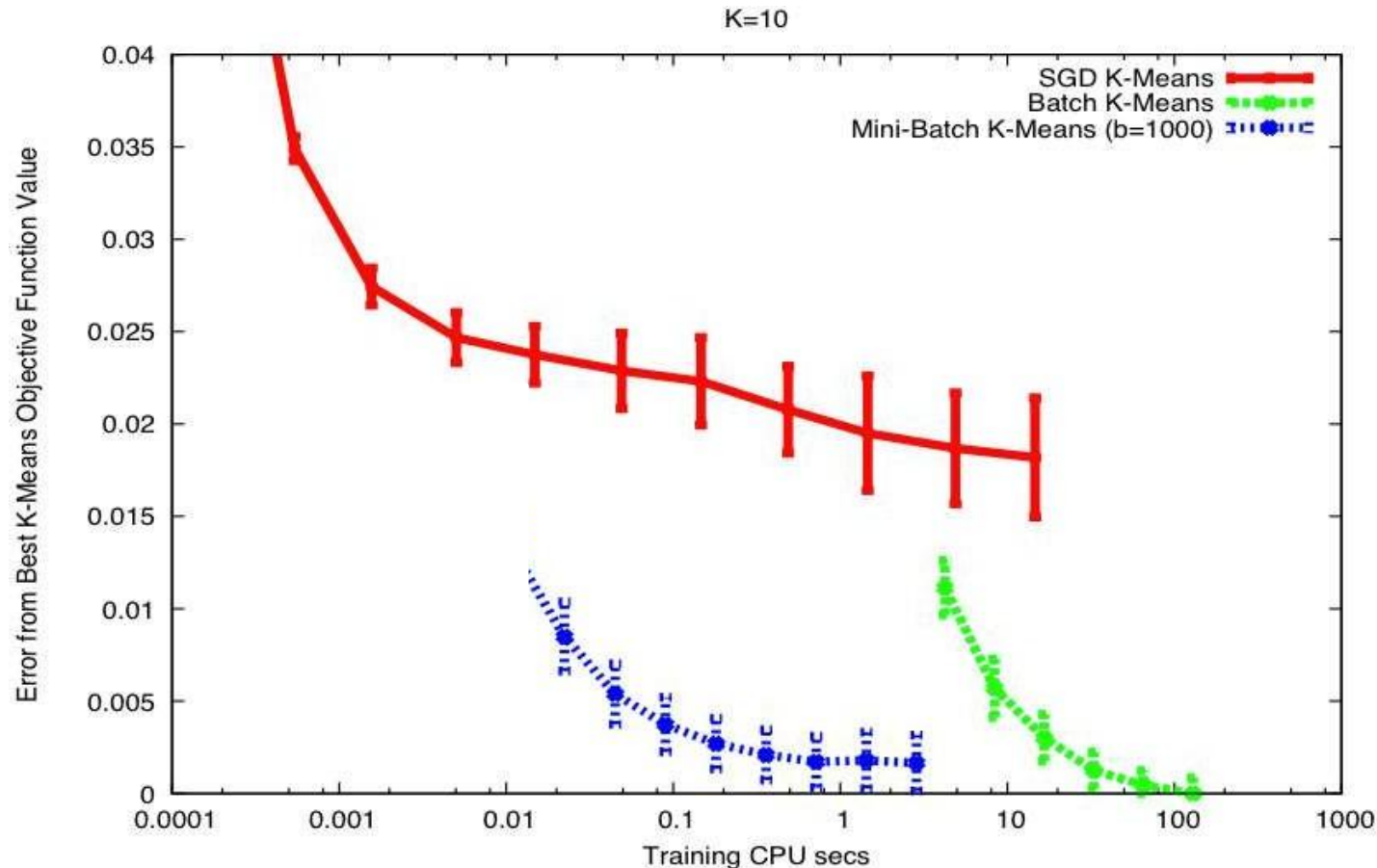
- The *expected value* of the *batch loss* is also the *expected divergence*

$$E[\text{MiniBatchLoss}(W)] = E[\text{div}(f(X; W), g(X))]$$

Minibatch convergence

- For convex functions, convergence rate for SGD is $\mathcal{O}\left(\frac{1}{\sqrt{k}}\right)$.
- For *mini-batch* updates with batches of size b , the convergence rate is $\mathcal{O}\left(\frac{1}{\sqrt{bk}} + \frac{1}{k}\right)$
 - Apparently an improvement of $\sqrt{b}$ over SGD
 - But since the batch size is b , we perform b times as many computations per iteration as SGD
 - We actually get a *degradation* of $\sqrt{b}$
- However, in practice
 - The objectives are generally not convex; mini-batches are more effective with the right learning rates
 - We also get additional benefits of vector processing

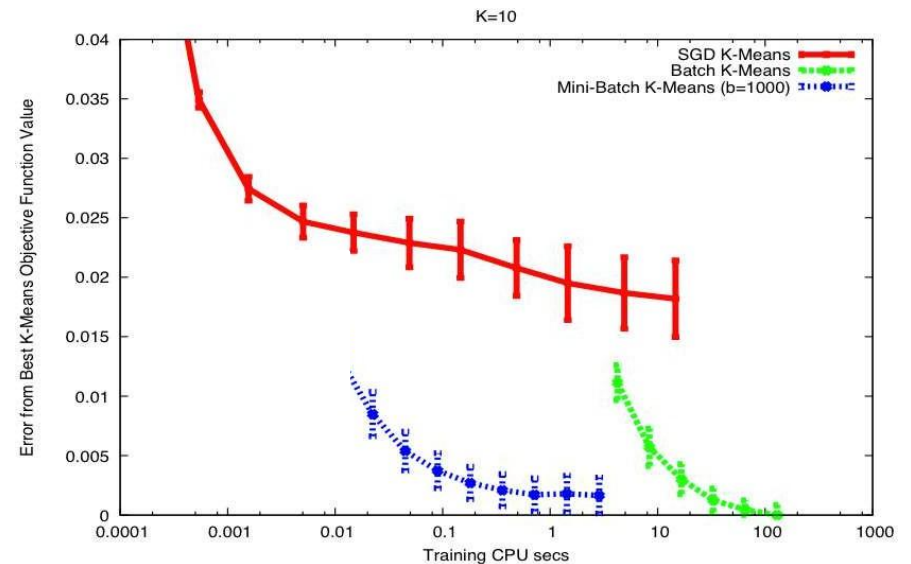
SGD example



- Mini-batch performs comparably to batch training on this simple problem
 - But converges orders of magnitude faster

Measuring Loss

- Convergence is generally defined in terms of the *overall training loss*
 - Not sample or batch loss



- Infeasible to actually measure the overall training loss after each iteration
- More typically, we estimate is as
 - Divergence or classification error on a held-out set
 - Average sample/batch loss over the past N samples/batches

Training and minibatches

- In practice, training is usually performed using mini-batches
 - The mini-batch size is generally set to the largest that your hardware will support (in memory) without compromising overall compute time
 - Larger minibatches = less variance
 - Larger minibatches = few updates per epoch
- Convergence depends on learning rate
 - Simple technique: fix learning rate until the error plateaus, then reduce learning rate by a fixed factor (e.g. 10)
 - ***Advanced methods***: Adaptive updates, where the learning rate is itself determined as part of the estimation

Story so far

- SGD: Presenting training instances one-at-a-time can be more effective than full-batch training
 - Provided they are provided in random order
- For SGD to converge, the learning rate must shrink sufficiently rapidly with iterations
 - Otherwise the learning will continuously “chase” the latest sample
- SGD estimates have higher variance than batch estimates
- Minibatch updates operate on *batches* of instances at a time
 - Estimates have lower variance than SGD
 - Convergence rate is theoretically worse than SGD
 - But we compensate by being able to perform batch processing

Training and minibatches

- Convergence depends on learning rate
 - Simple technique: fix learning rate until the error plateaus, then reduce learning rate by a fixed factor (e.g. 10)
 - ***Advanced methods:*** Adaptive updates, where the learning rate is itself determined as part of the estimation

Moving on: Topics for the day

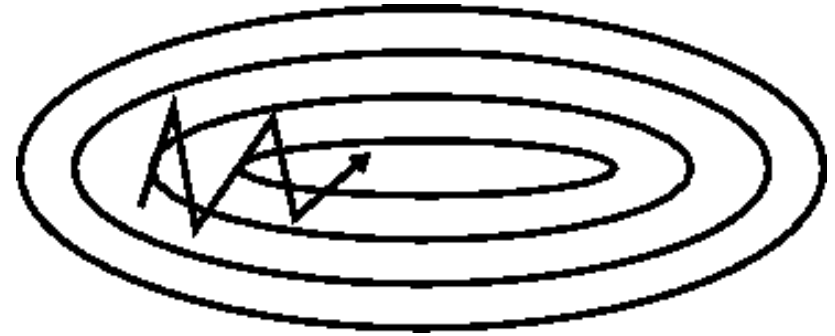
- Incremental updates
- Revisiting “trend” algorithms
- Generalization
- Tricks of the trade
 - Divergences..
 - Activations
 - Normalizations

Recall: Momentum Update

Plain gradient update



With momentum

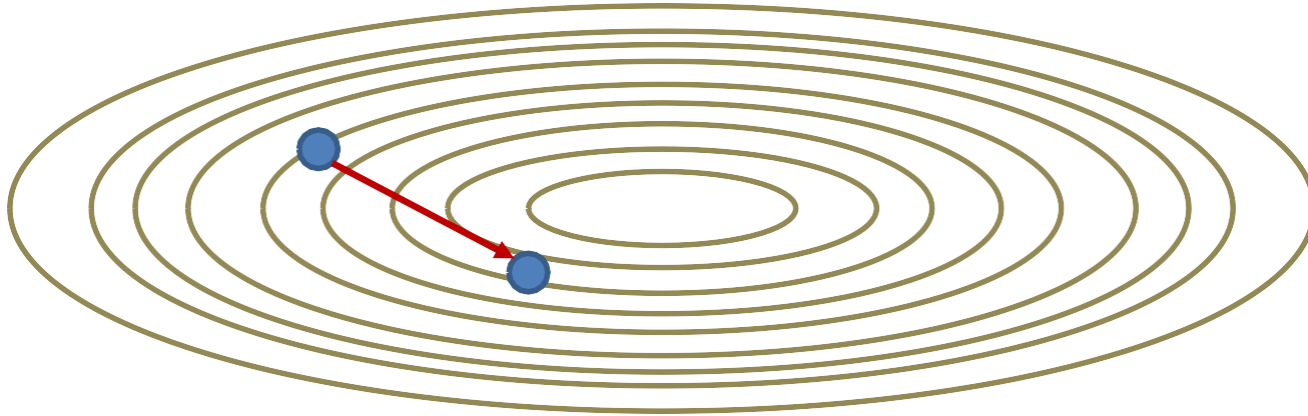


- The momentum method maintains a running average of all gradients until the *current* step

$$\Delta W^{(k)} = \beta \Delta W^{(k-1)} - \eta \nabla_W \text{Loss}(W^{(k-1)})^T$$
$$W^{(k)} = W^{(k-1)} + \Delta W^{(k)}$$

- Typical β value is 0.9
- The running average steps
 - Get longer in directions where gradient retains the same sign
 - Become shorter in directions where the sign keeps flipping

Recall: Momentum Update

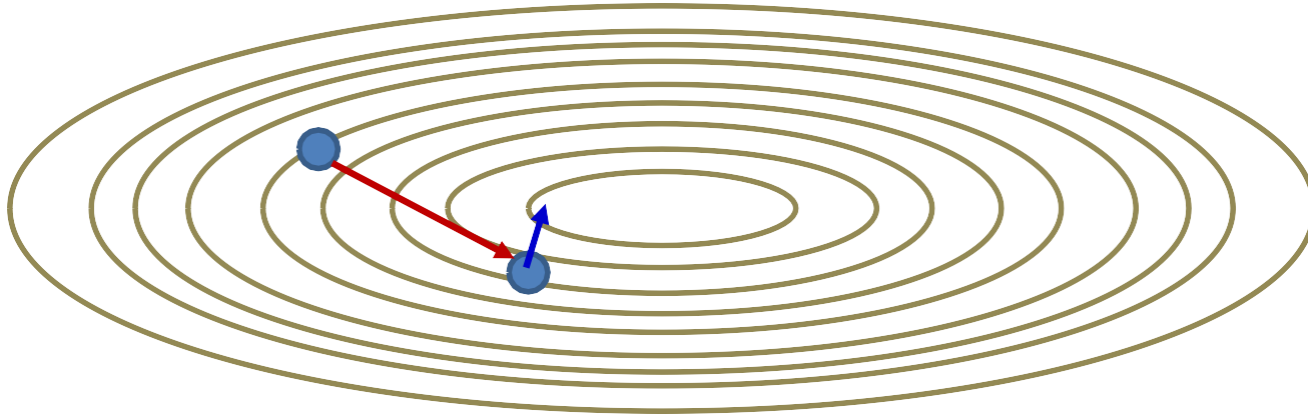


- The momentum method

$$\Delta W^{(k)} = \beta \Delta W^{(k-1)} - \eta \nabla_W \text{Loss}(W^{(k-1)})^T$$

- At any iteration, to compute the current step:

Recall: Momentum Update

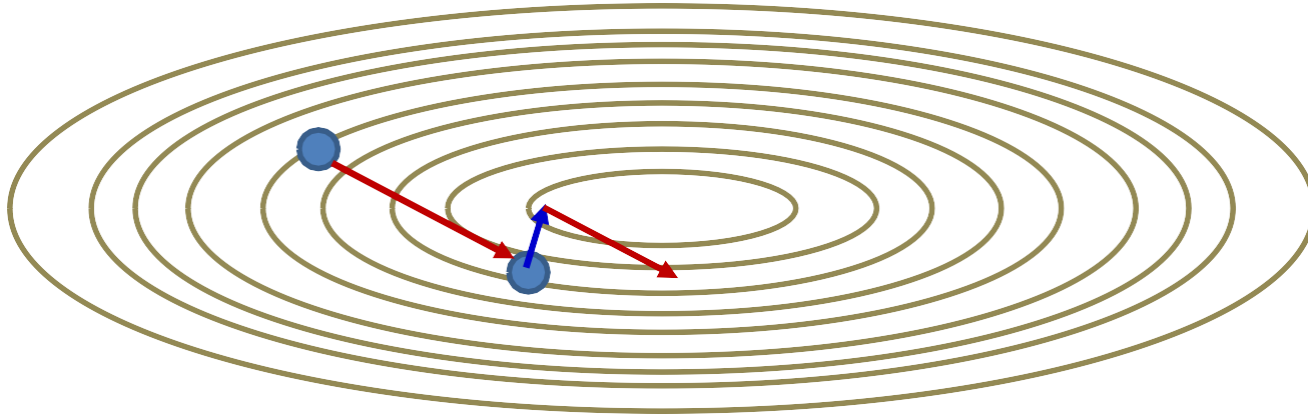


- The momentum method

$$\Delta W^{(k)} = \beta \Delta W^{(k-1)} - \eta \nabla_W \text{Loss}(W^{(k-1)})^T$$

- At any iteration, to compute the current step:
 - First compute the gradient step at the current location

Recall: Momentum Update

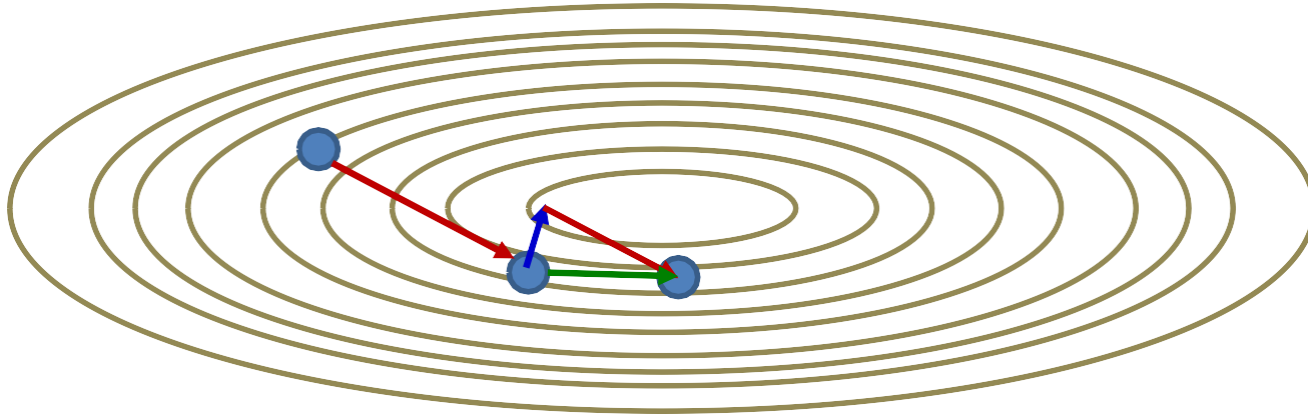


- The momentum method

$$\Delta W^{(k)} = \beta \Delta W^{(k-1)} - \eta \nabla_W \text{Loss}(W^{(k-1)})^T$$

- At any iteration, to compute the current step:
 - First compute the gradient step at the current location
 - Then add in the scaled *previous* step
 - Which is actually a running average

Recall: Momentum Update

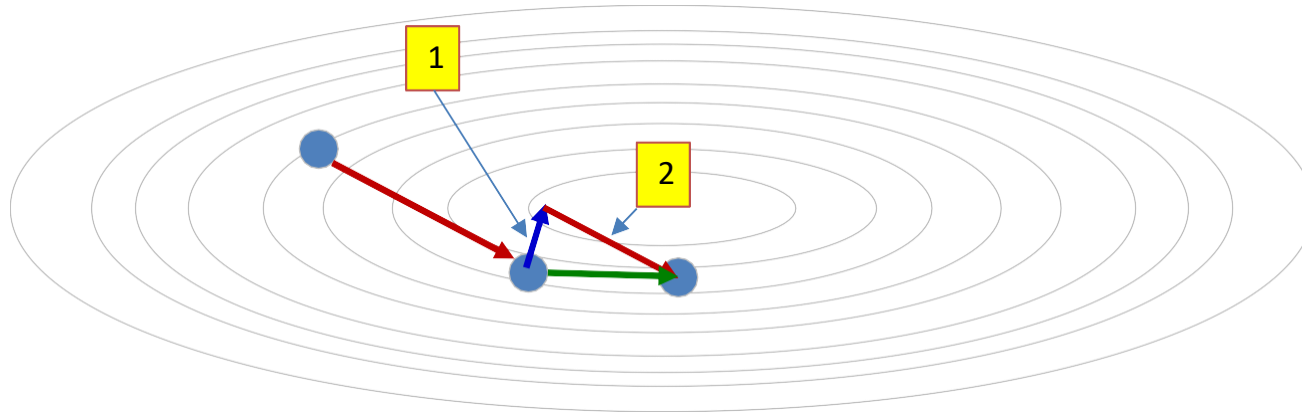


- The momentum method

$$\Delta W^{(k)} = \beta \Delta W^{(k-1)} - \eta \nabla_W \text{Loss}(W^{(k-1)})^T$$

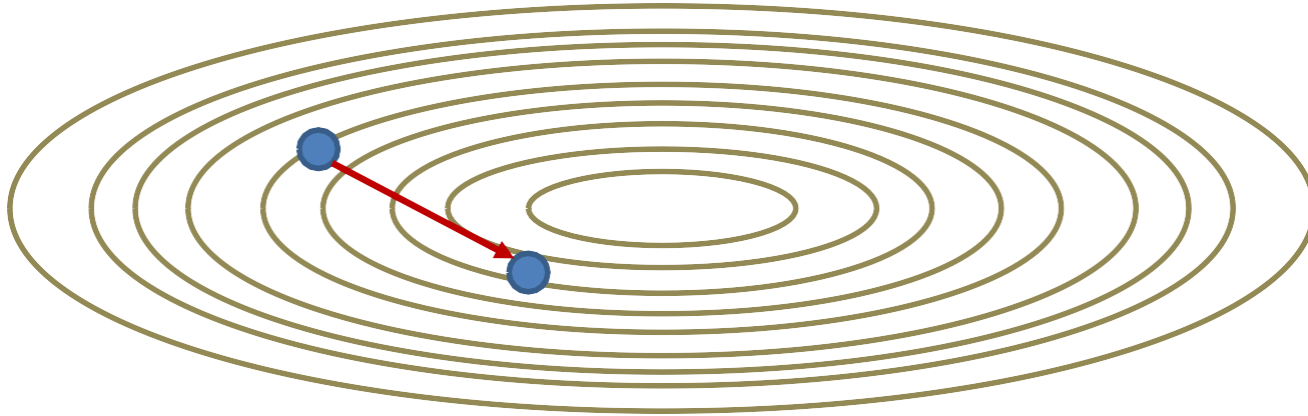
- At any iteration, to compute the current step:
 - First compute the gradient step at the current location
 - Then add in the scaled *previous* step
 - Which is actually a running average
 - To get the final step

Momentum update



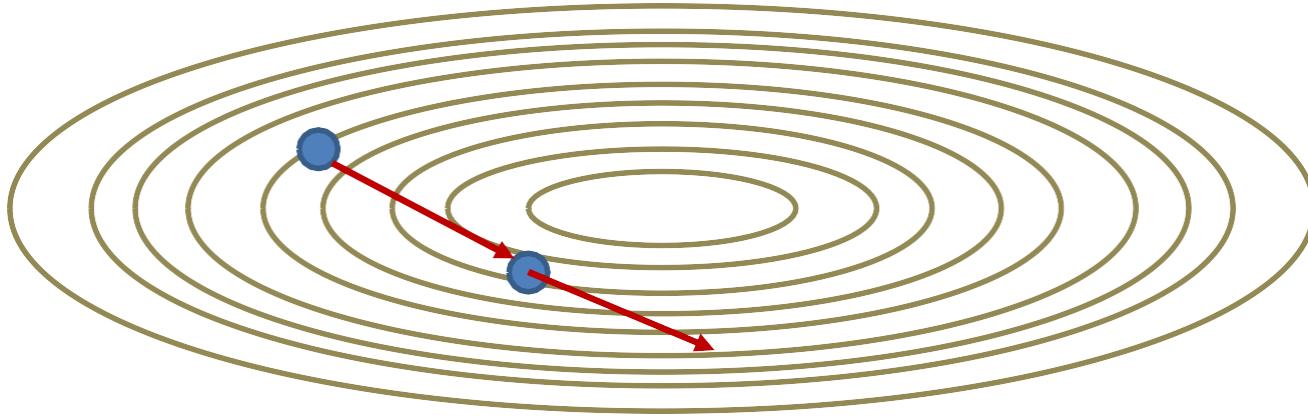
- Momentum update steps are actually computed in two stages
 - First: We take a step against the gradient at the current location
 - Second: Then we add a scaled version of the previous step
- The procedure can be made more optimal by reversing the order of operations..

Nestorov's Accelerated Gradient



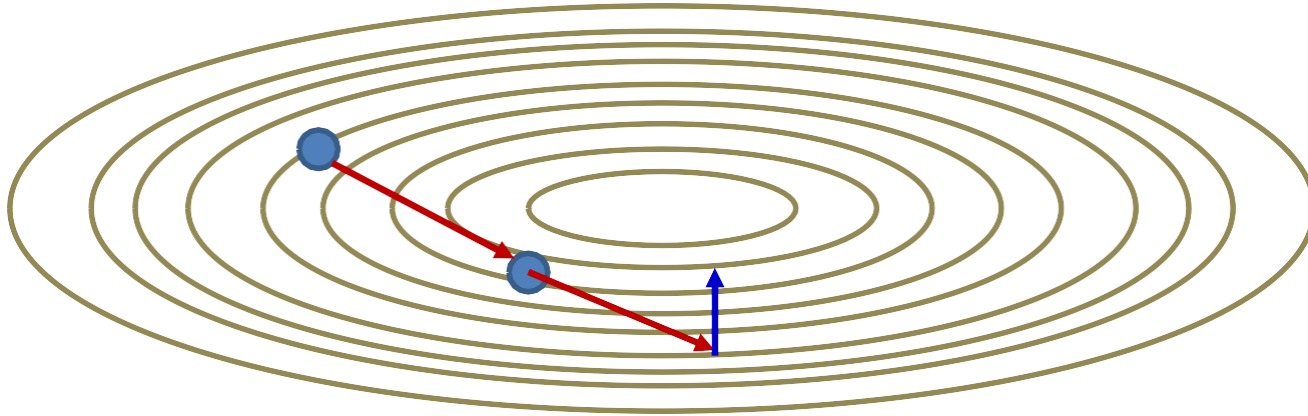
- Change the order of operations
- At any iteration, to compute the current step:

Nestorov's Accelerated Gradient



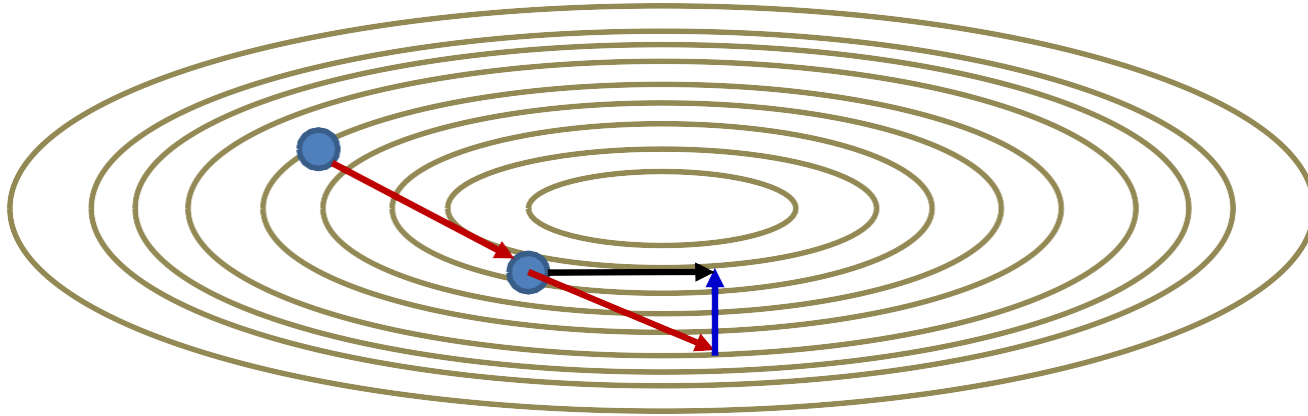
- Change the order of operations
- At any iteration, to compute the current step:
 - First extend the previous step

Nestorov's Accelerated Gradient



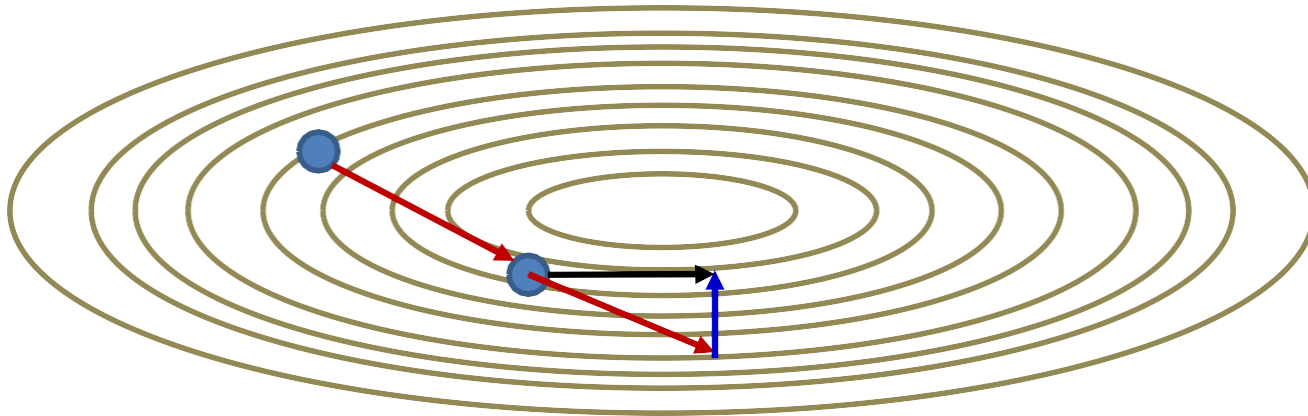
- Change the order of operations
- At any iteration, to compute the current step:
 - First extend the previous step
 - Then compute the gradient step at the resultant position

Nestorov's Accelerated Gradient



- Change the order of operations
- At any iteration, to compute the current step:
 - First extend the previous step
 - Then compute the gradient step at the resultant position
 - Add the two to obtain the final step

Nestorov's Accelerated Gradient

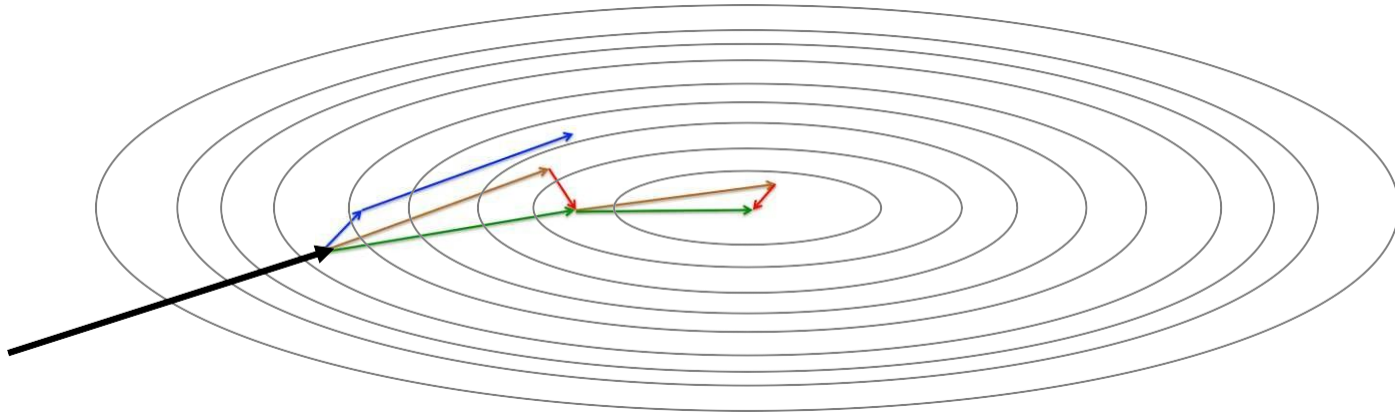


- Nestorov's method

$$\Delta W^{(k)} = \beta \Delta W^{(k-1)} - \eta \nabla_W \text{Loss}(W^{(k-1)} + \beta \Delta W^{(k-1)})^T$$

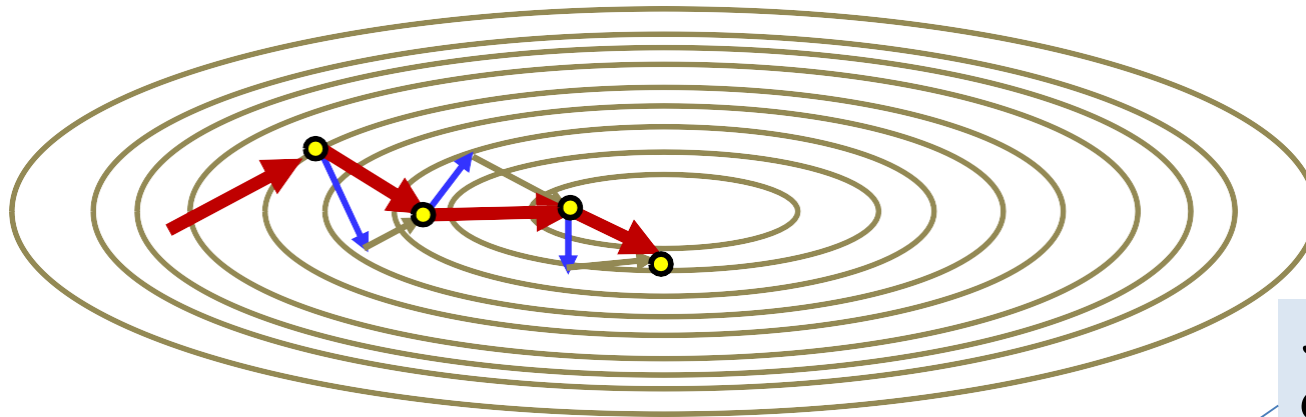
$$W^{(k)} = W^{(k-1)} + \Delta W^{(k)}$$

Nestorov's Accelerated Gradient



- Comparison with momentum (example from Hinton)
- Converges much faster

Momentum and incremental updates



SGD instance
or minibatch
loss

- The momentum method

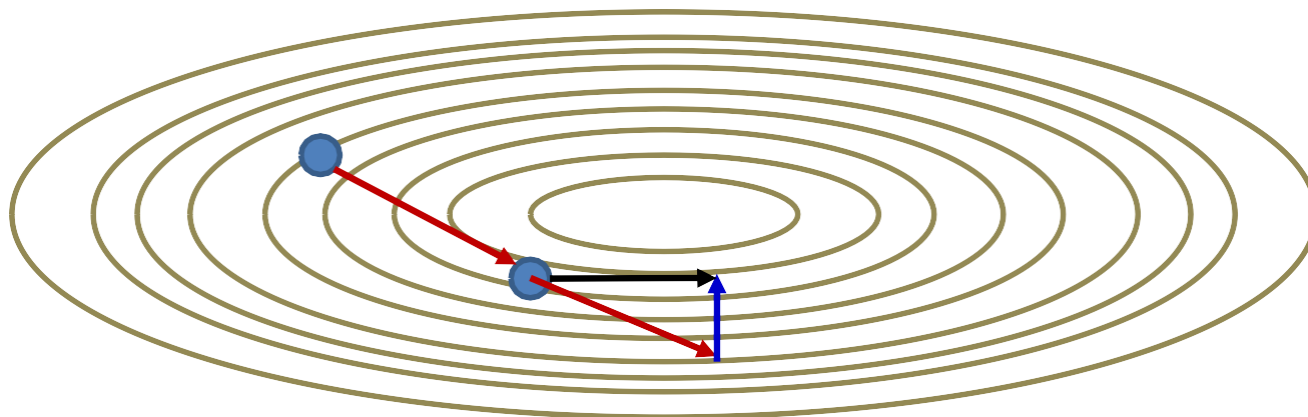
$$\Delta W^{(k)} = \beta \Delta W^{(k-1)} - \eta \nabla_W \text{Loss}(W^{(k-1)})^T$$

- Incremental SGD and mini-batch gradients tend to have high variance
- Momentum smooths out the variations
 - Smoother and faster convergence

Momentum: Mini-batch update

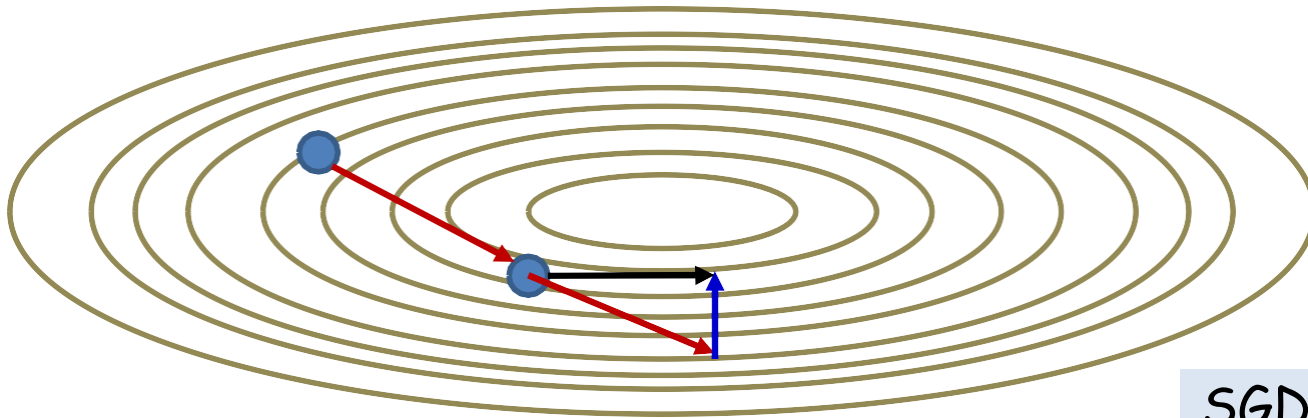
- Given $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$
- Initialize all weights $W_1, W_2, \dots, W_K$; $j = 0, \Delta W_k = 0$
- Do:
 - Randomly permute $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$
 - For $t = 1:b:T$
 - $j = j + 1$
 - For every layer k :
 - $\square_{W_k} Loss = 0$
 - For $t' = t : t+b-1$
 - For every layer k :
 - » Compute $\square_{W_k} Div(Y_{t'}, d_{t'})$
 - » $\square_{W_k} Loss += \frac{1}{b} \square_{W_k} Div(Y_{t'}, d_{t'})$
 - Update
 - For every layer k :
 - $\Delta W_k = \beta \Delta W_k - 5_j (\square_{W_k} Loss)^T$
 - $W_k = W_k + \Delta W_k$
- Until $Loss$ has converged

Nestorov's Accelerated Gradient



- At any iteration, to compute the current step:
 - First extend the previous step
 - Then compute the gradient at the resultant position
 - Add the two to obtain the final step
- This also applies directly to incremental update methods
 - The accelerated gradient smooths out the variance in the gradients

Nestorov's Accelerated Gradient



- Nestorov's method

$$\Delta W^{(k)} = \beta \Delta W^{(k-1)} - \eta \nabla_W \text{Loss}(W^{(k-1)} + \beta \Delta W^{(k-1)})^T$$

$$W^{(k)} = W^{(k-1)} + \Delta W^{(k)}$$

Nestorov: Mini-batch update

- Given $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$
- Initialize all weights $W_1, W_2, \dots, W_K$; $j = 0, \Delta W_k = 0$
- Do:
 - Randomly permute $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$
 - For $t = 1:b:T$
 - $j = j + 1$
 - For every layer k :
 - $W_k = W_k + \beta \Delta W_k$
 - $\square_{W_k} Loss = 0$
 - For $t' = t : t+b-1$
 - For every layer k :
 - » Compute $\square_{W_k} Div(Y_{t'}, d_{t'})$
 - » $\square_{W_k} Loss += \frac{1}{b} \square_{W_k} Div(Y_{t'}, d_{t'})$
 - Update
 - For every layer k :
 - $W_k = W_k - 5_j \square_{W_k} Loss^T$
 - $\Delta W_k = \beta \Delta W_k - 5_j \square_{W_k} Loss^T$
- Until $Loss$ has converged

The other term in the update

- Standard gradient descent rule

$$W \leftarrow W - \eta \nabla_W L(W)$$

- Gradient descent invokes two terms for updates
 - The derivative
 - and the learning rate

The other term in the update

- Standard gradient descent rule

$$W \leftarrow W - \eta \nabla_W L(W)$$

- Gradient descent invokes two terms for updates
 - The derivative
 - and the learning rate
- Momentum methods fix this term to reduce unstable oscillation

The other term in the update

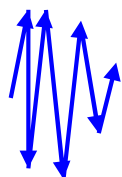
- Standard gradient descent rule

$$W \leftarrow W - \eta \nabla_W L(W)$$

- Gradient descent invokes two terms for updates
 - The derivative
 - and the learning rate
- Momentum methods fix this term to reduce unstable oscillation
- What about this term?

Adjusting the learning rate

Sequence of
gradients



vs



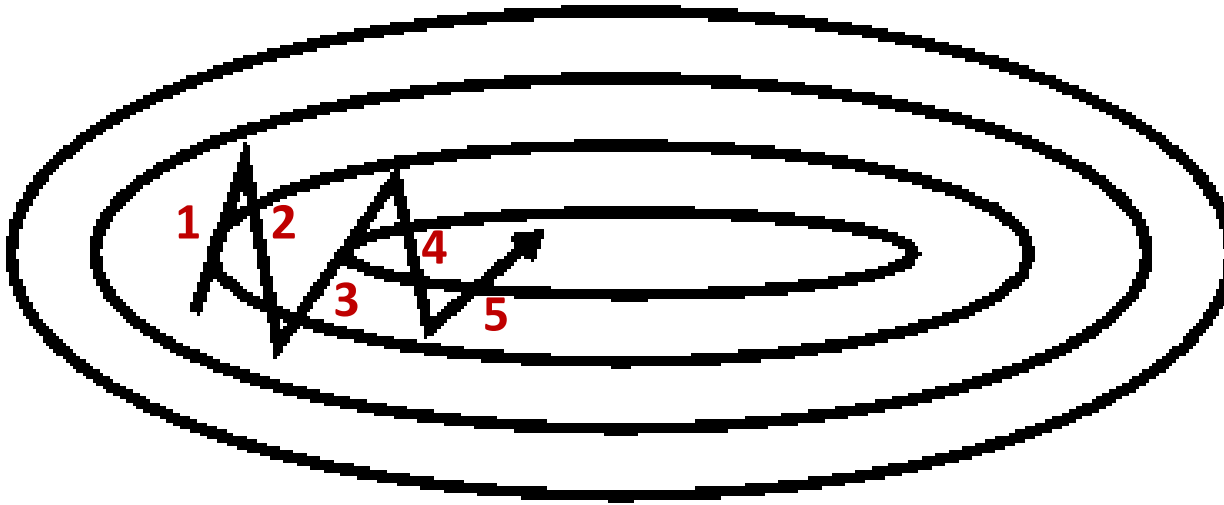
vs



With separate learning rates
in each direction, which should
have the lowest learning rate
in the vertical direction?

- Have separate learning rates for each component
- Directions in which the derivatives swing more should likely have lower learning rates
 - Is likely indicative of more wildly swinging behavior
- Directions of greater swing are indicated by total movement
 - Direction of greater movement should have lower learning rate

Smoothing the trajectory



Step	X component	Y component
1	1	+2.5
2	1	-3
3	2	+2.5
4	1	-2
5	1.5	1.5

- Observation: Steps in “oscillatory” directions show large total movement
 - In the example, total motion in the vertical direction is much greater than in the horizontal direction
- Solution: Lower learning rate in the vertical direction than in the horizontal direction
 - Based on total motion
 - As quantified by RMS value

RMS Prop

- Notation:
 - Formulae are *by parameter*
 - Derivative of loss w.r.t any individual parameter w is shown as $\partial_w D$
 - Batch or minibatch loss, or individual divergence for batch/minibatch/SGD
 - The *squared* derivative is $\partial_w^2 D = (\partial_w D)^2$
 - Short-hand notation represents the squared derivative, not the second derivative
 - The *mean squared* derivative is a running estimate of the average squared derivative. We will show this as $E[\partial_w^2 D]$
- Modified update rule: We want to
 - scale down learning rates for terms with large mean squared derivatives
 - scale up learning rates for terms with small mean squared derivatives

RMS Prop

- This is a variant on the *basic* mini-batch SGD algorithm
- **Procedure:**
 - Maintain a running estimate of the mean squared value of derivatives for each parameter
 - Scale learning rate of the parameter by the *inverse* of the *root mean squared* derivative

$$E[\partial_w^2 D]_k = \gamma E[\partial_w^2 D]_{k-1} + (1 - \gamma)(\partial_w^2 D)_k$$

$$w_{k+1} = w_k - \frac{\eta}{\sqrt{E[\partial_w^2 D]_k + \epsilon}} \partial_w D$$

RMS Prop

- This is a variant on the *basic* mini-batch SGD algorithm
- **Procedure:**
 - Maintain a running estimate of the mean squared value of derivatives for each parameter
 - Scale learning rate of the parameter by the *inverse* of the *root mean squared* derivative

$$E[\partial_w^2 D]_k = \gamma E[\partial_w^2 D]_{k-1} + (1 - \gamma)(\partial_w^2 D)_k$$

$$w_{k+1} = w_k - \frac{\eta}{\sqrt{E[\partial_w^2 D]_k + \epsilon}} \partial_w D$$

Note similarity to RPROP

The magnitude of the derivative is being normalized out

RMS Prop (updates are for each weight of each layer)

- Do:

- Randomly shuffle inputs to change their order
- Initialize: $k = 1$; for all weights w in all layers, $E[\partial_w^2 D]_k = 0$
- For all $t = 1:B:T$ (incrementing in blocks of B inputs)

- For all weights in all layers initialize $(\partial_w D)_k = 0$

- For $b = 0:B-1$

- Compute

- » Output $Y(X_{t+b})$

- » Compute gradient $\frac{d \text{Div}(Y(X_{t+b}), d_{t+b})}{dw}$

- » Compute $(\partial_w D)_k += \frac{1}{B} \frac{d \text{Div}(Y(X_{t+b}), d_{t+b})}{dw}$

- update: for all $w \in \{w_{\{ij\}}^k \mid \forall i, j, k\}$

$$E[\partial_w^2 D]_k = \gamma E[\partial_w^2 D]_{k-1} + (1 - \gamma)(\partial_w^2 D)_k$$

$$w_{k+1} = w_k - \frac{\gamma}{\sqrt{E[\partial_w^2 D]_k + c}} \partial_w D$$

- $k = k + 1$

- Until loss has converged

Typical values:

$$\gamma = 0.9$$

$$\eta = 0.001$$

All the terms in gradient descent

- Standard gradient descent rule

$$W \leftarrow W - \eta \nabla_W L(W)$$

- RMSprop only adapts the learning rate
 - by total movement
- Momentum only smooths the gradient

All the terms in gradient descent

- Standard gradient descent rule

$$W \leftarrow W - \eta \nabla_W L(W)$$

- RMSprop only adapts the learning rate
 - by total movement
- Momentum only smooths the gradient
- How about combining both?

ADAM: RMSprop with momentum

- RMS prop only adapts the learning rate
- Momentum only smooths the gradient
- ADAM combines the two
- **Procedure:**
 - Maintain a running estimate of the mean derivative for each parameter
 - Maintain a running estimate of the mean squared value of derivatives for each parameter
 - Learning rate is proportional to the *inverse* of the *root mean squared* derivative

$$m_k = \delta m_{k-1} + (1 - \delta)(\partial_w D)_k$$

$$v_k = \gamma v_{k-1} + (1 - \gamma)(\partial_w^2 D)_k$$

$$\hat{m}_k = \frac{m_k}{1 - \delta^k}, \quad \hat{v}_k = \frac{v_k}{1 - \gamma^k}$$

$$w_{k+1} = w_k - \frac{\eta}{\sqrt{\hat{v}_k} + \epsilon} \hat{m}_k$$

ADAM: RMSprop with momentum

- RMS prop only adapts the learning rate
- Momentum only smooths the gradient
- ADAM combines the two
- **Procedure:**
 - Maintain a running estimate of the mean derivative for each parameter
 - Maintain a running estimate of the mean squared value of derivatives for each parameter
 - Learning rate is proportional to the *inverse* of the root mean squared derivative

$$m_k = \delta m_{k-1} + (1 - \delta)(\partial_w D)_k$$

$$v_k = \gamma v_{k-1} + (1 - \gamma)(\partial_w^2 D)_k$$

$$\hat{m}_k = \frac{m_k}{1 - \delta^k},$$

$$\hat{v}_k = \frac{v_k}{1 - \gamma^k}$$

$$w_{k+1} = w_k - \frac{\eta}{\sqrt{\hat{v}_k + \epsilon}} \hat{m}_k$$

δ and γ terms do not dominate in early iterations

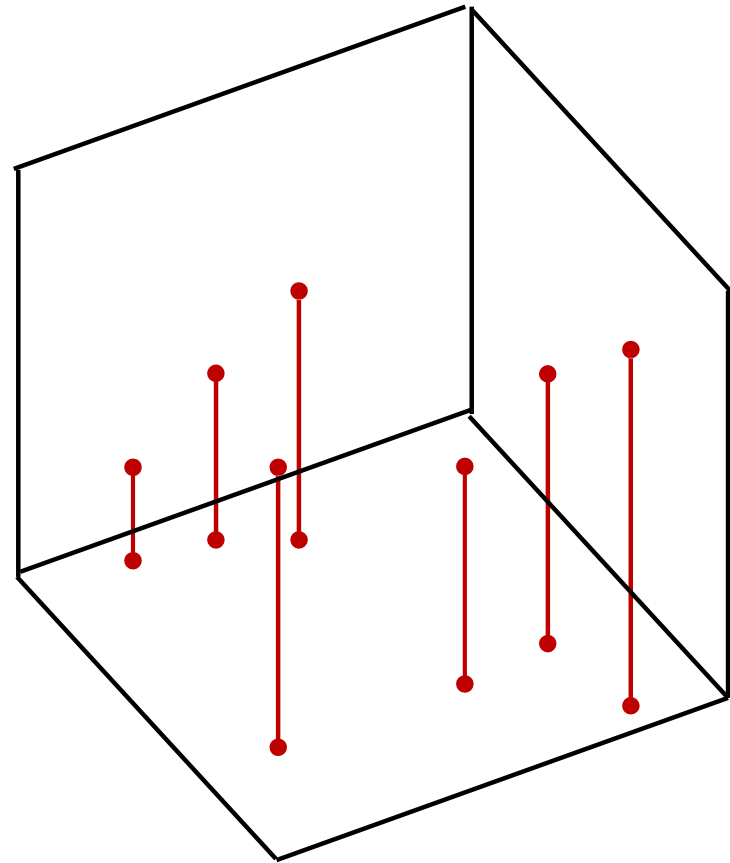
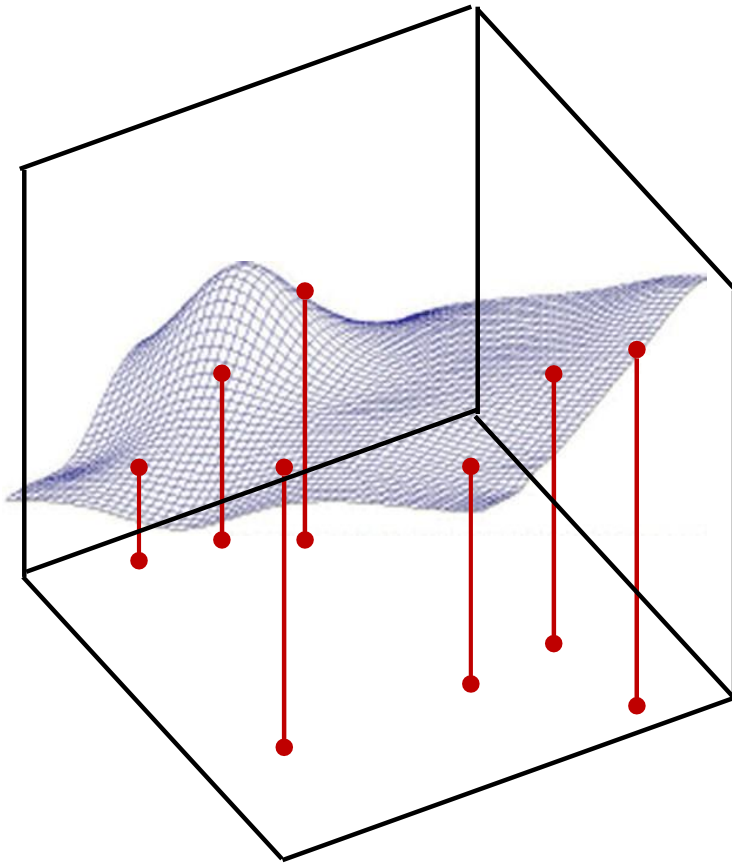
Other variants of the same theme

- Many:
 - Adagrad
 - AdaDelta
 - AdaMax
 - ...
- Generally no explicit learning rate to optimize
 - But come with other hyper parameters to be optimized
 - Typical params:
 - RMSProp: $\eta = 0.001, \gamma = 0.9$
 - ADAM: $\eta = 0.001, \delta = 0.9, \gamma = 0.999$

Story so far

- Gradient descent can be sped up by incremental updates
 - Convergence is guaranteed under most conditions
 - Learning rate must shrink with time for convergence
 - Stochastic gradient descent: update after each observation. Can be much faster than batch learning
 - Mini-batch updates: update after batches. Can be more efficient than SGD
- Convergence can be improved using smoothed updates
 - RMSprop and more advanced techniques

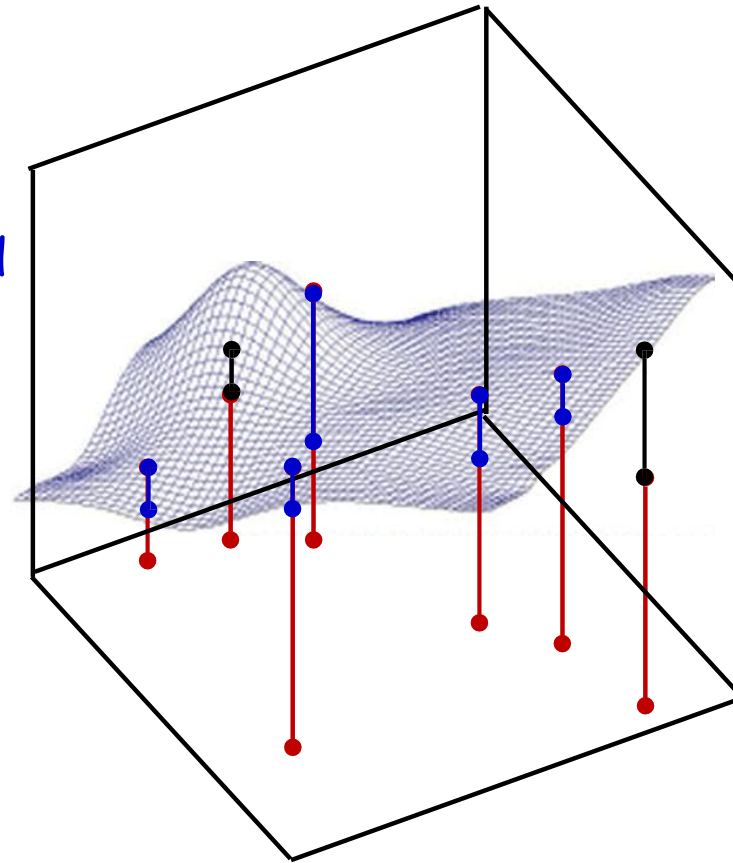
Learning the network



- We attempt to learn an entire function from just a few *snapshots* of it

General approach to training

Blue lines: error when function is *below* desired output

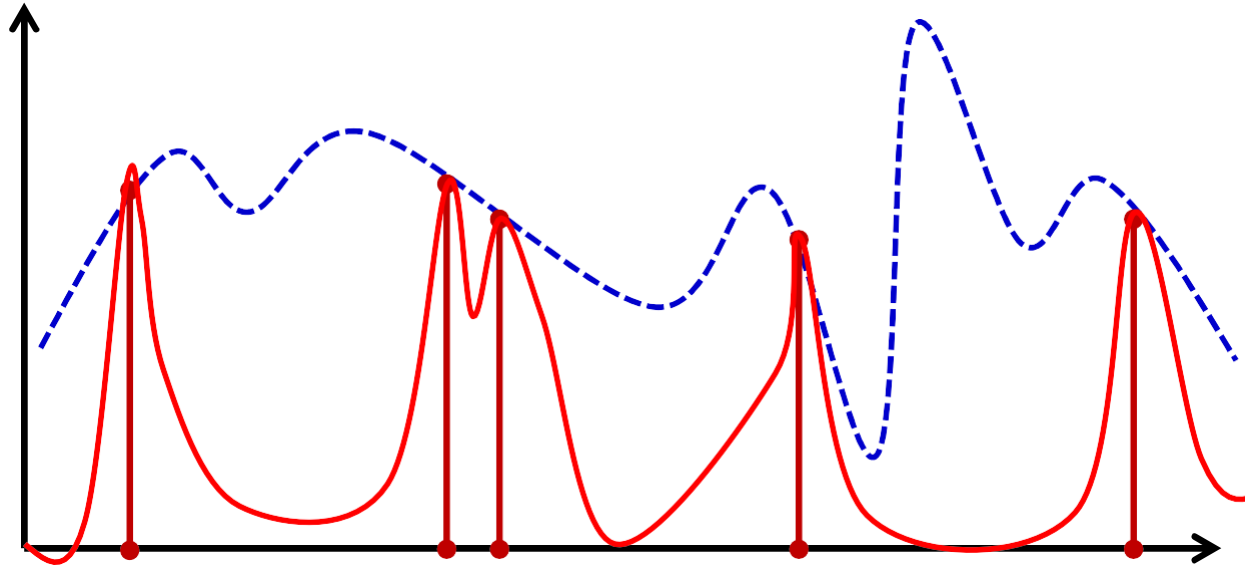


Black lines: error when function is *above* desired output

$$E = \sum_i (d_i - f(\mathbf{x}_i, \mathbf{W}))^2$$

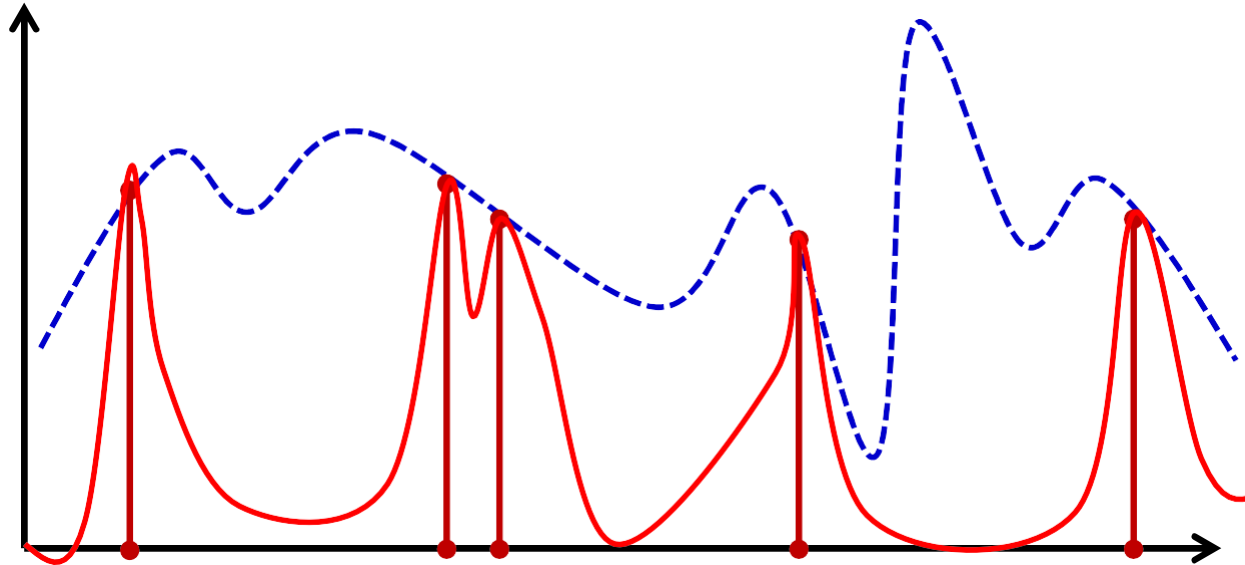
- Define a *divergence* between the **actual** network output for any parameter value and the *desired* output
 - Typically L2 divergence or KL divergence

Overfitting



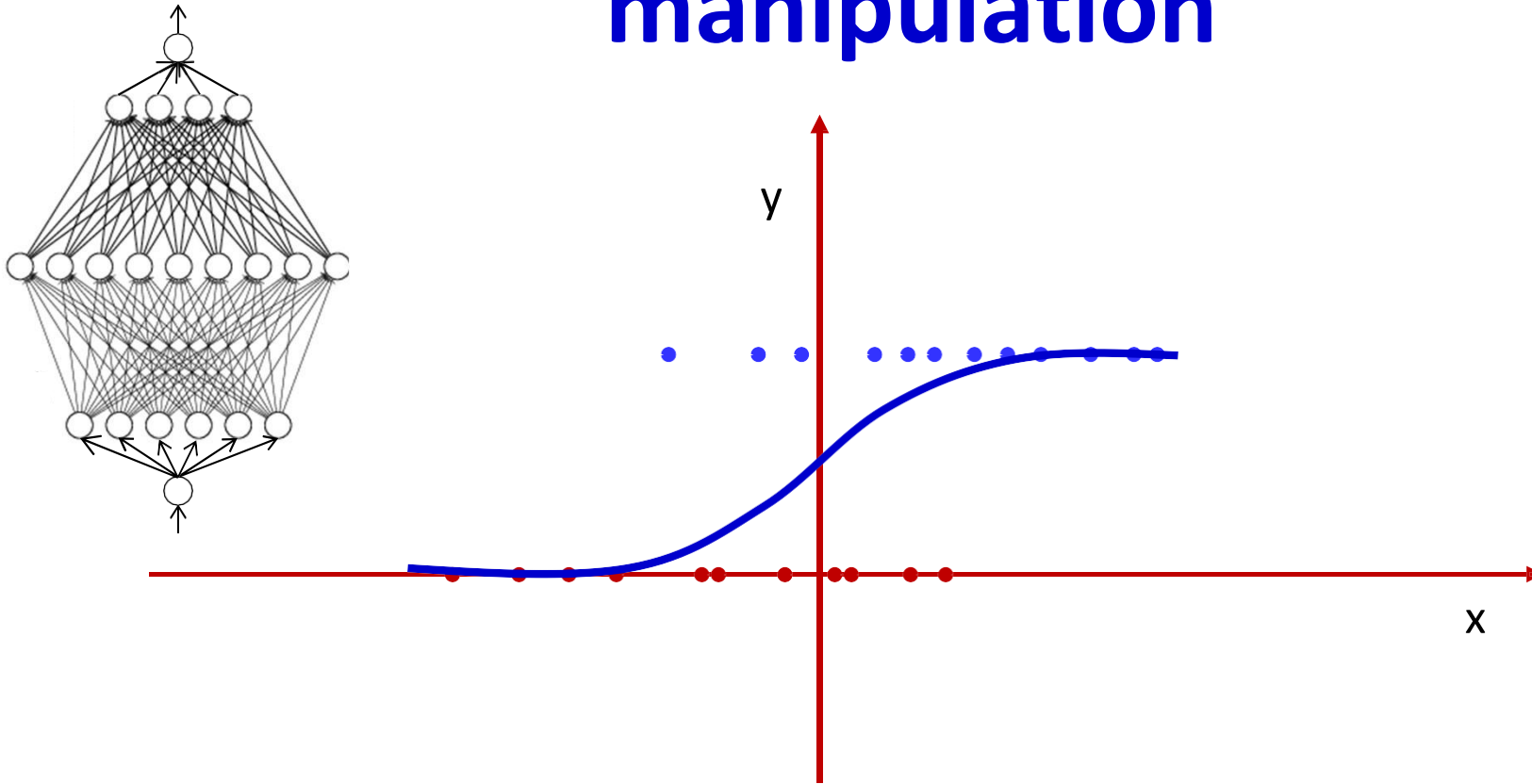
- Problem: Network may just learn the values at the inputs
 - Learn the red curve instead of the dotted blue one
 - Given only the red vertical bars as inputs

Overfitting and Smoothing



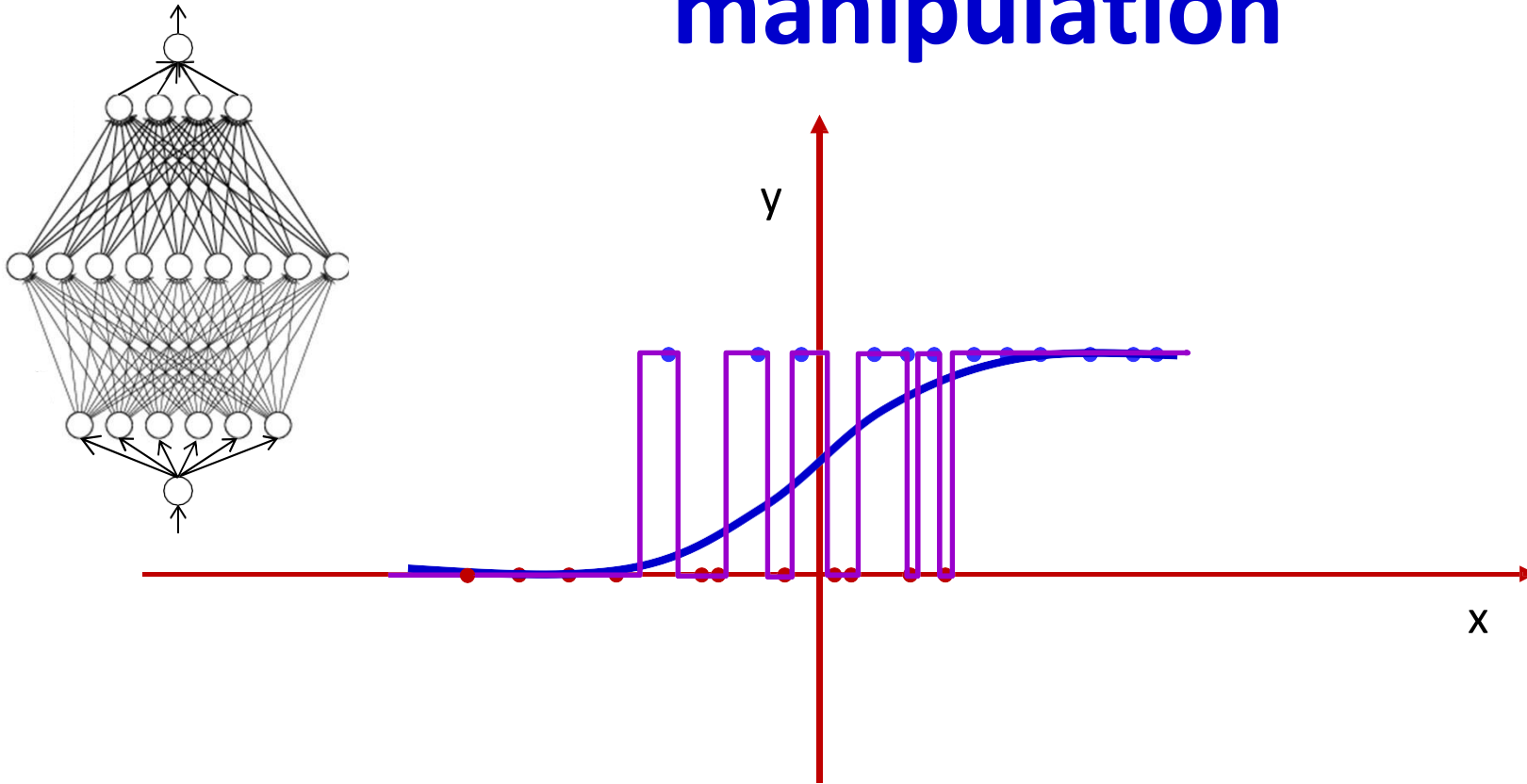
- Problem: Network may just learn the values at the inputs
 - Learn the red curve instead of the dotted blue one
 - Given only the red vertical bars as inputs
- Need additional “**smoothing**” constraints that will “fill in” the missing regions acceptably
 - Generalization

Smoothness through weight manipulation



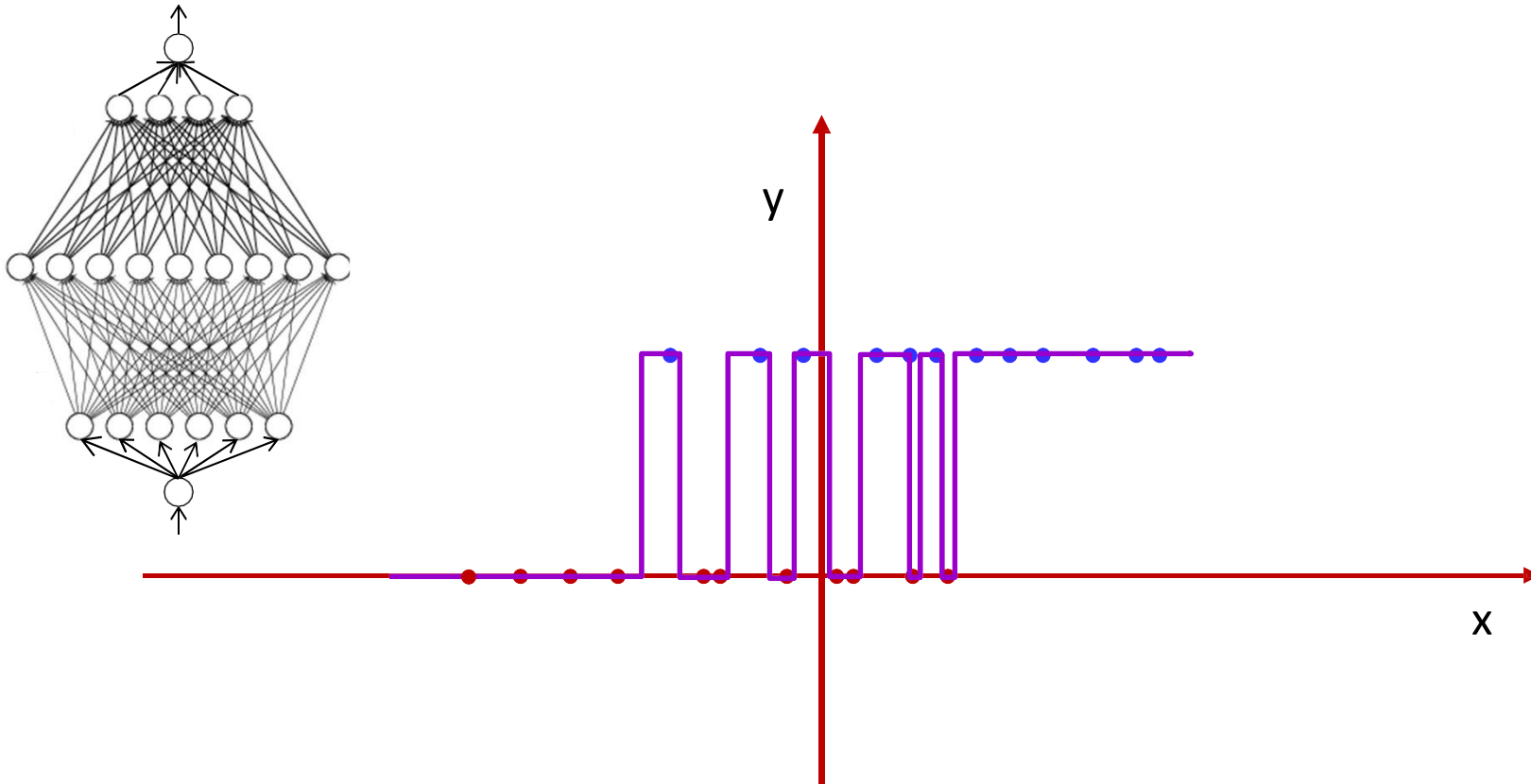
- Illustrative example: Simple binary classifier
 - The “desired” output is generally smooth

Smoothness through weight manipulation



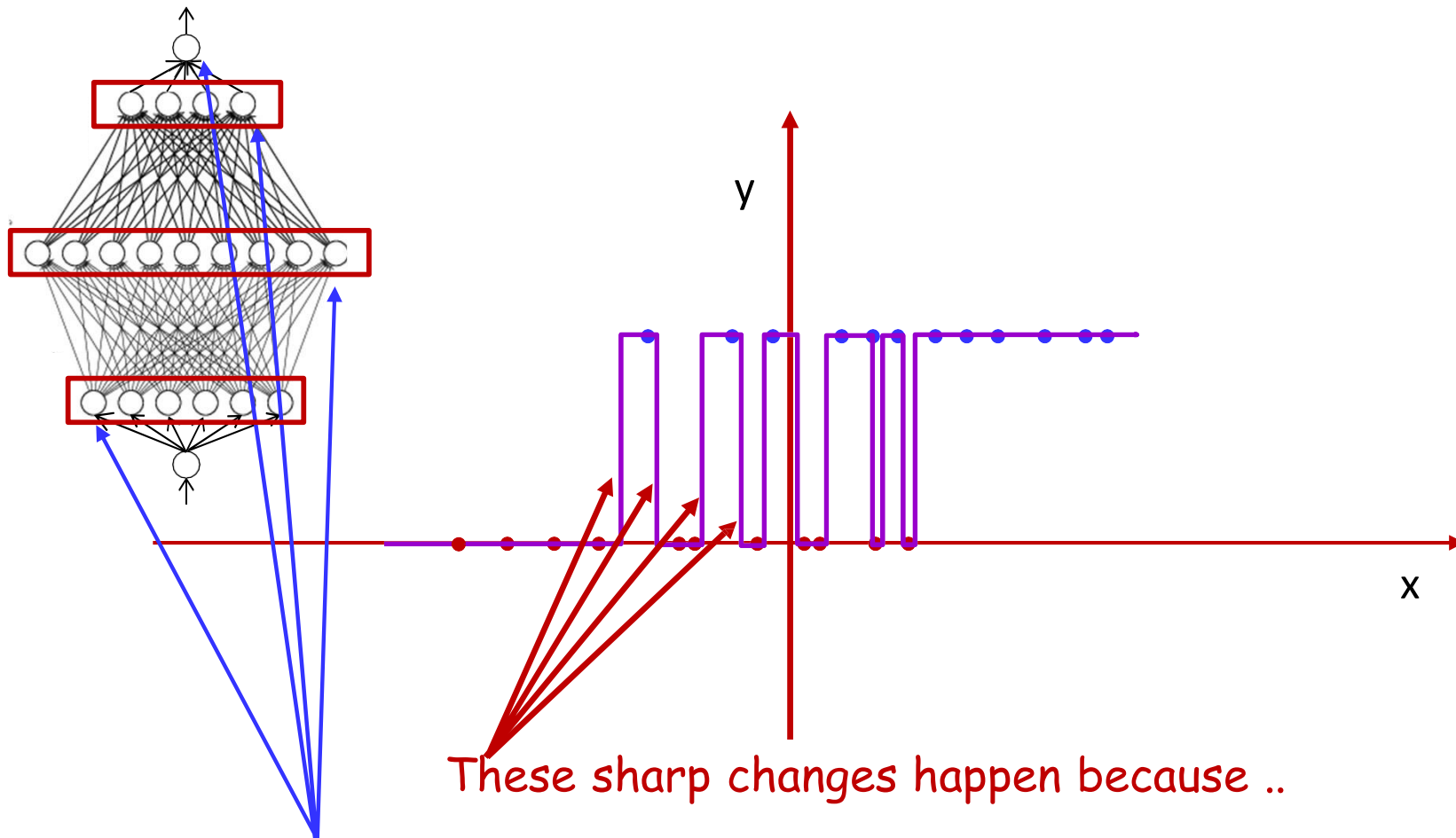
- Illustrative example: Simple binary classifier
 - The “desired” output is generally smooth
 - Capture statistical or average trends
 - An unconstrained model will model individual instances instead

The unconstrained model



- Illustrative example: Simple binary classifier
 - The “desired” output is generally smooth
 - Capture statistical or average trends
 - An unconstrained model will model individual instances instead

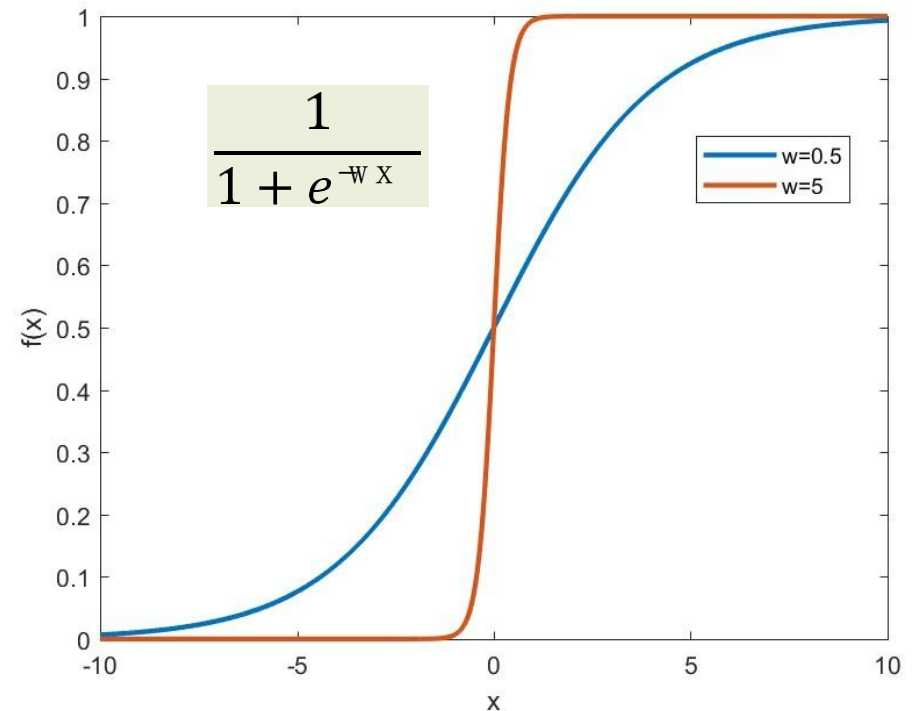
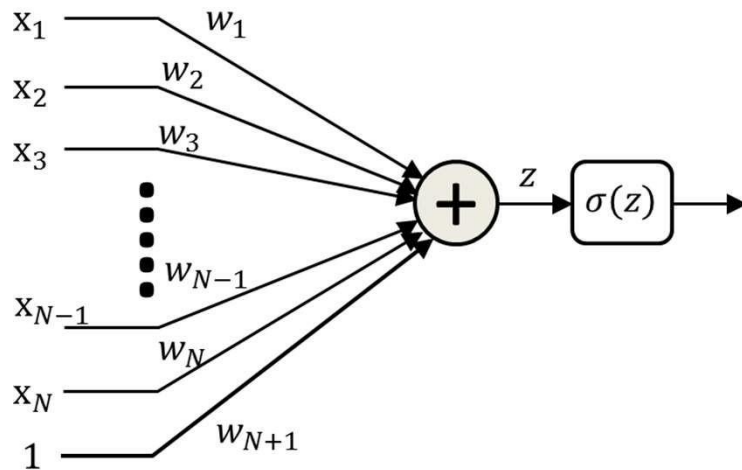
Why overfitting



These sharp changes happen because ..

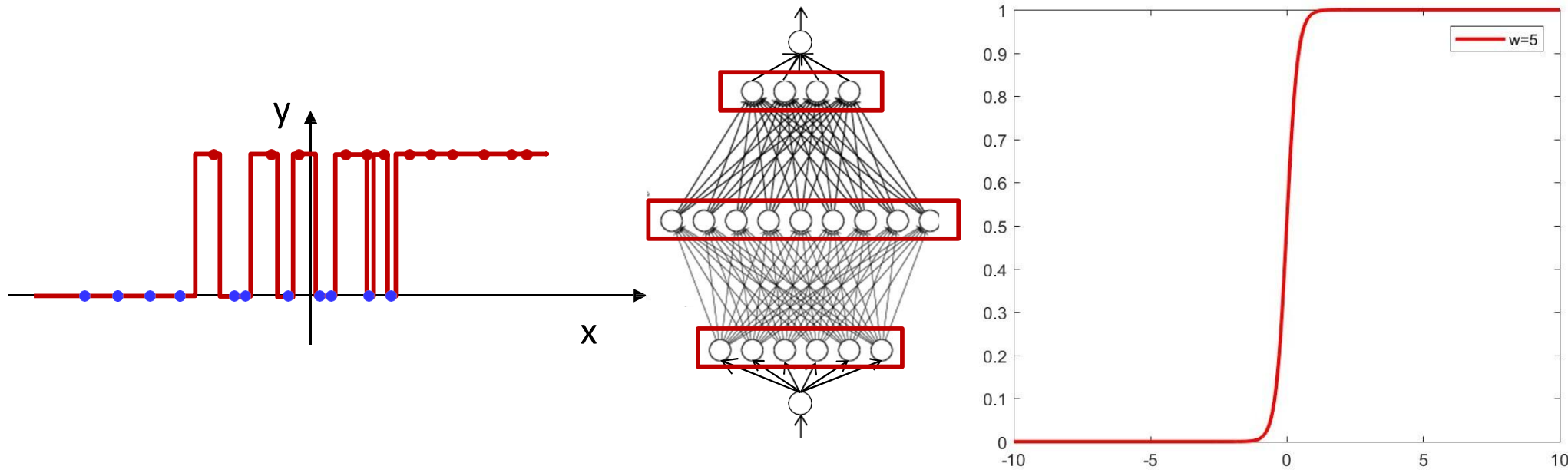
..the perceptrons in the network are individually capable of sharp changes in output

The individual perceptron



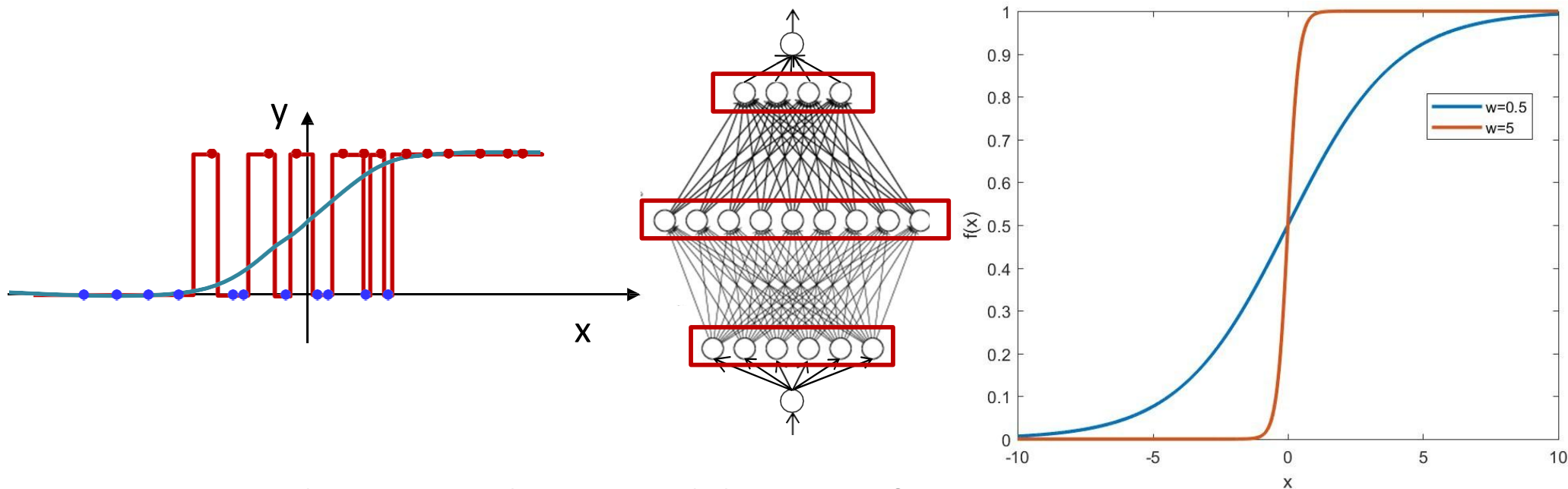
- Using a sigmoid activation
 - As $|w|$ increases, the response becomes steeper

Smoothness through weight manipulation



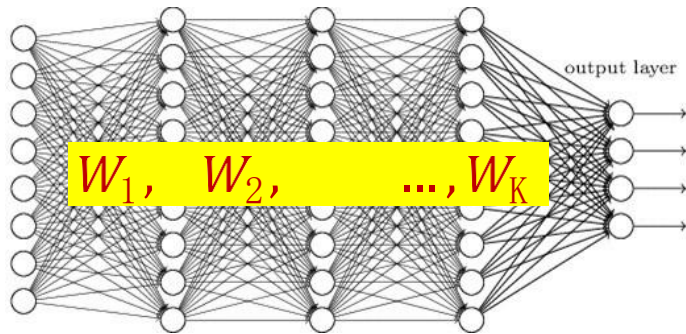
- Steep changes that enable overfitted responses are facilitated by perceptrons with large w

Smoothness through weight manipulation



- Steep changes that enable overfitted responses are facilitated by perceptrons with large w
- Constraining the weights w to be low will force slower perceptrons and smoother output response

Objective function for neural networks



Desired output of network: d_t

Error on i-th training input: $Div(Y_t, d_t; W_1, W_2, \dots, W_K)$

Training loss:

$$Loss(W_1, W_2, \dots, W_K) = \frac{1}{T} \sum_t Div(Y_t, d_t; W_1, W_2, \dots, W_K)$$

- Conventional training: minimize the loss:

$$\hat{W}_1, \hat{W}_2, \dots, \hat{W}_K = \operatorname{argmin}_{W_1, W_2, \dots, W_K} Loss(W_1, W_2, \dots, W_K)$$

Smoothness through weight constraints

- Regularized training: minimize the loss while also minimizing the weights

$$L(W_1, W_2, \dots, W_K) = \frac{1}{T} \sum_t \text{Div}(Y_t, d_t; W_1, W_2, \dots, W_K) + \frac{1}{2} \lambda \sum_k \|W_k\|_F^2$$

$$\hat{W}_1, \hat{W}_2, \dots, \hat{W}_K = \underset{W_1, W_2, \dots, W_K}{\operatorname{argmin}} L(W_1, W_2, \dots, W_K)$$

- λ is the regularization parameter whose value depends on how important it is for us to want to minimize the weights
- Increasing λ assigns greater importance to shrinking the weights
 - Make greater error on training data, to obtain a more acceptable network

Regularizing the weights

$$L(W_1, W_2, \dots, W_K) = \frac{1}{T} \sum_t \text{Div}(Y_t, d_t) + \frac{1}{2} \lambda \sum_k \|W_k\|_F^2$$

- Batch mode:

$$\nabla_{W_k} L = \frac{1}{T} \sum_t \nabla_{W_k} \text{Div}(Y_t, d_t) + \lambda W_k^T$$

- SGD:

$$\nabla_{W_k} L = \nabla_{W_k} \text{Div}(Y_t, d_t) + \lambda W_k^T$$

- Minibatch:

$$\nabla_{W_k} L = \frac{1}{b} \sum_{\tau=t}^{t+b-1} \nabla_{W_k} \text{Div}(Y_t, d_t) + \lambda W_k^T$$

- Update rule:

$$W_k \leftarrow W_k - \eta \nabla_{W_k} L^T = W_k - \eta \frac{1}{b} \sum_{\tau=t}^{t+b-1} \nabla_{W_k} \text{Div}(Y_t, d_t)^T - \eta \lambda W_k$$

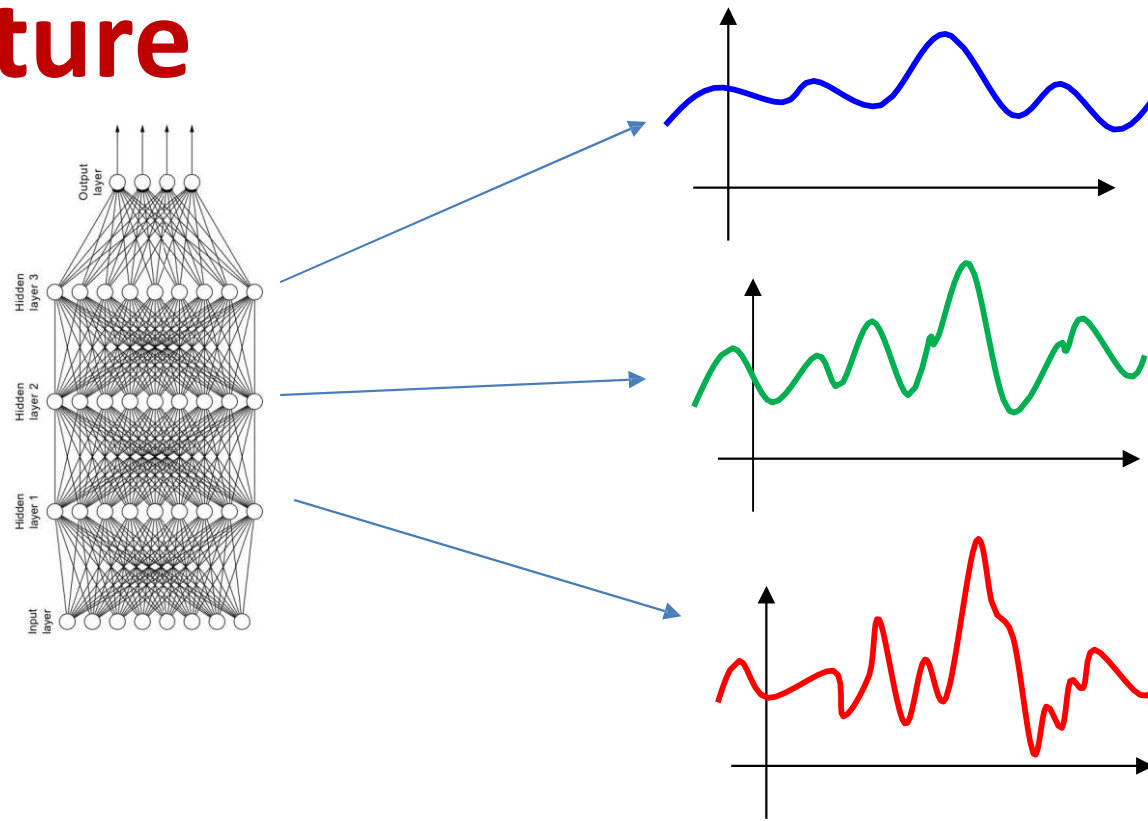
$$W_k \leftarrow (1 - \lambda) W_k - \eta \frac{1}{b} \sum_{\tau=t}^{t+b-1} \nabla_{W_k} \text{Div}(Y_t, d_t)$$

Incremental Update: Mini-batch update

- Given $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$
- Initialize all weights $W_1, W_2, \dots, W_K$; $j = 0$
- Do:
 - Randomly permute $(X_1, d_1), (X_2, d_2), \dots, (X_T, d_T)$
 - For $t = 1:b:T$
 - $j = j + 1$
 - For every layer k :
 - $\Delta W_k = 0$
 - For $t' = t : t+b-1$
 - For every layer k
 - » Compute $\square_{W_k} W_k d_{t'}$
 - » $\Delta W_k = \Delta W_k + \square_{W_k} W_k d_{t'} (\quad)^T$
 - Update
 - For every layer k :

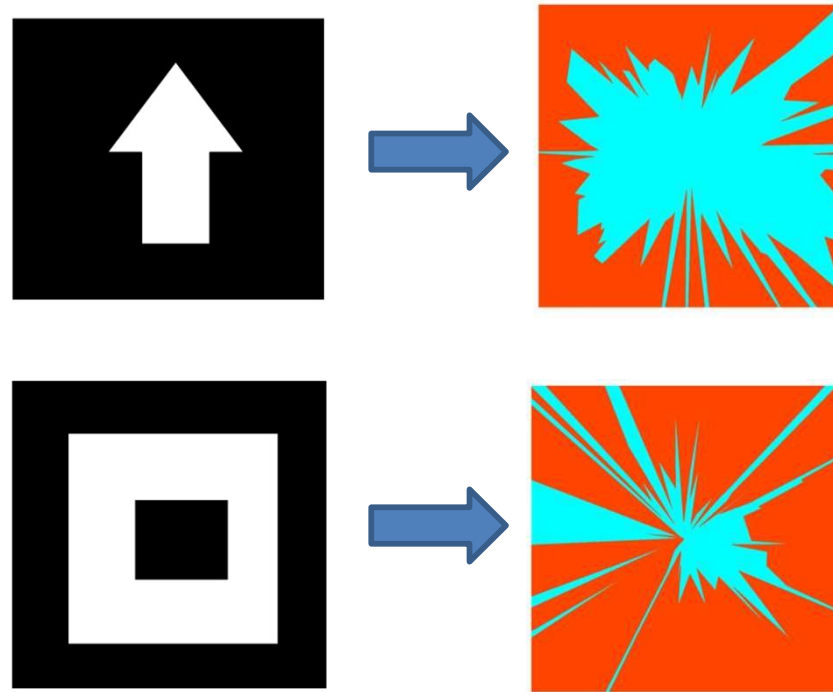
$$W_k = (1 - \lambda)W_k - \eta_j \Delta W_k \quad \leftarrow$$
- Until *Loss* has converged

Smoothness through network structure



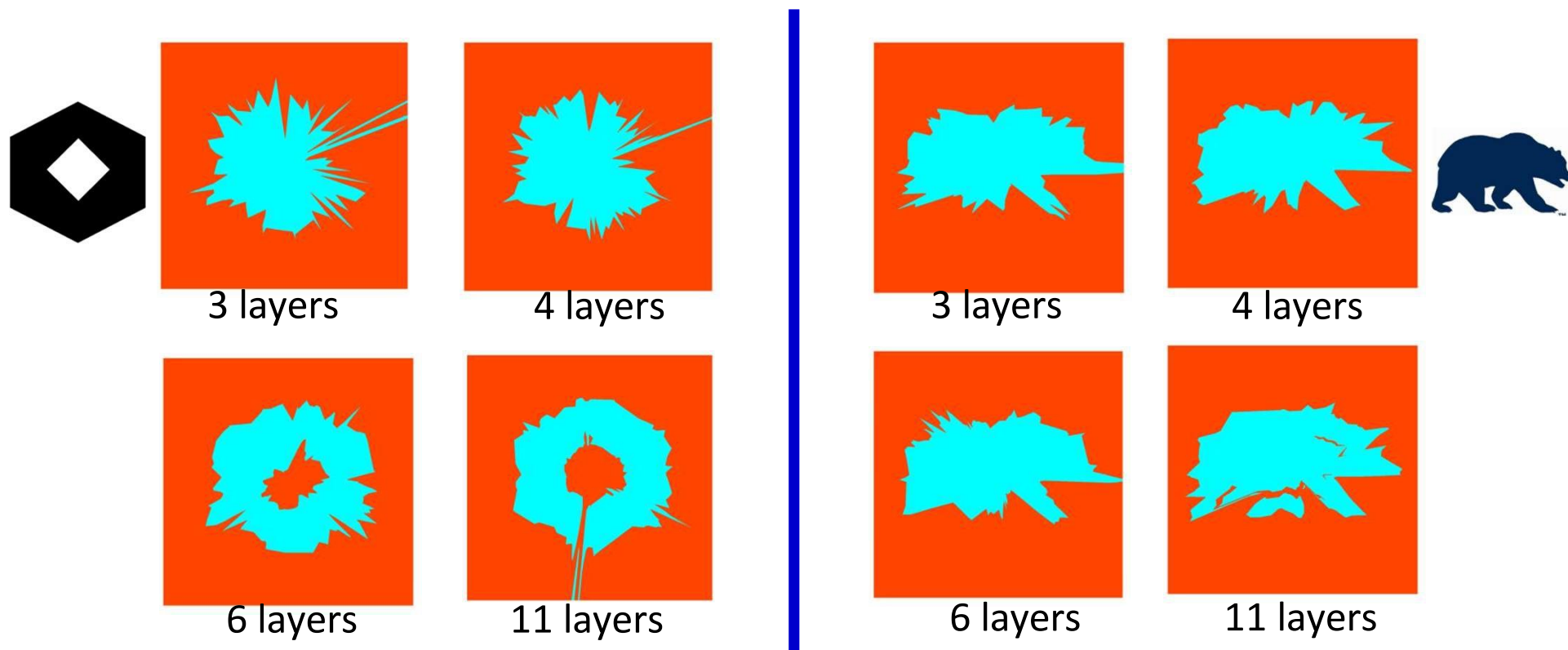
- Smoothness constraints can also be imposed through the network *structure*
- *For a given number of parameters deeper networks impose more smoothness than shallow ones*
 - Each layer works on the already smooth surface output by the previous layer¹¹⁸

Minimal correct architectures are hard to train



- Typical results (varies with initialization)
- 1000 training points – orders of magnitude more than you usually get
- All the training tricks known to mankind

But depth and training data help



- Deeper networks seem to learn better, for the same number of total neurons
 - ***Implicit smoothness constraints***
 - *As opposed to explicit constraints from more conventional regularization methods*
- Training with more data is also better 😊

10000 training instances



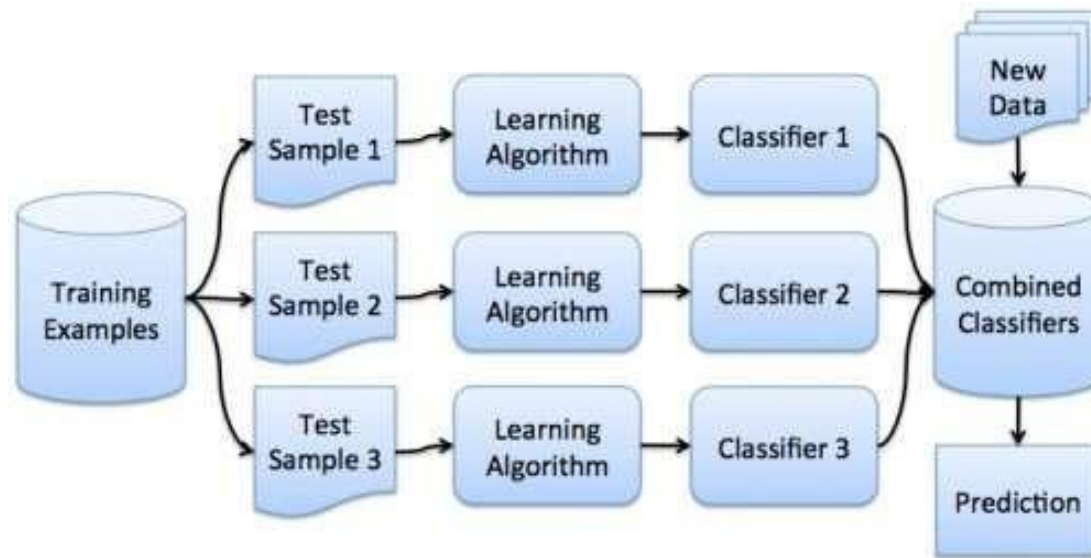
Story so far

- Gradient descent can be sped up by incremental updates
- Convergence can be improved using smoothed updates
- The choice of divergence affects both the learned network and results
- Covariate shift between training minibatches may cause problems and may be handled by batch normalization
- Data underspecification can result in overfitted models and must be handled by regularization and more constrained (generally deeper) network architectures

Regularization..

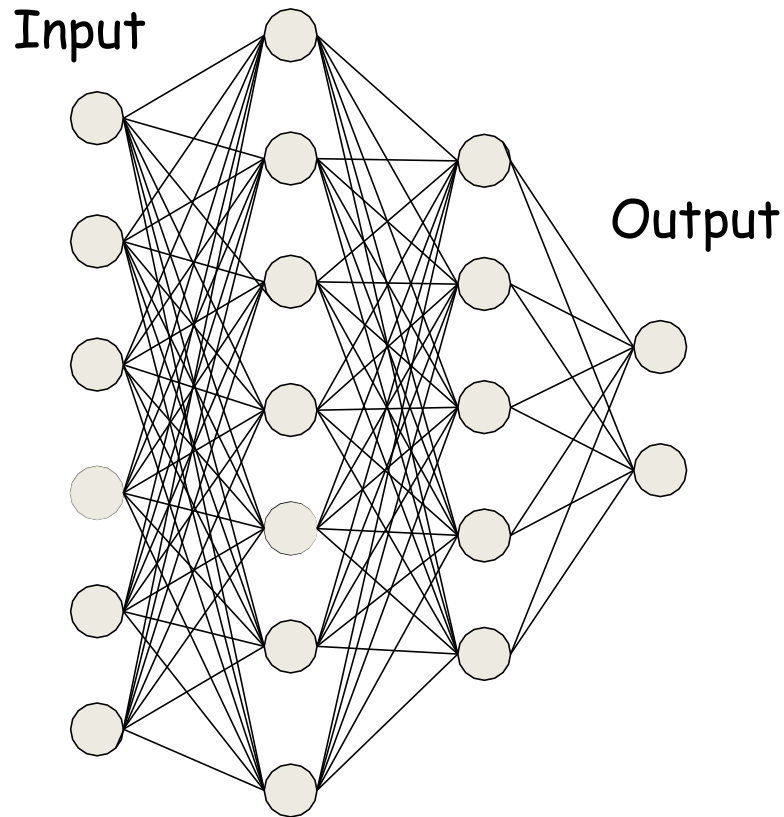
- Other techniques have been proposed to improve the smoothness of the learned function
 - L_1 regularization of network activations
 - Regularizing with added noise..
- Possibly the most influential method has been “dropout”

A brief detour.. Bagging



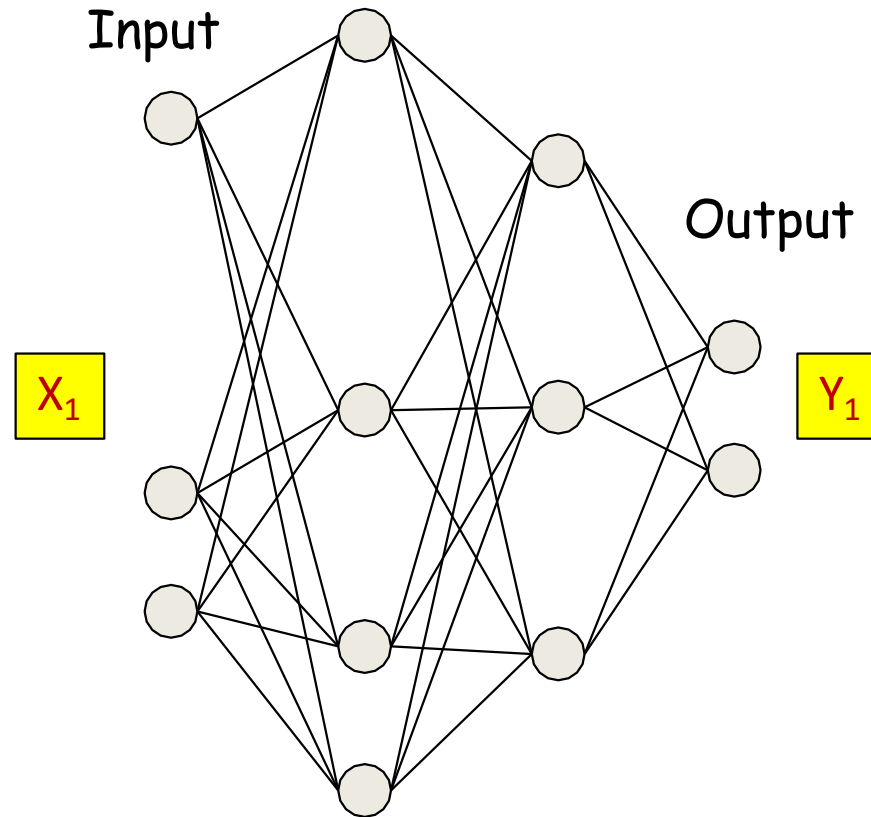
- Popular method proposed by Leo Breiman:
 - Sample training data and train several different classifiers
 - Classify test instance with entire ensemble of classifiers
 - Vote across classifiers for final decision
 - Empirically shown to improve significantly over training a single classifier from combined data
- Returning to our problem....

Dropout



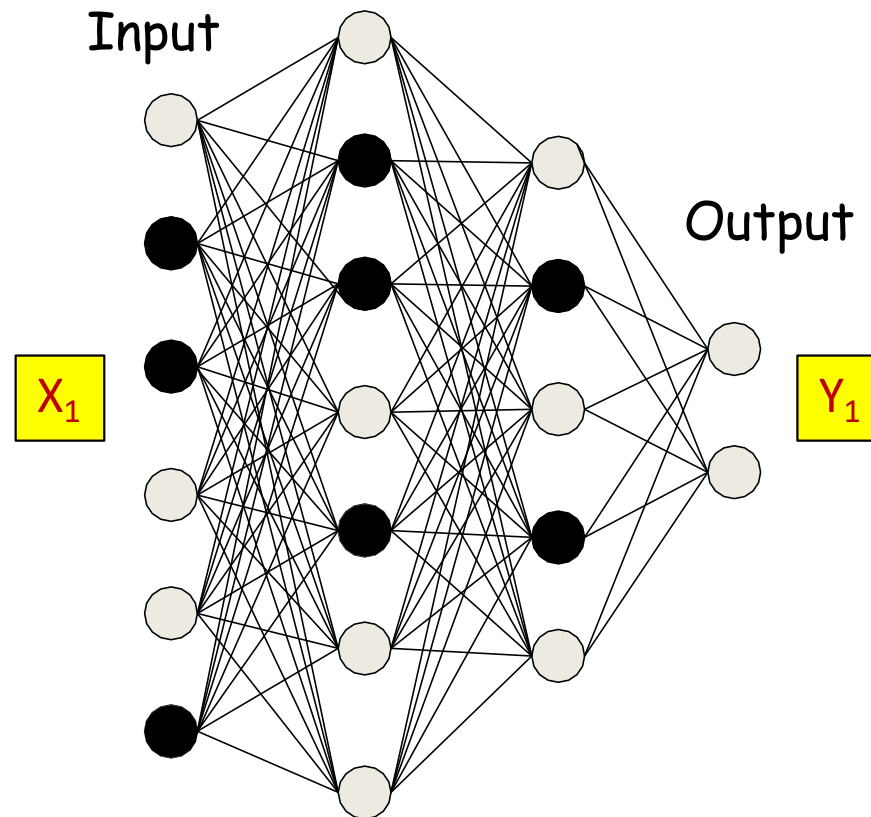
- **During training:** For each input, at each iteration, “turn off” each neuron with a probability $1-\alpha$

Dropout



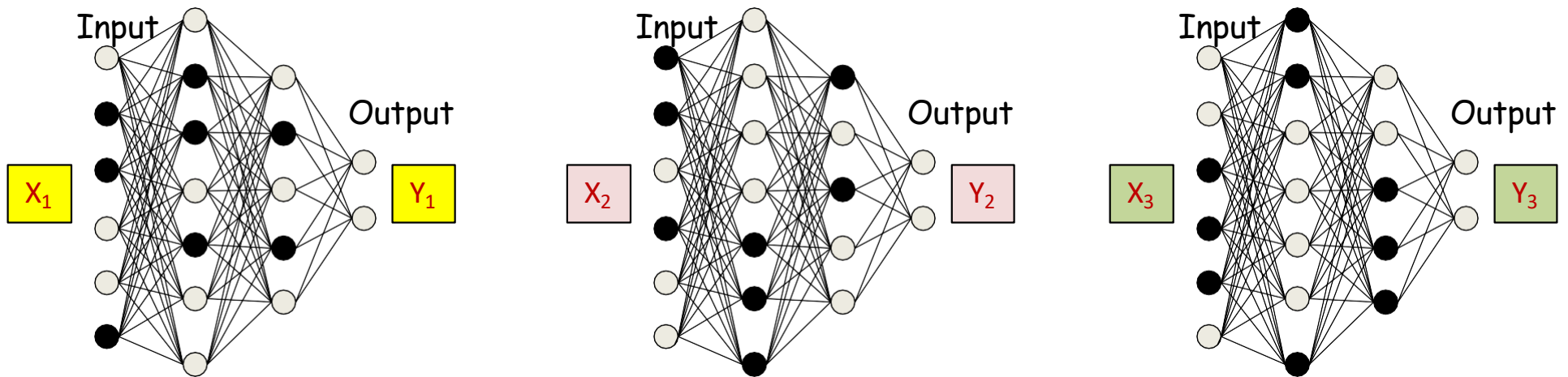
- **During training:** For each input, at each iteration, “turn off” each neuron with a probability $1-\alpha$
 - Also turn off inputs similarly

Dropout



- **During training:** For each input, at each iteration, “turn off” each neuron (including inputs) with a probability $1-\alpha$
 - In practice, set them to 0 according to the failure of a Bernoulli random number generator with success probability α

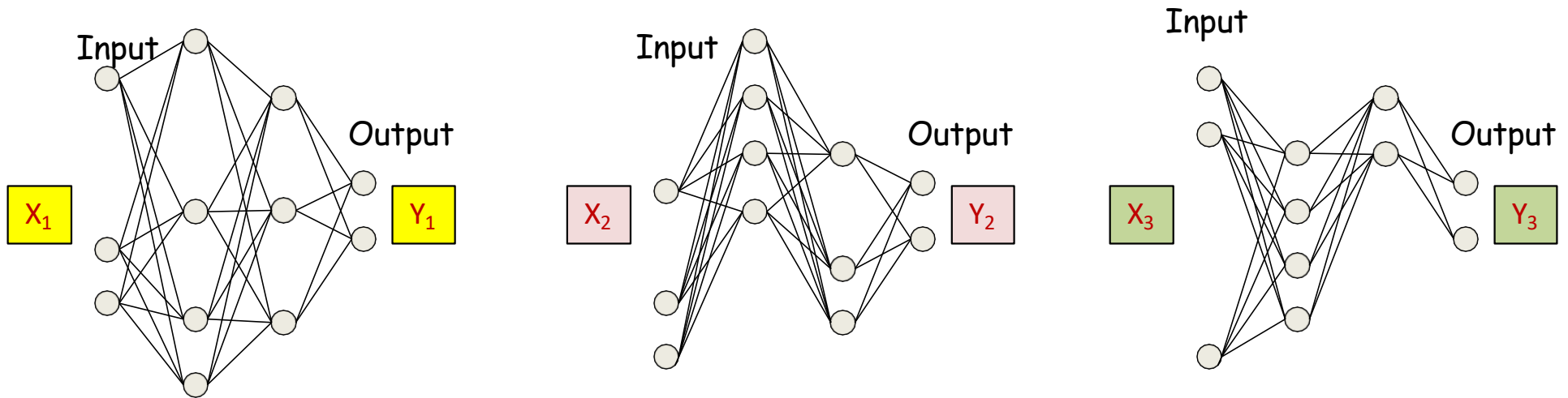
Dropout



The pattern of dropped nodes changes for each input
i.e. in every pass through the net

- **During training:** For each input, at each iteration, “turn off” each neuron (including inputs) with a probability $1-\alpha$
 - In practice, set them to 0 according to the failure of a Bernoulli random number generator with success probability α

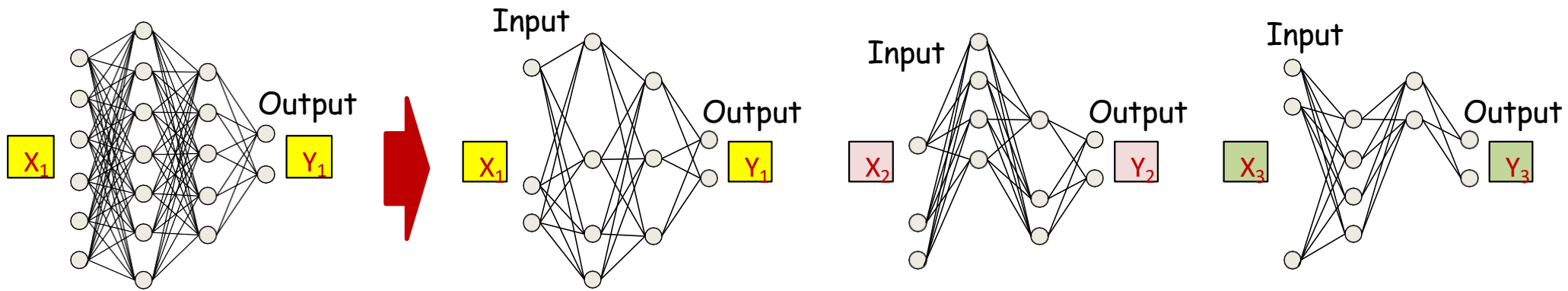
Dropout



The pattern of dropped nodes changes for *each input*
i.e. in every pass through the net

- **During training:** Backpropagation is effectively performed only over the remaining network
 - The effective network is different for different inputs
 - Gradients are obtained only for the weights and biases *from* “On” nodes *to* “On” nodes
 - For the remaining, the gradient is just 0

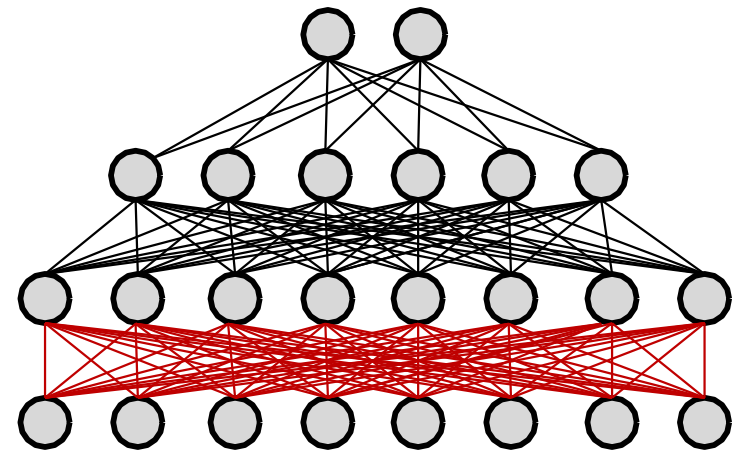
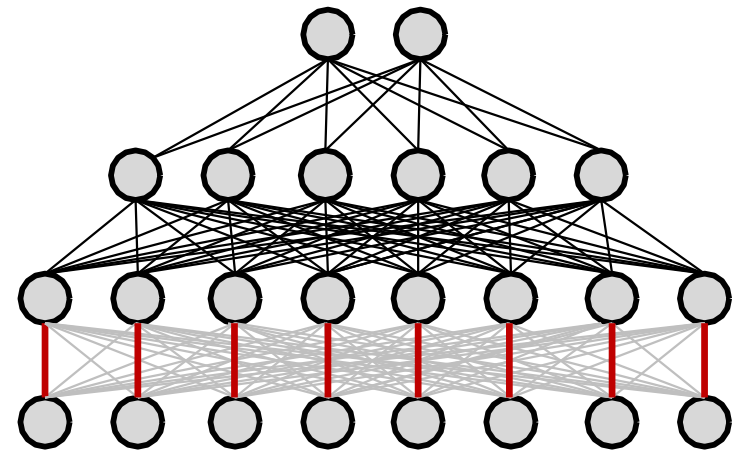
Statistical Interpretation



- For a network with a total of N neurons, there are 2^N possible sub-networks
 - Obtained by choosing different subsets of nodes
 - Dropout *samples* over all 2^N possible networks
 - Effectively learns a network that *averages* over all possible networks
 - Bagging

Dropout as a mechanism to increase pattern density

- Dropout forces the neurons to learn “rich” and redundant patterns
- E.g. without dropout, a non-compressive layer may just “clone” its input to its output
 - Transferring the task of learning to the rest of the network upstream
- Dropout forces the neurons to learn denser patterns
 - With redundancy



The forward pass

- Input: D dimensional vector $\mathbf{x} = [x_j, j = 1 \dots D]$
- Set:
 - $D_0 = D$, is the width of the 0th (input) layer
 - $y_j^{(0)} = x_j, j = 1 \dots D; \quad y_0^{(k=1 \dots N)} = x_0 = 1$
- For layer $k = 1 \dots N$

Mask takes value 1 with prob. α , 0 with prob $1 - \alpha$

– $mask(k - 1, j) = \text{Bernoulli}(\alpha), j = 1 \dots D_{k-1}$

– $y_j^{(k-1)} = y_j^{(k-1)} \cdot mask(k - 1, j), j = 1 \dots D_{k-1}$

– For $j = 1 \dots D_k$

- $z_j^{(k)} = \sum_{i=0}^{N_k} w_{1,j \ i}^{(k)} y_{i,j}^{(k-1)} + b_j^{(k)}$

- $y_j^{(k)} = f_k(z_j^{(k)})$

- Output:
 - $Y = y_j^{(N)}, j = 1 \dots D_N$

Backward Pass

- Output layer (N) :

$$-\frac{\partial Div}{\partial Y_i} = \frac{\partial Div(Y, d)}{\partial y_i^{(N)}}$$

$$-\frac{\partial Div}{\partial z_i^{(k)}} = f'_k \left(z_i^{(k)} \right) \frac{\partial Div}{\partial y_i^{(k)}}$$

- For layer $k = N - 1$ *downto* 0

– For $i = 1 \dots D_k$

- $\frac{\partial Div}{\partial y_i^{(k)}} = mask(k, i) \sum_j w_{ij}^{(k+1)} \frac{\partial Div}{\partial z_j^{(k+1)}}$
- $\frac{\partial Div}{\partial z_i^{(k)}} = f'_k \left(z_i^{(k)} \right) \frac{\partial Div}{\partial y_i^{(k)}}$
- $\frac{\partial Div}{\partial w_{ij}^{(k+1)}} = y_i^{(k)} \frac{\partial Div}{\partial z_j^{(k+1)}}$ for $j = 1 \dots D_{k+1}$

Testing with Dropout

- Dropout effectively trains 2^N networks
- On test data the “Bagged” output, in principle, is the ensemble average over all 2^N networks and is thus the statistical expectation of the output over all networks

$$Y = E \left[network \left(y_j^{(k)}, j = 1 \dots D_k, k = 1 \dots K \right) \right]$$

- Explicitly showing the network as a function of the outputs of individual neurons in the net
- We cannot explicitly compute this expectation
- Instead, we will use the following approximation

$$E \left[network \left(y_j^{(k)}, \forall k, j \right) \right] \approx network \left(E[y_j^{(k)}] \forall k, j \right)$$

- Where $E[y_j^{(k)}]$ is the expected output of the jth neuron in the kth layer over all networks in the ensemble
- I.e. approximate the expectation of a function as the function of expectations
- We require $E[y_j^{(k)}]$ to compute this

What each neuron computes

- Each neuron actually has the following activation:

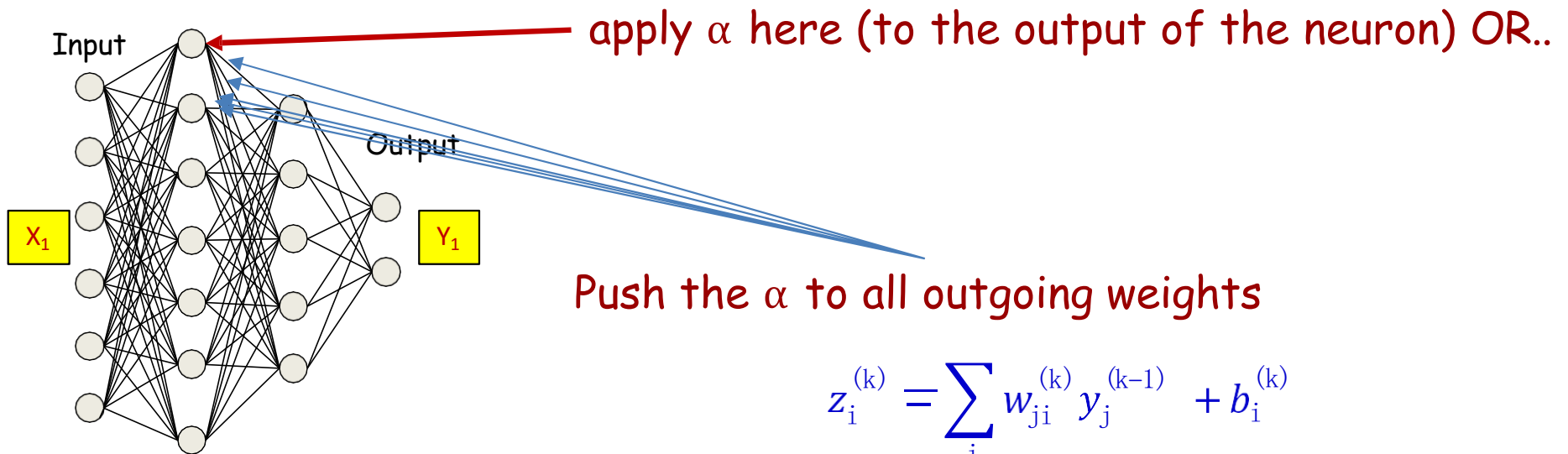
$$y_i^{(k)} = D\sigma\left(\sum_j w_{ji}^{(k)} y_j^{(k-1)} + b_i^{(k)}\right)$$

- Where D is a Bernoulli variable that takes a value 1 with probability α
- D may be switched on or off for individual sub networks, but over the ensemble, the *expected output* of the neuron is

$$\mathbb{E}[y_i^{(k)}] = \alpha\sigma\left(\sum_j w_{ji}^{(k)} y_j^{(k-1)} + b_i^{(k)}\right)$$

- During *test* time, we will use the *expected* output of the neuron
 - Consists of simply scaling the output of each neuron by α

Dropout during test: implementation



$$y_i^{(k)} = \alpha \sigma(z_i^{(k)})$$

$$\begin{aligned} z_i^{(k)} &= \sum_j w_{ji}^{(k)} y_j^{(k-1)} + b_i^{(k)} \\ &= \sum_j w_{ji}^{(k)} \alpha \sigma(z_j^{(k-1)}) + b_i^{(k)} \\ &= \sum_j (\alpha w_{ji}^{(k)}) \sigma(z_j^{(k-1)}) + b_i^{(k)} \end{aligned}$$

$$W_{test} = \alpha W_{trained}$$

- Instead of multiplying every output by α , multiply all weights by α

Inference with dropout (testing)

- Input: D dimensional vector $\mathbf{x} = [x_j, j = 1 \dots D]$

- Set:

- $D_0 = D$, is the width of the 0th (input) layer

- $y_j^{(0)} = x_j, j = 1 \dots D; \quad y_0^{(k=1 \dots N)} = x_0 = 1$

- For layer $k = 1 \dots N$

- For $j = 1 \dots D_k$

- $z_j^{(k)} = \sum_{i=0}^{N_k} w_{i,j}^{(k)} y_i^{(k-1)} + b_j^{(k)}$

- $y_j^{(k)} = \alpha f_k(z_j^{(k)})$

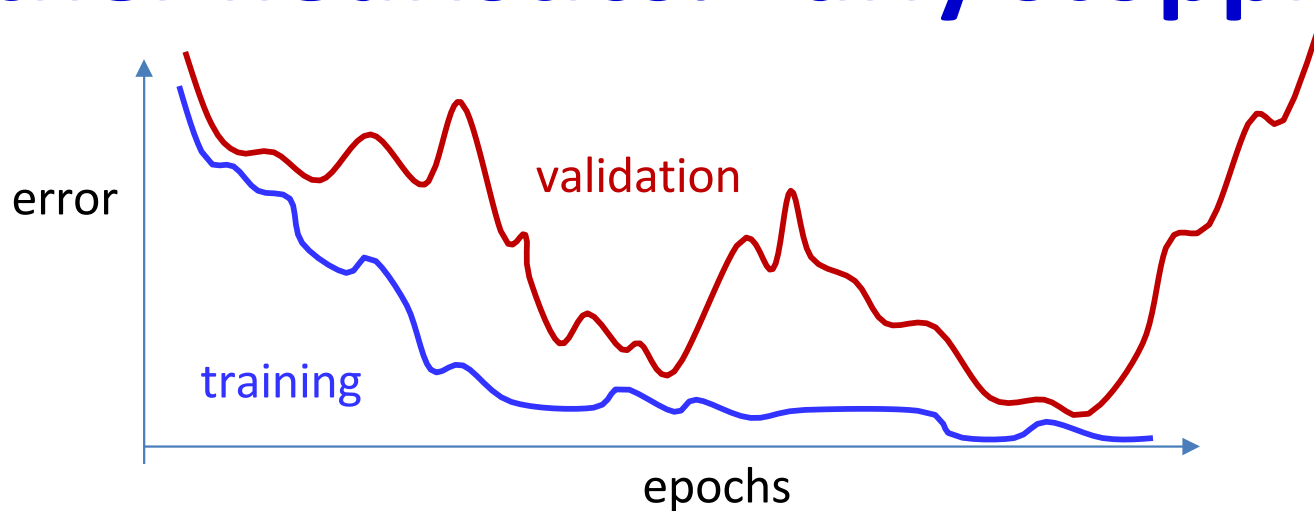
- Output:

- $Y = y_j^{(N)}, j = 1 \dots D_N$

Story so far

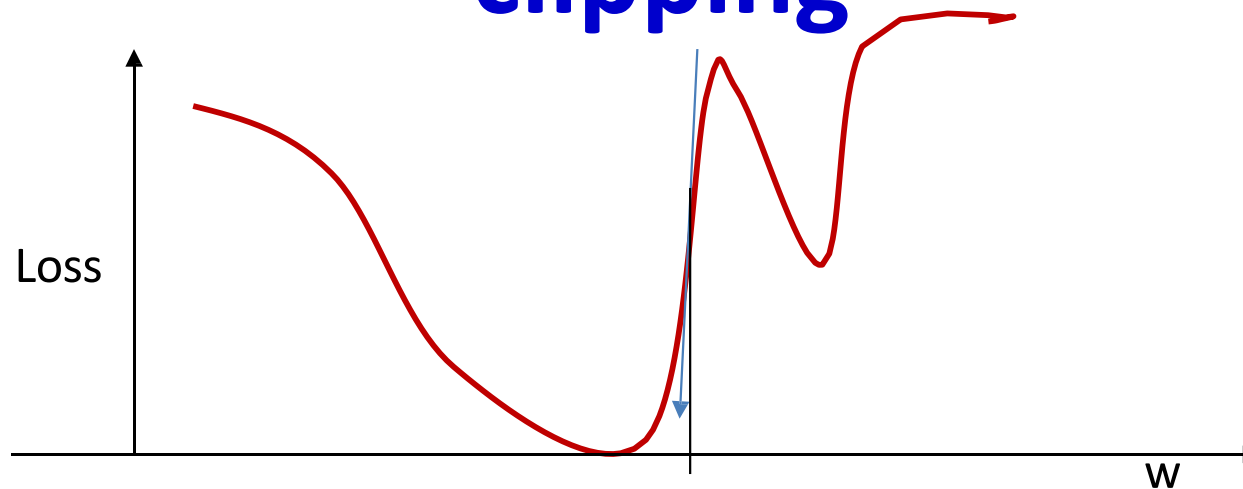
- Gradient descent can be sped up by incremental updates
- Convergence can be improved using smoothed updates
- The choice of divergence affects both the learned network and results
- Covariate shift between training minibatches may cause problems and may be handled by batch normalization
- Data underspecification can result in overfitted models and must be handled by regularization and more constrained (generally deeper) network architectures
- “Dropout” is a stochastic data/model erasure method that sometimes forces the network to learn more robust models

Other heuristics: Early stopping



- Continued training can result in over fitting to training data
 - Track performance on a held-out validation set
 - Apply one of several early-stopping criterion to terminate training when performance on validation set degrades significantly

Additional heuristics: Gradient clipping



- Often the derivative will be too high
 - When the divergence has a steep slope
 - This can result in instability
- **Gradient clipping**: set a ceiling on derivative value
 - if $\partial_w D > \theta$ then $\partial_w D = \theta$*
 - Typical θ value is 5

Additional heuristics: Data Augmentation



CocaColaZero1_1.png



CocaColaZero1_2.png



CocaColaZero1_3.png



CocaColaZero1_4.png



CocaColaZero1_5.png



CocaColaZero1_6.png



CocaColaZero1_7.png



CocaColaZero1_8.png

- Available training data will often be small
- “Extend” it by distorting examples in a variety of ways to generate synthetic labelled examples
 - E.g. rotation, stretching, adding noise, other distortion

Other tricks

- Normalize the input:
 - Normalize entire training data to make it 0 mean, unit variance
 - Equivalent of batch norm on input
- A variety of other tricks are applied
 - Initialization techniques
 - Xavier, Kaiming, SVD, etc.
 - Key point: neurons with identical connections that are identically initialized will never diverge
 - Practice makes man perfect

Setting up a problem

- Obtain training data
 - Use appropriate representation for inputs and outputs
- Choose network architecture
 - More neurons need more data
 - Deep is better, but harder to train
- Choose the appropriate divergence function
 - Choose regularization
- Choose heuristics (batch norm, dropout, etc.)
- Choose optimization algorithm
 - E.g. ADAM
- Perform a grid search for hyper parameters (learning rate, regularization parameter, ...) on held-out data
- Train
 - Evaluate periodically on validation data, for early stopping if required

In closing

- Have outlined the process of training neural networks
 - Some history
 - A variety of algorithms
 - Gradient-descent based techniques
 - Regularization for generalization
 - Algorithms for convergence
 - Heuristics
- Practice makes perfect..